

Información general

Fecha de creación	24 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Principios Avanzados de Programación Orientada a Objetos

Introducción

En el mundo del desarrollo de software, crear un programa que funcione es solo el primer paso, el punto de partida. La verdadera maestría, aquella que distingue a un programador competente de un verdadero arquitecto de software, reside en la capacidad de construir soluciones que no solo resuelven un problema hoy, sino que también son capaces de crecer, adaptarse y evolucionar con el tiempo. Esto es, en esencia, escribir buen software.

Pero ¿qué hace que un software sea bueno? No se trata solo de que no tenga errores. Se trata de su calidad interna: su claridad, su robustez y su mantenibilidad. Aquí es donde entran en juego los principios avanzados de la Programación Orientada a Objetos (POO). Conceptos como la alta cohesión, el bajo acoplamiento y un manejo de excepciones estratégico son los pilares sobre los que se construyen las aplicaciones más exitosas del mercado, desde las aplicaciones bancarias que usamos a diario en Colombia hasta las plataformas de Streaming con millones de usuarios.

Ignorar estos principios es como construir un edificio con cimientos débiles: puede que se mantenga en pie al principio, pero añadir un nuevo piso o reparar una grieta se convierte en una tarea titánica y peligrosa. En este documento, no solo aprenderás a escribir código, sino a diseñarlo con intención y visión a futuro.

Aprenderás a construir software que no solo funcione hoy, sino que sea fácil de evolucionar mañana.

Objetivos

El propósito de este material es llevar tus habilidades de programación más allá de la simple implementación de funcionalidades, para que puedas justificar tus decisiones de diseño con argumentos técnicos sólidos.

Al finalizar el estudio de este documento, estarás en capacidad de:

- Identificar los conceptos de cohesión, acoplamiento y reutilización en fragmentos de código Java existentes, reconociendo "mal código " que afectan la calidad.
- Aplicar de manera efectiva las estructuras de manejo de excepciones (try-catch-finally) para construir programas robustos que anticipen y gestionen fallos de manera controlada.
- Analizar el impacto directo que un diseño con alto acoplamiento y baja cohesión tiene sobre la dificultad para mantener y escalar una aplicación, utilizando métricas simples para evaluar la calidad del código.
- Evaluar y proponer mejoras concretas a un diseño de software existente, refactorizando el código para aplicar los principios avanzados de la POO y justificando el impacto positivo de dichos cambios.

Fundamentos teóricos

Todo software, desde el más simple hasta el más complejo, posee una estructura interna, una especie de esqueleto invisible para el usuario final. La calidad de esta estructura es lo que determina si una aplicación será robusta y adaptable o frágil y costosa de mantener. En este capítulo, exploraremos los principios arquitectónicos que forman los cimientos del software de calidad, no como reglas abstractas, sino como herramientas prácticas para tomar mejores decisiones de diseño.

1.1. Cohesión: El arte de hacer una sola cosa y hacerla bien

La **cohesión** mide qué tan enfocadas están las responsabilidades de un módulo de software, como una clase. Una clase con **alta cohesión** tiene una única misión, bien definida, y todos sus componentes (métodos y atributos) trabajan en conjunto para cumplirla. Por el contrario, una clase con **baja cohesión** intenta hacer demasiadas cosas no relacionadas entre sí, convirtiéndose en un "cajón de sastre" difícil de entender y modificar. Como señalan diversos autores, el objetivo es diseñar componentes que tengan responsabilidades claras y específicas (Barnes & Kölling, 2017). Pensemos en una analogía culinaria:

- **Alta Cohesión:** Un cuchillo de chef. Es una herramienta diseñada con un propósito claro: cortar. Todas sus características (el filo, el peso, el mango) contribuyen a esa única función. Es predecible y eficiente.
- **Baja Cohesión:** Una navaja suiza multifuncional. Aunque útil en ciertas situaciones, si solo queremos cortar verduras, tendremos que lidiar con el peso y la incomodidad de la lupa, las tijeras y el destornillador que no necesitamos. Si una de sus herramientas se daña, afecta a todo el conjunto.

En POO, una clase Factura que solo se encarga de calcular totales, añadir ítems y aplicar impuestos es altamente cohesiva. Si esa misma clase también se encargara de enviar correos electrónicos, conectarse a la base de datos y validar usuarios, su cohesión sería muy baja, volviéndola frágil y confusa (Moreno Pérez, 2015).

1.2. Acoplamiento: Los lazos que unen o ahogan tu código

El **acoplamiento** se refiere al grado de interdependencia entre dos o más módulos de software. Un **bajo acoplamiento** (o acoplamiento débil) significa que las clases son independientes entre sí; un cambio en una clase no debería obligar a realizar cambios en otras. Por otro lado, un **alto acoplamiento** (o acoplamiento fuerte) crea una red de dependencias frágiles, donde modificar un componente puede generar una cascada de errores inesperados en todo el sistema. El ideal es "minimizar el acoplamiento entre objetos para reducir el impacto de los cambios" (Vélez Serrano et al., 2011, p. 54). Usemos una analogía de la vida cotidiana:

- **Bajo Acoplamiento:** La relación entre tu televisor y el enchufe de la pared. El televisor solo necesita saber que existe una fuente de energía estándar. Puedes cambiar de televisor (un LG por un Samsung) sin necesidad de llamar a un electricista para que modifique el enchufe. La interfaz (el tomacorriente) es universal y estable.
- **Alto Acoplamiento:** Un celular antiguo con un cargador de diseño exclusivo. Si pierdes o se daña ese cargador específico, el celular se vuelve inútil. El teléfono depende *directamente* de esa implementación particular del cargador.

En el software, si una clase Reporte necesita acceder directamente a las variables internas de la clase Usuario para funcionar, el acoplamiento es alto. La solución de bajo acoplamiento sería que la clase Usuario ofrezca un método público como `getDatosParaReporte()`, ocultando su funcionamiento interno y proporcionando una interfaz estable (López Goytia, 2015).

A continuación, la **Figura 1** ilustra esta diferencia.

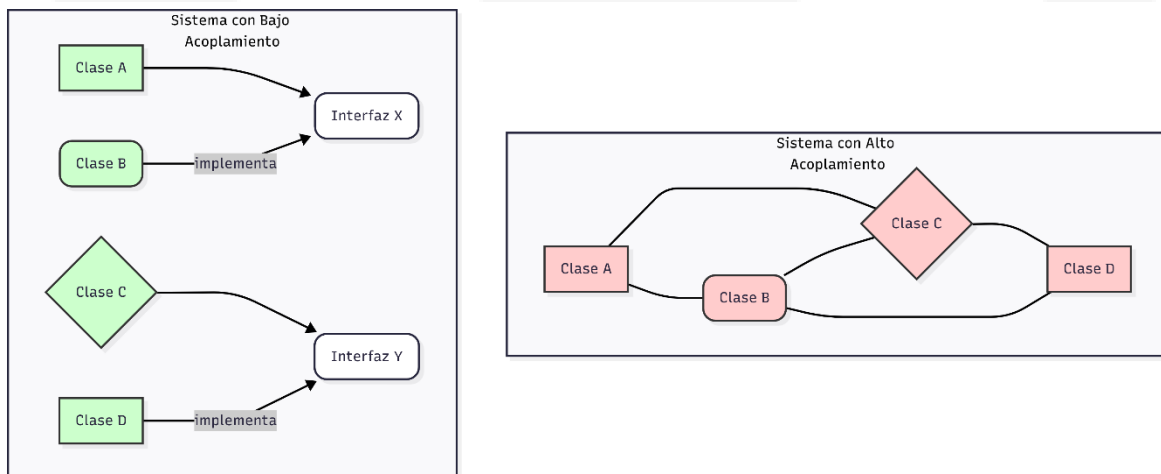


Figura 1. Comparación visual entre acoplamiento alto y bajo.

El diagrama muestra a la derecha cuatro clases enredadas con múltiples conexiones directas, representando alto acoplamiento. A la izquierda, las clases se comunican a través de interfaces, representando bajo acoplamiento.

1.3. Reutilización de Código: No reinventes la rueda, mejórala

Uno de los grandes beneficios de la POO es la capacidad de reutilizar código, lo que ahorra tiempo, reduce errores y aumenta la productividad. La reutilización no es simplemente "copiar y pegar"; es un principio de diseño que se logra principalmente a través de dos mecanismos: la **herencia** y la **composición**. Como lo explica Ceballos Sierra (2018), la herencia permite que una clase (subclase) adquiera los atributos y métodos de otra (superclase), mientras que la composición permite construir objetos complejos a partir de otros más simples ("un objeto *tiene* otro objeto").

La elección entre herencia y composición es una decisión de diseño crucial. La herencia crea un acoplamiento fuerte entre la superclase y la subclase, mientras que la composición favorece la flexibilidad y el bajo acoplamiento. Un buen diseñador sabe cuándo especializar (herencia) y cuándo delegar (composición).

1.4. Manejo de Excepciones: La red de seguridad de tu código

Un programa interactúa constantemente con elementos fuera de su control: la red puede caerse, un archivo puede no existir, o un usuario puede ingresar un dato inválido. Una **excepción** es un evento anómalo que ocurre durante la ejecución de un programa e interrumpe su flujo normal. El **manejo de excepciones** es el mecanismo que nos permite capturar estos eventos y responder a ellos de manera controlada, evitando que la aplicación se cierre abruptamente.

En lugar de dejar que el programa "muera", el código se envuelve en bloques especiales (try-catch en Java) que "intentan" ejecutar una operación riesgosa y "capturan" la excepción si algo sale mal, permitiéndonos mostrar un mensaje amigable al usuario, registrar el error o intentar una acción alternativa (Blasco, 2019). Un buen manejo de excepciones es un rasgo distintivo del software profesional y robusto, pues demuestra que la aplicación fue diseñada pensando no solo en el "camino feliz", sino también en los posibles problemas.

Aplicaciones en la vida cotidiana

Los principios teóricos del software adquieren su verdadero valor cuando los vemos funcionar en el mundo real. Cada vez que usamos una aplicación en nuestro celular, realizamos una compra en línea o interactuamos con un servicio digital, estamos siendo testigos del resultado de un buen (o mal) diseño de software. En este capítulo, vamos a "abrir el capó" de aplicaciones y sistemas que conocemos en Colombia para descubrir cómo la cohesión, el acoplamiento y el manejo de excepciones son los verdaderos motores de una experiencia digital de calidad.

2.1. La Arquitectura Invisible de las Super-Apps Colombianas

Pensemos en una aplicación de alta demanda como **Rappi**. Su éxito no solo radica en su idea de negocio, sino en su capacidad para ser un "todo en uno": restaurantes, supermercados, farmacias, favores, viajes. ¿Cómo es posible gestionar tal complejidad sin que el sistema colapse? La respuesta está en una arquitectura modular, un claro reflejo de la **alta cohesión** y el **bajo acoplamiento**.

- **Cohesión en Acción (Micro-Responsabilidades):** Dentro de Rappi, cada función es un módulo altamente cohesivo. No hay una sola entidad "Rappi" que lo haga todo. En su lugar, tenemos servicios especializados:
 - **Servicio de Autenticación:** Su única misión es verificar quién eres (login, registro, recuperación de contraseña). No sabe ni le interesa si vas a pedir una pizza o un mercado.
 - **Servicio de Catalogo Restaurantes:** Se dedica exclusivamente a gestionar menús, precios y disponibilidad. No tiene idea de quién es el usuario ni cómo va a pagar.
 - **Servicio de Procesamiento Pagos:** Su única responsabilidad es comunicarse con las pasarelas de pago (Wompi, PayU) para procesar una transacción. Recibe un monto y una orden, nada más.
 - **Servicio de geolocalización:** Maneja mapas, calcula rutas y sigue la ubicación del repartidor en tiempo real.

Esta separación (alta cohesión) permite que equipos de desarrolladores trabajen en paralelo en diferentes módulos sin generar conflictos. Además, si el módulo de catálogos falla, los usuarios que ya están en proceso de pago no se ven afectados (Nadal, 2021).

- **Bajo Acoplamiento como Estrategia de Crecimiento:** El bajo acoplamiento es lo que le da a Rappi su "superpoder" de escalar. Cuando decidieron añadir "Rappi Travel", no tuvieron que modificar el motor de pagos ni el sistema de autenticación. Simplemente crearon un nuevo servicio de ReservasViajes y lo "enchufaron" a los servicios ya existentes. El servicio de pagos no necesitó un `if (esViaje) { cobrarAsi; }`

else if (esRestaurante) { cobrarAsa; }. Simplemente recibió una orden de cobro, indiferente a su origen. Esta independencia es la clave para una evolución rápida y segura del producto.

2.1.1. Las APIs: El Contrato del Bajo Acoplamiento

Una manifestación muy concreta del bajo acoplamiento son las **APIs (Interfaces de Programación de Aplicaciones)**. Pensemos en cualquier e-commerce en Colombia que permita pagar a través de **PSE**. El e-commerce no tiene acceso directo a la base de datos de tu banco. Sería un caos de seguridad y una pesadilla de acoplamiento.

En su lugar, PSE ofrece una API, que es un "contrato" o un conjunto de reglas de comunicación. El e-commerce simplemente le dice a la API: "El usuario X quiere pagar Y pesos desde el banco Z". La API de PSE se encarga del resto de forma segura y oculta la complejidad. El e-commerce y el banco están **débilmente acoplados** a través de este intermediario. Si el banco cambia su sistema interno mañana, mientras siga respetando el "contrato" de la API, el e-commerce no se verá afectado en lo más mínimo.

2.2. Algo salió mal: El Manejo de Excepciones que Salva el Negocio

Todos hemos visto mensajes de error, pero un manejo de excepciones profesional va más allá de evitar que la aplicación se cierre. Es una herramienta crucial para proteger la **integridad de los datos y la lógica del negocio**.

- **Excepciones Técnicas (Protegiendo al Usuario):**
 1. **Fallo de Red:** Estás a punto de confirmar tu compra en Mercado Libre y se cae la señal de internet. Una aplicación robusta captura la excepción de conexión (ConnectException) y hace dos cosas: primero, te informa ("No tienes conexión a internet") y segundo, y más importante, **revierte la transacción localmente**, asegurando que no quedes en un estado inconsistente donde crees que pagaste, pero el servidor nunca recibió la confirmación.
 2. **Archivo Inexistente:** Intentas descargar un extracto bancario en PDF desde la aplicación de Davivienda. El servidor, por alguna razón, no encuentra el archivo. En lugar de un error genérico, la aplicación captura una FileNotFoundException y presenta un mensaje específico: "El extracto solicitado no está disponible en este momento. Por favor, inténtelo más tarde o contacte a soporte".
- **Excepciones de Negocio (Protegiendo las Reglas):** Las excepciones también son una forma elegante de hacer cumplir las reglas del negocio. Imaginemos el código de un cajero automático:

```
public void retirarDinero(double monto) throws SaldoInsuficienteException {  
    if (monto > this.saldoDisponible) {  
        throw new SaldoInsuficienteException("No es posible retirar " + monto + ". Saldo  
disponible: " + this.saldoDisponible);  
    }  
    this.saldoDisponible -= monto;  
    // dispensar billetes  
}
```

Aquí, `SaldoInsuficienteException` no es un error técnico. Es una excepción **creada a medida** para representar un evento que, aunque posible, viola una regla fundamental del negocio. Como lo describe Vegas (2022) en sus trabajos sobre programación avanzada, el manejo de excepciones permite separar la lógica de negocio principal del manejo de casos anómalos, haciendo el código mucho más limpio y legible.

2.3. La Deuda Técnica: Historia de una Startup

Imaginemos a "TuEnvíoYa", una startup de logística fundada en Bogotá. En sus inicios, dos programadores construyeron la plataforma rápidamente para salir al mercado. Crearon una única y gigantesca clase llamada `GestionEnvio` que hacía de todo: calculaba tarifas, asignaba mensajeros, procesaba pagos, enviaba notificaciones por correo y actualizaba la base de datos. El código era un ejemplo de **baja cohesión y alto acoplamiento**.

A medida que el negocio crecía, los desarrolladores empezaron a sufrir los siguientes "dolores" o **"code smells" (malos olores de código)**:

- **El Efecto Dominó (Fragilidad):** Un día, el gobierno cambió el cálculo del IVA. El desarrollador modificó las 3 líneas de código correspondientes en el método `calcularTarifa()`. Al día siguiente, el sistema de notificaciones por correo electrónico dejó de funcionar. Nadie entendía por qué, hasta que descubrieron que una variable usada para formatear los correos se reutilizaba sin cuidado dentro del cálculo de tarifas. El sistema estaba tan acoplado que un cambio trivial generaba fallos en cascada.
- **Código Inmortal (Rigidez):** Querían ofrecer un nuevo servicio: "Envío Programado". Sin embargo, la lógica para asignar un mensajero estaba tan mezclada con la de los pagos y las notificaciones que era imposible reutilizarla. La única opción era copiar y pegar cientos de líneas de código, duplicando el problema.
- **Arqueología de Código (Complejidad):** Un nuevo desarrollador tardó una semana entera en entender cómo se procesaba un solo envío. La clase `GestionEnvio` tenía más de 5,000 líneas y docenas de variables globales. Modificarla era como

desactivar una bomba, había que seguir cada rastro de cada variable para asegurarse de que el cambio no explotara en otro lugar.

La refactorización no fue un lujo, sino una necesidad para la supervivencia del negocio. La lección fue clara: la "velocidad" inicial de un mal diseño es un espejismo que se paga con una parálisis casi total en el futuro.



Casos de estudio

La teoría y los ejemplos nos han dado el mapa, pero es en la resolución de problemas complejos donde se forja la verdadera pericia. En este capítulo, nos sumergiremos en un escenario profesional detallado que ilustra cómo la aplicación rigurosa de los principios de POO no es solo una buena práctica técnica, sino una necesidad estratégica para el éxito de un negocio.

3.1. El Contexto: "AgroTech-CUN" y su Crisis de Crecimiento

"AgroTech-CUN" es una startup ficticia con sede en Tocancipá que ofrece una plataforma de software para que los agricultores de la sabana de Bogotá gestionen sus cultivos. Su producto estrella es un "Módulo de Reportes de Cosecha" que analiza datos de siembra, clima y suelo para predecir el rendimiento y la rentabilidad.

Durante el primer año, la plataforma fue un éxito. Sin embargo, a medida que la base de clientes creció y solicitaron nuevas funcionalidades, el equipo de desarrollo se encontró con un muro.

3.2. El Desafío: Un Módulo "Monolítico" al Borde del Colapso

El problema residía en una única y monstruosa clase llamada `ReporteManager.java`. Esta clase de 8.000 líneas de código era responsable de todo el proceso, evidenciando graves fallas de diseño:

1. **Baja Cohesión Crítica:** La clase se conectaba directamente a la base de datos, ejecutaba complejos algoritmos de predicción, formateaba los resultados en PDF y Excel, y enviaba los reportes por correo electrónico. Era el ejemplo perfecto de una "God Class" o clase todopoderosa.
2. **Alto Acoplamiento Extremo:** Estaba fuertemente acoplada a la librería iText para PDF, a la librería Apache POI para Excel, al driver de la base de datos PostgreSQL y al servidor de correo. Cualquier actualización en una de estas librerías externas requería una revisión completa y riesgosa de toda la clase.
3. **Manejo de Excepciones Inexistente:** No había bloques try-catch. Si la conexión a la base de datos fallaba por un instante, todo el proceso de generación de reportes se abortaba y el programa se cerraba con un error críptico, dejando al agricultor sin su informe.

El impacto en el negocio era devastador:

- **Tiempo de Desarrollo Lento:** Añadir un nuevo campo al reporte (ej. "nivel de humedad") tomaba tres semanas de desarrollo y pruebas.
- **Inflexibilidad:** Los clientes pedían los reportes en formato CSV, pero implementarlo se consideraba "imposible" sin reescribir todo el módulo.

- **Pérdida de Clientes:** La inestabilidad y lentitud del módulo estaban generando una alta tasa de cancelación del servicio.

3.3. La Solución Aplicada: Refactorización Estratégica Basada en Principios POO

Un arquitecto de software fue contratado para liderar el rescate. En lugar de "apagar el incendio" con parches, propuso una refactorización completa, guiada por los principios de diseño. El proceso de "diseñar y programar" debía ser replanteado desde sus bases (Vélez Serrano et al., 2011).

Paso 1: Descomponer por Responsabilidades (Aumentar Cohesión)

La clase `ReporteManager.java` fue desmantelada. Se crearon varias clases nuevas, cada una con una única y clara responsabilidad (alta cohesión):

- `RepositorioCosecha.java`: Su única misión era interactuar con la base de datos para obtener los datos crudos.
- `MotorDeCalculo.java`: Contenía la lógica de negocio pura para procesar los datos y generar las predicciones. No sabía nada de bases de datos ni de formatos de archivo.
- `ServicioDeNotificacion.java`: Encargada exclusivamente de enviar correos electrónicos.

Paso 2: Invertir las Dependencias (Disminuir Acoplamiento)

Para el formato, se aplicó el **Principio de Inversión de Dependencias**. Se creó una interfaz llamada `IFormateadorReporte`:

```
// Interfaz que define el "contrato" para cualquier formateador
public interface IFormateadorReporte {
    File formatear(ReporteData datos);
}
```

Luego, se crearon implementaciones concretas de esta interfaz:

- `FormateadorPDF.java` **implements** `IFormateadorReporte`
- `FormateadorExcel.java` **implements** `IFormateadorReporte`

El nuevo orquestador del proceso ya no dependía de las clases concretas, sino de la abstracción. Esto significa que para añadir el formato CSV, los desarrolladores solo tuvieron que crear una nueva clase `FormateadorCSV.java` que implementara la interfaz, sin tocar una sola línea del código existente. Esta técnica es fundamental para crear software flexible y extensible (Ceballos Sierra, 2018).

Paso 3: Implementar una Estrategia de Excepciones

Se introdujo un manejo de excepciones robusto. El RepositorioCosecha ahora podía lanzar una `DataAccessLayerException` si la base de datos fallaba, y cada formateador podía lanzar una `FormattingException`. El orquestador principal envolvía las llamadas en bloques `try-catch`, permitiendo un control granular de los errores: si fallaba la generación del PDF, el sistema podía intentar generar el de Excel como alternativa o, como mínimo, mostrar un mensaje claro al usuario sin abortar la aplicación.

3.4. El Impacto: Medición del Éxito Post-Refactorización

Los resultados de la refactorización se midieron con datos concretos, demostrando el valor de la inversión en calidad de software:

- **Métricas de Negocio:**
 - **Time-to-Market:** El tiempo para implementar un nuevo formato de reporte (como CSV) se redujo de "imposible" a **2 días**.
 - **Satisfacción del Cliente:** Las quejas por errores en el módulo de reportes disminuyeron en un **95%** en los siguientes tres meses.
 - **Retención:** La tasa de cancelación de clientes se redujo en un **40%**.
- **Métricas Técnicas:**
 - **Mantenibilidad:** El índice de complejidad ciclomática del código se redujo en un **70%**.
 - **Calidad:** La cobertura de pruebas unitarias pasó del 5% al **90%**, permitiendo hacer cambios con confianza.
 - **Rendimiento:** La generación de reportes se aceleró en un **30%** gracias a la optimización de las consultas a la base de datos, ahora aisladas en su propia clase.

Este caso demuestra que la cohesión, el acoplamiento y el manejo de excepciones no son conceptos académicos, sino herramientas vitales que tienen un impacto directo y medible en la agilidad, la calidad y la rentabilidad de un proyecto de software.

Herramientas y técnicas

Saber la teoría es importante, pero la verdadera maestría en el desarrollo de software se demuestra con el teclado. Este capítulo es eminentemente práctico. Te guiará a través de ejercicios concretos en Java, utilizando el IDE NetBeans, para que puedas implementar los principios de cohesión, acoplamiento y manejo de excepciones. Pasaremos de ser observadores a ser constructores de software de calidad.

4.1. Taller Práctico con Java y NetBeans

Abre tu entorno de desarrollo NetBeans y crea un nuevo proyecto de Java. Vamos a trabajar en dos ejercicios diseñados para solidificar los conceptos clave.

Ejercicio 1: Implementando una Red de Seguridad con try-catch-finally

Objetivo: Aprender a manejar excepciones de forma controlada para evitar que el programa se cierre inesperadamente al interactuar con archivos.

Escenario: Vamos a escribir un programa que intenta leer la primera línea de un archivo de texto llamado config.txt. Este archivo podría no existir, lo que provocaría un error si no lo manejamos adecuadamente.

Instrucciones:

1. Crea una nueva clase LectorArchivos.java.
2. Copia y pega el siguiente código en tu clase:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LectorArchivos {

    public void leerConfiguracion() {
        BufferedReader reader = null; // Declaramos el lector fuera del try
        try {
            // Paso 1: Intentamos abrir el archivo. Esta línea puede lanzar una IOException.
            System.out.println("Intentando leer el archivo 'config.txt'...");
            reader = new BufferedReader(new FileReader("config.txt"));
            String primeraLinea = reader.readLine();

            if (primeraLinea != null) {
                System.out.println("Éxito. La primera línea es: " + primeraLinea);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            if (reader != null) {
                try {
                    reader.close();
                } catch (IOException e) {
                    System.out.println("Error al cerrar el lector: " + e.getMessage());
                }
            }
        }
    }
}
```

```
} else {  
    System.out.println("El archivo está vacío.");  
}  
  
} catch (IOException e) {  
    // Paso 2: Si ocurre un error (ej. el archivo no existe), se captura aquí.  
    System.err.println("Error: No se pudo encontrar o leer el archivo 'config.txt'.");  
    System.err.println("Detalle técnico: " + e.getMessage());  
  
} finally {  
    // Paso 3: Este bloque se ejecuta SIEMPRE, haya o no un error.  
    // Es el lugar ideal para cerrar recursos y evitar fugas de memoria.  
    System.out.println("Bloque 'finally': intentando cerrar el archivo.");  
    if (reader != null) {  
        try {  
            reader.close();  
        } catch (IOException e) {  
            System.err.println("Error al intentar cerrar el lector de archivos.");  
        }  
    }  
}  
}
```

Para probarlo: Ejecuta el método leerConfiguracion.

Primero, sin crear el archivo config.txt en la raíz de tu proyecto; verás cómo el bloque catch se activa. Luego, crea el archivo, escribe algo en él y vuelve a ejecutarlo; verás el flujo exitoso. Este ejercicio demuestra cómo un programa robusto se anticipa a los problemas (Flórez Fernández, 2012).

Ejercicio 2: Refactorizando una "God Class"

Objetivo: Aplicar los principios de alta cohesión y bajo acoplamiento para mejorar un diseño de software deficiente.

Escenario: Se te entrega la siguiente clase GestionTodo.java, que gestiona usuarios, tareas y las guarda en un archivo. Es un ejemplo clásico de baja cohesión.

```
// CÓDIGO CON "MALOS OLORES" - ANTES DE REFACTORIZAR  
import java.io.FileWriter;  
import java.io.IOException;  
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class GestionTodo {  
    private List<String> usuarios = new ArrayList<>();  
    private List<String> tareas = new ArrayList<>();  
  
    public void agregarUsuario(String nombre) {  
        usuarios.add(nombre);  
        System.out.println("Usuario agregado: " + nombre);  
    }  
  
    public void agregarTarea(String descripcion) {  
        tareas.add(descripcion);  
        System.out.println("Tarea agregada: " + descripcion);  
    }  
  
    // Esta clase también sabe cómo guardar en disco, ¡rompiendo la cohesión!  
    public void guardarEnArchivo() {  
        try (FileWriter writer = new FileWriter("datos.txt")) {  
            writer.write("--- USUARIOS ---\n");  
            for (String u : usuarios) {  
                writer.write(u + "\n");  
            }  
            writer.write("--- TAREAS ---\n");  
            for (String t : tareas) {  
                writer.write(t + "\n");  
            }  
            System.out.println("Datos guardados en archivo.");  
        } catch (IOException e) {  
            System.err.println("Error al guardar el archivo: " + e.getMessage());  
        }  
    }  
}
```

Tu Misión: Refactoriza el código anterior dividiendo la clase GestionTodo en **tres clases más cohesivas y débilmente acopladas**.

1. GestionUsuario.java: Responsable únicamente de la lógica de usuarios.
2. GestionTarea.java: Responsable únicamente de la lógica de tareas.
3. PersistenciaDatos.java: Responsable únicamente de guardar los datos en un archivo.

Justificación: Al separar las responsabilidades, logras que el sistema sea más fácil de mantener y extender. Mañana, si necesitas cambiar el formato de guardado de archivo a una base de datos, solo tendrás que modificar `PersistenciaDatos.java`, sin tocar la lógica de usuarios o tareas. Este es el poder de un buen diseño orientado a objetos (Barnes & Kölling, 2017).

4.2. Visualizando el Diseño: Más Allá del Código

Aunque el código es la implementación final, las herramientas de modelado nos ayudan a pensar y discutir el diseño antes de escribir una sola línea. Herramientas como **UML (Lenguaje Unificado de Modelado)** permiten crear diagramas de clases que visualizan las relaciones, dependencias y responsabilidades en un sistema.

Antes de realizar la refactorización del Ejercicio 2, por ejemplo, un buen profesional dibujaría un diagrama de clases de la solución propuesta. Esto ayuda a identificar problemas de acoplamiento y a validar la cohesión de cada componente. Como afirma Vélez Serrano et al. (2011), usar UML y Java en conjunto es una práctica que fortalece enormemente el proceso de diseño y programación.

Incluso sin software especializado, puedes usar herramientas en línea como draw.io o simplemente papel y lápiz para esbozar tus clases, sus atributos y métodos, y las flechas que indican sus relaciones. Esta práctica de visualización es invaluable para pasar de programador a arquitecto de software.

Conclusión

A lo largo de este documento, hemos viajado desde los fundamentos teóricos del buen diseño de software hasta su aplicación en el mundo real y profesional. Hemos desglosado conceptos que a primera vista pueden parecer abstractos —**alta cohesión, bajo acoplamiento y manejo de excepciones**— para descubrir que son, en realidad, las herramientas más prácticas y poderosas en el arsenal de un desarrollador de software.

La lección más importante no es memorizar una definición, sino comprender la filosofía que subyace a estos principios: construir software no es solo un acto técnico, sino un ejercicio de **previsión, organización y profesionalismo**. Escribir código que "funciona" es el requisito mínimo; el verdadero objetivo es crear soluciones que sean mantenibles, escalables y robustas. Como vimos en el caso de "AgroTech-CUN", la deuda técnica generada por un mal diseño no es un problema abstracto, sino un lastre que frena la innovación y pone en riesgo el negocio.

La Programación Orientada a Objetos, en su nivel avanzado, te invita a pensar más como un arquitecto y menos como un simple constructor. Se trata de preguntarte constantemente: ¿Cuál es la responsabilidad única de esta clase? ¿Qué tan dependiente es este módulo de los demás? ¿Qué podría salir mal aquí y cómo puedo manejarlo de forma elegante?

Al cerrar este capítulo, te invito a llevar esta mentalidad a cada línea de código que escribas. No veas estos principios como reglas restrictivas, sino como un camino hacia la maestría en tu oficio. El software que construyas hoy será el legado con el que otros desarrolladores (o incluso tu "yo" del futuro) tendrán que trabajar mañana. Asegúrate de que sea un legado del que te sientas orgulloso.

Para continuar tu reflexión, te dejo con estas preguntas abiertas:

1. De los "malos olores de código" que vimos (como la "God Class"), ¿cuál crees que es el más común en los proyectos en los que has participado y por qué crees que ocurre con tanta frecuencia?
2. Más allá de try-catch, ¿qué otras estrategias o prácticas podrías investigar para crear aplicaciones aún más resilientes y a prueba de fallos en tus futuros proyectos?

Referencias.

Aristizábal Martínez, D. A., & Quiceno Metaute, S. M. (2023). *Lógica de programación básica orientada a objetos con ejercicios resueltos* (1ª ed.). Instituto Tecnológico Metropolitano.

Barnes, D. J., & Kölling, M. (2017). *Programación orientada a objetos con Java usando BlueJ*. Pearson Educación.

Blasco, F. (2019). *Programación Orientada a Objetos en JAVA*. RA-MA Editorial.

Ceballos Sierra, F. J. (2018). *Programación orientada a objetos con C++* (5ª ed.). RA-MA Editorial.

Flórez Fernández, H. A. (2012). *Programación orientada a objetos usando java*. Ecoe Ediciones.

López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*. Grupo Editorial Patria.

Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial.

Nadal, M. (2021). *Curso de programación Java* (1ª ed.). Difusora Larousse - Anaya Multimedia.

Vegas, J. (2022). *Java 17 Programación Avanzada*. RA-MA Editorial.

Vélez Serrano, J. F., Peña, A., & Giraldo, W. (2011). *Diseñar y programar, todo es empezar: Una introducción a la programación orientada a objetos usando UML y Java*. Dykinson.

Información general

Fecha de creación	24 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Interfaces y Clases Abstractas

Introducción

Imagina que eres el arquitecto de una aplicación de *streaming* musical como Spotify o de una plataforma de comercio electrónico como Mercado Libre. Un día, el equipo de negocio decide que es crucial integrar un nuevo método de pago —digamos, Nequi o DaviPlata— para capturar una mayor porción del mercado colombiano. En un sistema rígidamente construido, esta tarea podría ser una pesadilla: habría que modificar módulos centrales, arriesgándose a introducir errores en funcionalidades que antes operaban perfectamente. Este fenómeno, conocido como "**alto acoplamiento**", es uno de los mayores enemigos del software sostenible, pues convierte cada nueva funcionalidad en un proyecto costoso, lento y propenso a fallos (López Goytia, 2015).

Aquí es donde la Programación Orientada a Objetos nos ofrece herramientas de diseño de una elegancia y potencia excepcionales: las **interfaces** y las **clases abstractas**. Lejos de ser meras construcciones sintácticas, son los pilares filosóficos que nos permiten construir software que no se rompe, sino que evoluciona.

Pensemos en una **interfaz** como el estándar de un puerto USB. A tu computador no le interesa si conectas un mouse, una cámara o un teclado; no necesita conocer los detalles internos de cada dispositivo. Solo le importa que el dispositivo "cumpla el contrato" del estándar USB para poder comunicarse. En software, una interfaz `MetodoDePago` es ese contrato: define qué acciones debe ser capaz de realizar cualquier método de pago (`realizarCobro()`, `validarTransaccion()`), pero no cómo lo hace. Esto nos permite agregar Nequi, PSE o cualquier otro sistema futuro simplemente creando una nueva clase que "firme" ese contrato, sin tocar el núcleo de la aplicación.

Por otro lado, una **clase abstracta** es como el plano maestro de una franquicia, por ejemplo, Tostao'. Todas las tiendas Tostao' comparten una estructura y lógica comunes: tienen una marca, un método para preparar el café y una forma de registrar las ventas. Esos elementos compartidos residen en la clase abstracta `Tienda`. Sin embargo, cada tienda concreta debe implementar ciertos detalles: `ubicacionExacta()`, `gestionDelInventarioLocal()` o `contratacionDePersonal()`. La clase abstracta provee el esqueleto y el comportamiento común, pero delega la implementación específica a sus subclases.

Dominar estas dos herramientas marca la transición de ser un simple programador a convertirse en un **diseñador de software**. Permite aplicar principios de diseño robusto como **SOLID**, y nos capacita para construir sistemas que acogen el cambio en lugar de temerle. Por ello, dominar las interfaces y clases abstractas no es solo aprender a escribir código, es aprender a diseñar soluciones que perduren en el tiempo.

Objetivos

Al finalizar el estudio de este documento, el estudiante de Ingeniería de sistemas de la CUN estará en capacidad de diseñar, implementar y evaluar soluciones de software flexibles y mantenibles en Java, utilizando interfaces y clases abstractas para desacoplar componentes, gestionar la variabilidad y promover la reutilización de código, aplicando estos conceptos en la resolución de problemas contextualizados al entorno profesional.

Objetivos Específicos

Para alcanzar el objetivo general, el estudiante deberá ser capaz de:

1. **Diferenciar:** Explicar las diferencias semánticas y sintácticas entre una interfaz y una clase abstracta en Java, identificando a través de ejemplos cotidianos los escenarios de aplicación más adecuados para cada una.
2. **Implementar:** Construir clases que implementen interfaces y hereden de clases abstractas en el IDE NetBeans, resolviendo problemas prácticos que demuestren cómo estos mecanismos reducen el acoplamiento y mejoran la reutilización del código.
3. **Diseñar:** Diseñar pequeñas arquitecturas de software basadas en el principio de "programar hacia una interfaz, no hacia una implementación". Argumentar y justificar la elección de una u otra forma de abstracción para crear sistemas extensibles, como los que demandan los *frameworks* modernos o los patrones de diseño.

Fundamentos teóricos

La Programación Orientada a Objetos es, en esencia, el arte de modelar el mundo real —o sistemas complejos— a través de objetos que interactúan entre sí. Sin embargo, para que este modelo sea útil y manejable, no podemos quedarnos en los detalles de cada objeto individual. Necesitamos una forma de gestionar las ideas y los conceptos generales. Aquí es donde entra en juego la **abstracción**, uno de los pilares fundamentales de la POO (Joyanes Aguilar, 2020). La abstracción consiste en enfocarse en las características esenciales de un objeto, ignorando los detalles irrelevantes. Las clases abstractas y las interfaces son las herramientas que materializan este principio en código.

1.1. Clases Abstractas: Moldes con Comportamiento Común

Una clase abstracta es una plantilla o un "molde semi-construido". Se utiliza para definir un comportamiento y atributos comunes que compartirán varias subclases, pero que por sí misma es demasiado genérica para existir como un objeto concreto. Pensemos en el concepto de **"Vehículo"** en el contexto de una aplicación de logística en Colombia. Sabemos que todo vehículo tiene una placa, una capacidadDeCarga y debe poder arrancar() y detenerse(). Sin embargo, nadie compra un "vehículo genérico"; uno compra un "Carro", una "Moto" o un "Camión".

Una clase abstracta Vehiculo nos permite definir ese esqueleto común.

Sintaxis en Java:

```
// Se usa la palabra clave "abstract"
public abstract class Vehiculo {
    protected String placa;
    protected double capacidadDeCarga;

    // Constructor para inicializar atributos comunes
    public Vehiculo(String placa, double capacidadDeCarga) {
        this.placa = placa;
        this.capacidadDeCarga = capacidadDeCarga;
    }

    // Método concreto: comportamiento común ya implementado
    public void arrancar() {
        System.out.println("El vehículo con placa " + placa + " ha arrancado.");
    }

    public void detenerse() {
        System.out.println("El vehículo se ha detenido.");
    }
}
```

}

```
// Método abstracto: comportamiento que CADA subclase DEBE implementar
public abstract double calcularSOAT();
```

```
}
```

La característica principal es que puede combinar **métodos concretos** (como arrancar(), que es igual para todos) y **métodos abstractos** (como calcularSOAT(), cuyo cálculo depende de si es una moto o un camión). Debido a que tiene partes sin construir (los métodos abstractos), **no se puede crear una instancia directa** de una clase abstracta (Flórez Fernández, 2012). Es decir, new Vehiculo() generaría un error.

1.2. Interfaces: Contratos de Comportamiento

Si una clase abstracta es un molde, una **interfaz** es un **contrato 100% abstracto**. No define cómo se hace algo, ni siquiera parcialmente; simplemente establece *qué* se debe poder hacer. Una interfaz es una colección de métodos abstractos y constantes que obliga a cualquier clase que la "firme" (implemente) a proporcionar una implementación para cada uno de sus métodos (Barnes & Kölling, 2017).

Siguiendo con analogías locales, pensemos en la interfaz de pagos de TransMilenio. La tarjeta "TuLlave" es una interfaz. El sistema de torniquetes no necesita saber si la tarjeta fue recargada en una taquilla, una máquina o por internet. Solo le exige a la tarjeta que cumpla un contrato: debe ser capaz de presentarSaldo() y debitarPasaje().

Sintaxis en Java:

```
// Se usa la palabra clave "interface"
public interface RastreableGPS {
    // Por defecto, todos los métodos son public y abstract
    void obtenerCoordenadas();
    void enviarAlertaPosicion();
}

// Una clase puede implementar una o más interfaces
public class Camion extends Vehiculo implements RastreableGPS {
    // ... constructor y otros métodos ...

    @Override
    public double calcularSOAT() {
        // Lógica específica para camiones
        return 500000 + (this.capacidadDeCarga * 100);
    }
}
```

```

// Se obliga a implementar los métodos del "contrato" RastreableGPS
@Override
public void obtenerCoordenadas() {
    System.out.println("Obteniendo coordenadas del camión " + this.placa);
}

```

```

@Override
public void enviarAlertaPosicion() {
    System.out.println("Alerta de posición enviada para el camión.");
}
}

```

La gran ventaja de las interfaces es que una clase puede implementar **múltiples interfaces**, a diferencia de la herencia de clases, que en Java es simple (una clase solo puede heredar de una superclase) (Vélez Serrano et al., 2011). Nuestro Camion podría implementar RastreableGPS, RequiereRefrigeracion y HabilitadoParaCargaPeligrosa, adquiriendo así múltiples "contratos" de comportamiento.

1.3. Diferencias Clave: ¿Clase Abstracta o Interfaz?

La decisión de cuándo usar una u otra es una de las habilidades clave en el diseño de software. Aunque ambas promueven la abstracción, sirven a propósitos distintos. La siguiente figura resume sus diferencias y similitudes.

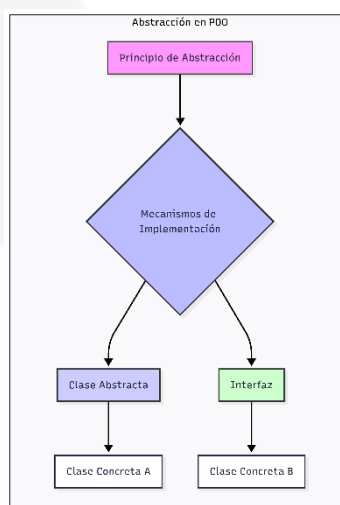


Figura 1 Diagrama de Venn comparando Clases Abstractas e Interfaces

En la Figura 1 se observa un diagrama de Venn que muestra las características exclusivas de las clases abstractas (atributos de estado, métodos concretos, herencia simple) y de

las interfaces (implementación múltiple, métodos default/static), y sus características compartidas (métodos abstractos, no se pueden instanciar).

En resumen (Moreno Pérez, 2015):

- Usa una **clase abstracta** cuando:
 - Quieres compartir código (atributos y métodos concretos) entre varias clases estrechamente relacionadas.
 - Esperas que las clases que heredan tengan muchos métodos o campos en común.
 - Necesitas declarar campos no estáticos o no finales.
- Usa una **interfaz** cuando:
 - Esperas que clases no relacionadas implementen el mismo comportamiento (ej. RastreableGPS puede ser implementado por Camion, Celular, Contenedor).
 - Quieres especificar el comportamiento de un tipo de dato, pero no te preocupa quién lo implementa.
 - Quieres aprovechar la herencia múltiple de tipos.

1.4. Interfaces Funcionales: El Futuro es Ahora

Con la evolución de Java (versión 8 en adelante), las interfaces recibieron una actualización muy potente: las **interfaces funcionales**. Son interfaces que definen un único método abstracto. Su poder reside en que pueden ser representadas por **expresiones lambda**, simplificando enormemente el código para tareas como manejar eventos o procesar colecciones de datos (Blasco, 2019). Un ejemplo claro es la interfaz Runnable, que solo tiene el método run().

Esta característica es fundamental en la programación moderna y la base de la programación reactiva y el procesamiento de *streams* en Java.

Aplicaciones en la vida cotidiana

Una vez comprendidos los fundamentos teóricos, es crucial conectar estos conceptos con el mundo tangible. Las interfaces y clases abstractas son el andamiaje invisible detrás de muchas de las aplicaciones que facilitan nuestra vida diaria. Su verdadero poder no reside en su sintaxis, sino en su capacidad para organizar la complejidad, permitir la evolución y crear sistemas robustos. A continuación, exploraremos tres escenarios prácticos y cercanos que ilustran su aplicación.

2.1. Un Ecosistema de Pagos en Colombia: Flexibilidad para Transacciones

Imaginemos el sistema de *checkout* de un comercio electrónico colombiano como Falabella o Éxito. Los clientes esperan poder pagar con una multitud de opciones: Tarjeta de Crédito (Visa, Mastercard), PSE, Nequi, DaviPlata, o incluso en efectivo a través de Efecty. Desde el punto de vista del programador, cada uno de estos métodos tiene un proceso de validación y confirmación completamente diferente.

¿Cómo gestionar esta diversidad sin crear un código caótico y difícil de mantener? La respuesta es una **interfaz**. Podemos diseñar un contrato llamado `MetodoDePago`.

```
public interface MetodoDePago {  
    // Cada método de pago debe poder iniciar un proceso de cobro  
    boolean procesarPago(double monto);  
  
    // Cada método debe poder verificar el estado de la transacción  
    String obtenerEstado();  
}
```

Con este contrato, podemos crear clases concretas para cada opción de pago. Cada clase "firma" el contrato y se compromete a implementar los métodos `procesarPago()` y `obtenerEstado()` a su manera.

```
public class PagoPSE implements MetodoDePago {  
    @Override  
    public boolean procesarPago(double monto) {  
        // Lógica específica para conectar con la pasarela de PSE  
        System.out.println("Redirigiendo al portal de bancos para pago con PSE por " + monto);  
        // ... Lógica de comunicación con ACH ...  
        return true;  
    }  
    @Override  
    public String obtenerEstado() {  
        return "PSE: Transacción Aprobada.";  
    }  
}
```

```
}  
}
```

```
public class PagoNequi implements MetodoDePago {  
    @Override  
    public boolean procesarPago(double monto) {  
        // Lógica para generar un QR o enviar una notificación push a Nequi  
        System.out.println("Generando solicitud de pago a Nequi por " + monto);  
        // ... Lógica de API de Nequi ...  
        return true;  
    }  
    @Override  
    public String obtenerEstado() {  
        return "Nequi: Esperando confirmación del usuario.";  
    }  
}
```

El módulo principal del carrito de compras ahora no necesita saber los detalles de cada sistema. Simplemente opera con objetos del tipo `MetodoDePago`. Esta abstracción, lograda mediante la interfaz, permite que el sistema sea increíblemente **extensible**. Si el día de mañana surge un nuevo método de pago digital, solo necesitamos crear una nueva clase que implemente la interfaz, sin modificar una sola línea del código del *checkout* (Joyanes Aguilar, 2020).

2.2. "Plug-and-Play": El Poder de los Plugins en NetBeans

Muchos hemos usado herramientas como el IDE NetBeans o editores como Visual Studio Code. Una de sus características más potentes es la capacidad de instalar extensiones o *plugins* desarrollados por terceros. ¿Cómo es posible que un programa pueda ejecutar código que no existía cuando fue compilado? Nuevamente, gracias a las interfaces. El núcleo de NetBeans define una serie de interfaces para diferentes tipos de extensiones. Por ejemplo, podría existir una interfaz `CodeFormatter` para herramientas que formatean código o `SyntaxHighlighter` para las que colorean la sintaxis.

```
public interface CodeFormatter {  
    String format(String code);  
}
```

Un desarrollador externo que quiera crear un plugin para formatear código Python, simplemente debe crear una clase que implemente esta interfaz.

```
public class PythonFormatter implements CodeFormatter {
```

@Override

```
public String format(String code) {  
    // Lógica para aplicar las reglas de estilo de Python (PEP 8)  
    String formattedCode = ... // Proceso de formateo  
    return formattedCode;  
}  
}
```

Cuando instalamos el plugin, NetBeans detecta la nueva clase, comprueba que "firma" el contrato `CodeFormatter` y la hace disponible en su menú de herramientas. El núcleo del IDE nunca tuvo que saber sobre la existencia de Python; solo sabe cómo trabajar con cualquier objeto que cumpla el contrato `CodeFormatter`. Este es un ejemplo perfecto del **Principio de Inversión de Dependencia**, uno de los pilares del diseño de software SOLID (López Goytia, 2015), donde los módulos de alto nivel no dependen de los de bajo nivel, sino que ambos dependen de abstracciones.

2.3. Héroes y Villanos en un Videojuego: Definiendo Comportamientos Comunes

Ahora consideremos el desarrollo de un videojuego. Tendremos muchos tipos de personajes: guerreros, magos, arqueros, ogros, dragones. Todos estos personajes comparten características y acciones fundamentales: tienen `puntosDeVida`, `nivelDeEnergia`, y todos pueden `moverse()`, `recibirDaño()` y `atacar()`.

Sería muy repetitivo implementar esta lógica en cada clase de personaje. Aquí es donde una **clase abstracta** brilla. Podemos crear una clase `Personaje` que contenga todos los atributos y métodos comunes.

```
public abstract class Personaje {  
    protected int puntosDeVida;  
    protected int nivelDeEnergia;  
  
    public Personaje(int vidaInicial, int energiaInicial) {  
        this.puntosDeVida = vidaInicial;  
        this.nivelDeEnergia = energiaInicial;  
    }  
  
    public void recibirDaño(int cantidad) {  
        this.puntosDeVida -= cantidad;  
        if (this.puntosDeVida <= 0) {  
            System.out.println("El personaje ha sido derrotado.");  
        }  
    }  
}
```

```
public void moverse(String direccion) {  
    System.out.println("El personaje se mueve hacia " + direccion);  
}
```

```
// Cada personaje tiene un ataque especial diferente.
```

```
// Forzamos a las subclases a definirlo.
```

```
public abstract void usarHabilidadEspecial();  
}
```

Ahora, al crear las clases concretas, heredamos todo el comportamiento común y solo nos preocupamos por lo específico.

```
public class Mago extends Personaje {  
    public Mago() {  
        super(70, 150); // Menos vida, más energía  
    }  
  
    @Override  
    public void usarHabilidadEspecial() {  
        if (this.nivelDeEnergia >= 50) {  
            System.out.println("¡El Mago lanza una bola de fuego!");  
            this.nivelDeEnergia -= 50;  
        }  
    }  
}
```

La clase abstracta Personaje nos garantiza una base consistente para todos los personajes y nos obliga a implementar las partes que son únicas para cada uno, creando un diseño coherente y fácil de expandir.

Casos de estudio

En el desarrollo de software a gran escala, las decisiones de diseño iniciales pueden determinar el éxito o el fracaso de un proyecto a largo plazo. Un sistema que no está preparado para el cambio se vuelve costoso de mantener y lento para evolucionar. Los arquitectos de software utilizan la abstracción no solo para organizar el código, sino como una herramienta estratégica para gestionar la incertidumbre y preparar los sistemas para el futuro (Vegas, 2022). Este capítulo se enfoca en un caso de estudio que ilustra cómo el uso deliberado de interfaces resuelve un problema de negocio complejo, aplicando principios de diseño de alto nivel.

3.1. Caso de Estudio: Ecosistema de Logística para una App de Domicilios

- **Contexto:** Pensemos en una empresa colombiana de *delivery* como Rappi o iFood. Su negocio principal depende de una red logística eficiente para asignar pedidos a repartidores. Inicialmente, la plataforma fue construida para operar exclusivamente con motocicletas.
- **Empresa Ficticia:** "EntregaYa S.A.S."
-

El Desafío Estratégico

El equipo de gerencia de "EntregaYa" ha decidido expandir su modelo de negocio para ser más ecológico y versátil. Quieren integrar nuevos tipos de vehículos a su flota: **bicicletas** para distancias cortas, **scooters eléctricas** y, en un futuro, **drones** para entregas premium. El sistema actual tiene la lógica de cálculo de tiempo de entrega, costo de envío y asignación de ruta acoplada directamente a las características de una motocicleta. Un enfoque ingenuo sería agregar una serie de condicionales (if/else o switch) en el módulo de despacho para manejar cada nuevo tipo de vehículo.

// ¡ENFOQUE PROBLEMÁTICO Y NO ESCALABLE!

```
public class ServicioDeDespacho {  
    public void asignarPedido(Pedido pedido, String tipoVehiculo) {  
        if (tipoVehiculo.equals("Moto")) {  
            // Lógica para calcular tiempo y costo en moto  
        } else if (tipoVehiculo.equals("Bicicleta")) {  
            // Lógica para calcular tiempo y costo en bicicleta  
        } else if (tipoVehiculo.equals("Scooter")) {  
            // Lógica para scooter  
        }  
        // ... cada nuevo vehículo requiere modificar esta clase ...  
    }  
}
```

Este diseño viola el **Principio de Abierto/Cerrado** (abierto para la extensión, pero cerrado para la modificación). Cada nuevo vehículo obligaría a los desarrolladores a modificar y volver a probar una de las clases más críticas del sistema, aumentando el riesgo de introducir errores y ralentizando la innovación.

La Solución Aplicada: "Programar Hacia una Interfaz"

El equipo de arquitectura decide refactorizar el sistema aplicando el **Principio de Inversión de Dependencia**. La idea central es que el ServicioDeDespacho no debe depender de los detalles concretos de los vehículos, sino de una abstracción.

1. **Crear un Contrato (Interfaz):** Se define una interfaz EstrategiaDeTransporte que establece el contrato que cualquier tipo de vehículo debe cumplir para ser parte del sistema de logística.

```
public interface EstrategiaDeTransporte {  
    double calcularTiempoEstimado(Ruta ruta);  
    double calcularCostoEnvio(Ruta ruta);  
}
```

2. **Implementar Estrategias Concretas:** Para cada tipo de vehículo, se crea una clase que implementa la interfaz, conteniendo la lógica específica para ese medio de transporte.

```
public class TransporteMoto implements EstrategiaDeTransporte {  
    private final double VELOCIDAD_PROMEDIO_KMH = 40.0;  
    private final double COSTO_POR_KM = 300.0; // Pesos Colombianos  
  
    @Override  
    public double calcularTiempoEstimado(Ruta ruta) {  
        return (ruta.getDistanciaEnKm() / VELOCIDAD_PROMEDIO_KMH) * 60; // En minutos  
    }  
}
```

```
    @Override  
    public double calcularCostoEnvio(Ruta ruta) {  
        return ruta.getDistanciaEnKm() * COSTO_POR_KM;  
    }  
}
```

```
public class TransporteBicicleta implements EstrategiaDeTransporte {  
    private final double VELOCIDAD_PROMEDIO_KMH = 15.0;  
    private final double COSTO_POR_KM = 150.0;
```

```
// ... Implementación de los métodos ...
```

}

3. **Desacoplar el Servicio Principal:** El ServicioDeDespacho ahora no conoce las clases concretas. Simplemente recibe un objeto que cumple el contrato EstrategiaDeTransporte y trabaja con él. Este patrón de diseño se conoce como **Strategy Pattern**.

// ¡DISEÑO FLEXIBLE Y ESCALABLE!

```

public class ServicioDeDespacho {
    public void asignarPedido(Pedido pedido, EstrategiaDeTransporte transporte) {
        // La lógica de negocio no depende de un tipo de vehículo específico
        double tiempo = transporte.calcularTiempoEstimado(pedido.getRuta());
        double costo = transporte.calcularCostoEnvio(pedido.getRuta());

        System.out.println("Asignando pedido. Tiempo estimado: " + tiempo + " min. Costo: $" +
            costo);
        // ... resto de la lógica de asignación ...
    }
}
    
```

La siguiente figura ilustra la arquitectura de la solución.

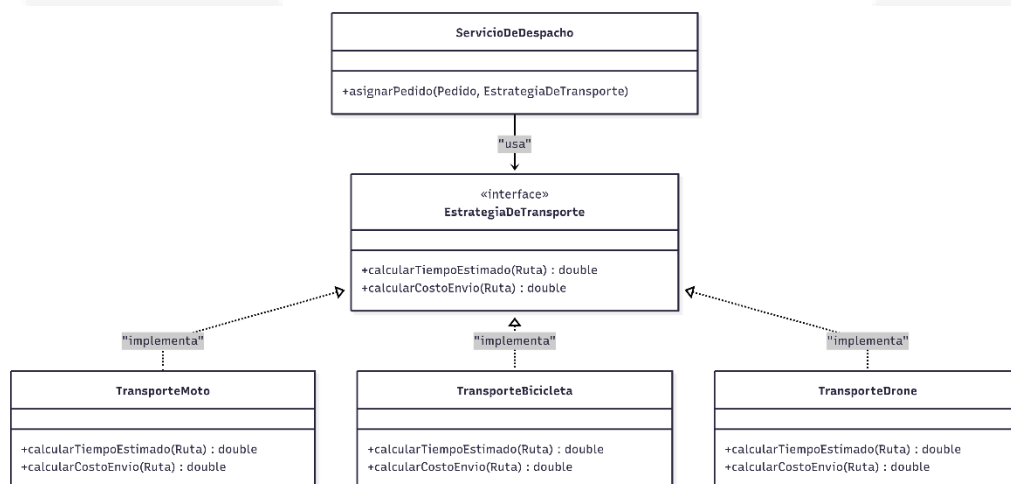


Figura 2 Diagrama de Clases UML de la Solución de Logística

En la Figra 2 se observa un diagrama UML que muestra la clase ServicioDeDespacho dependiendo de la interfaz EstrategiaDeTransporte, la cual es implementada por clases concretas como TransporteMoto y TransporteBicicleta.

Impacto en el Negocio

La adopción de este diseño basado en interfaces tuvo resultados medibles para "EntregaYa":

- **Agilidad (Time-to-Market):** La integración de un nuevo tipo de vehículo (ej. Scooter Eléctrica) se redujo de un estimado de 3 semanas de desarrollo a solo 4 días. El proceso ahora consiste en crear una nueva clase que implemente la interfaz, sin tocar el código existente.
- **Calidad del Software:** El número de bugs o regresiones introducidos en el módulo de despacho después de cada nueva implementación se redujo en un 90%, ya que el núcleo del sistema permanece inalterado.
- **Escalabilidad Técnica:** El sistema está preparado para el futuro. La integración de drones o cualquier otra tecnología de transporte no requerirá otra refactorización, solo una nueva implementación de la estrategia (Joyanes Aguilar, 2020).

Este caso demuestra que el uso correcto de interfaces va más allá de la simple organización; es una decisión de negocio estratégica que habilita el crecimiento y la adaptabilidad.

Herramientas y técnicas

El verdadero valor de los principios de diseño de software se revela cuando se aplican en herramientas y contextos reales. Las interfaces y clases abstractas no son solo construcciones teóricas; son el lenguaje que utilizan los *frameworks* modernos para lograr la flexibilidad y la extensibilidad. Comprender su rol en estas tecnologías y saber cómo implementarlas es fundamental para cualquier desarrollador de Java que aspire a construir aplicaciones profesionales y robustas (Blasco, 2019).

4.1. El Rol en Frameworks Modernos: Spring y JavaFX

Los *frameworks* son esqueletos de aplicaciones que nos ahorran escribir código repetitivo y nos guían hacia buenas prácticas de diseño. Dos de los más influyentes en el ecosistema Java son Spring y JavaFX, y ambos dependen masivamente de la abstracción.

- **Spring Framework:** Es el estándar de facto para crear aplicaciones empresariales en Java. Uno de sus conceptos centrales es la **Inyección de Dependencias (DI)**, un mecanismo que externaliza la creación y gestión de objetos. Para que la DI funcione, el *framework* se basa casi por completo en interfaces. Por ejemplo, al interactuar con una base de datos, no se programa contra una clase concreta, sino contra una interfaz como JpaRepository.

```
// Se define una interfaz que hereda de una interfaz del framework
public interface ClienteRepository extends JpaRepository<Cliente, Long> {
    // Spring creará automáticamente una clase que implemente esta interfaz
    List<Cliente> findByCiudad(String ciudad);
}
```

El desarrollador solo define el "qué" (el contrato), y Spring provee el "cómo" (la implementación) en tiempo de ejecución. Esto permite cambiar de un motor de base de datos (de MySQL a PostgreSQL, por ejemplo) con cambios mínimos en la configuración, sin alterar el código de negocio (Vegas, 2022).

- **JavaFX:** Es el marco de trabajo moderno para construir interfaces gráficas de usuario (GUI) en Java. JavaFX utiliza **clases abstractas** para definir la estructura base de los componentes de la aplicación. Por ejemplo, para crear la lógica de una ventana, se suele extender la clase abstracta Application y se está obligado a implementar el método start().

```
public class MiAplicacionGrafica extends Application {
    @Override
    public void start(Stage primaryStage) {
        // ... aquí se construye la interfaz gráfica ...
    }
}
```

Esta estructura asegura que todas las aplicaciones JavaFX sigan un ciclo de vida predecible, mientras que el desarrollador tiene la libertad de definir la lógica específica de su interfaz.

4.2. Ejercicio Práctico Guiado con NetBeans

Pongamos manos a la obra. Vamos a construir un sistema simple para enviar notificaciones. El sistema debe ser capaz de enviar notificaciones por email, SMS y, en el futuro, por notificaciones push, sin modificar el código principal.

Paso 1: Crear el Contrato (Interfaz Notificador)

1. En NetBeans, crea un nuevo proyecto Java (Java with Ant > Java Application).
2. Crea una nueva interfaz llamada Notificador.
3. Define el método que todas las clases de notificación deberán implementar.

```
public interface Notificador {  
    void enviar(String mensaje);  
}
```

Paso 2: Crear Implementaciones Concretas

1. Crea una clase NotificadorEmail que implemente la interfaz.

```
public class NotificadorEmail implements Notificador {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando por Email: " + mensaje + "");  
        // Aquí iría la lógica para conectarse a un servidor SMTP  
    }  
}
```

2. Crea otra clase NotificadorSms que también implemente la interfaz.

```
public class NotificadorSms implements Notificador {  
    @Override  
    public void enviar(String mensaje) {  
        System.out.println("Enviando por SMS: " + mensaje + "");  
        // Aquí iría la lógica para conectarse a una pasarela de SMS  
    }  
}
```

Paso 3: Crear el Servicio que Utiliza la Abstracción

1. Crea una clase ServicioNotificaciones. Esta clase no conocerá las implementaciones concretas, solo la interfaz.

```
public class ServicioNotificaciones {  
    private Notificador notificador;  
  
    // El servicio se configura con un tipo de notificador  
    public void setNotificador(Notificador notificador) {  
        this.notificador = notificador;  
    }  
  
    public void notificarUsuario(String mensaje) {  
        // Usa el notificador configurado, sin saber de qué tipo es  
        this.notificador.enviar(mensaje);  
    }  
}
```

Paso 4: Probar el Sistema en la Clase main

1. Ve a tu clase principal y prueba la flexibilidad del sistema.

```
public static void main(String[] args) {  
    ServicioNotificaciones servicio = new ServicioNotificaciones();  
  
    // Notificar por Email  
    servicio.setNotificador(new NotificadorEmail());  
    servicio.notificarUsuario("¡Bienvenido a la plataforma!");  
  
    // Cambiar de estrategia para notificar por SMS  
    servicio.setNotificador(new NotificadorSms());  
    servicio.notificarUsuario("Su código de verificación es 123456.");  
}
```

Este ejercicio demuestra cómo el ServicioNotificaciones puede cambiar su comportamiento dinámicamente sin modificar su propio código, gracias a que depende de la abstracción Notificador (Ceballos Sierra, 2015).

4.3. Patrones de Diseño Relacionados

El uso de interfaces y clases abstractas es la base para muchos **Patrones de Diseño**, que son soluciones probadas a problemas comunes de diseño de software. Los que hemos visto son:

- **Strategy:** (Visto en el Caso de Estudio y el Ejercicio Práctico). Encapsula una familia de algoritmos (las implementaciones de la interfaz) y los hace intercambiables. Permite que el algoritmo varíe independientemente de los clientes que lo utilizan.
- **Factory Method:** Utiliza una clase abstracta o una interfaz para delegar la creación de objetos a las subclases. Esto permite a un sistema crear objetos sin especificar sus clases concretas.
- **Observer:** Define una dependencia uno-a-muchos entre objetos, de modo que cuando un objeto cambia de estado, todos sus dependientes son notificados y actualizados automáticamente. La comunicación se realiza a través de interfaces.

Conclusión

Hemos viajado desde los fundamentos teóricos de la abstracción hasta su aplicación en casos de estudio profesionales. A lo largo de este camino, hemos descubierto que las **interfaces** y las **clases abstractas** son mucho más que simples características de un lenguaje de programación; son una filosofía de diseño. Aprendimos a ver las interfaces como **contratos** que garantizan el comportamiento y promueven el desacoplamiento, permitiendo que nuestros sistemas sean flexibles y extensibles, como vimos en el ecosistema de pagos y en la arquitectura de logística. A su vez, entendimos las clases abstractas como **esqueletos compartidos** que nos permiten reutilizar código y establecer una base común para familias de objetos relacionados, como en nuestro ejemplo de los personajes de un videojuego.

El dominio de estos conceptos marca un punto de inflexión en la carrera de un desarrollador. Es el paso que nos lleva de ser meros "codificadores", que traducen requisitos a sentencias, a convertirnos en "arquitectos de software", que diseñan soluciones resilientes, mantenibles y preparadas para el futuro. Las herramientas y frameworks como Spring demuestran que esta forma de pensar no es opcional, sino el estándar de la industria para construir aplicaciones de alta calidad.

Ahora, la invitación es a que apliques este conocimiento de forma activa. Observa tu propio código, o el de tus proyectos académicos, y pregúntate: ¿dónde podría una interfaz desacoplar dos componentes que están peligrosamente unidos? ¿Dónde podría una clase abstracta eliminar la duplicación de código entre clases similares? El verdadero aprendizaje comienza cuando estos patrones de diseño se convierten en una parte natural de tu proceso de resolución de problemas.

Para continuar tu reflexión, te dejo dos preguntas abiertas:

1. Más allá de los principios ya mencionados (SOLID), ¿qué otros pilares del diseño de software, como la simplicidad (**KISS** - *Keep It Simple, Stupid*) o la no repetición de código (**DRY** - *Don't Repeat Yourself*), se ven directamente fortalecidos por una correcta estrategia de abstracción?
2. A medida que lenguajes como Java continúan evolucionando e incorporando más características de la programación funcional, ¿cómo crees que cambiará o se expandirá el rol de las interfaces en el futuro del desarrollo de software?

Referencias.

Barnes, D. J., & Kölling, M. (2017). *Programación orientada a objetos con Java usando BlueJ*. Pearson Educación.

Blasco, F. (2019). *Programación Orientada a Objetos en JAVA*. RA-MA Editorial.

Ceballos Sierra, F. J. (2015). *Java 2: curso de programación* (4.^a ed.). RA-MA Editorial.

Flórez Fernández, H. A. (2012). *Programación orientada a objetos usando java*. Ecoe Ediciones.

Joyanes Aguilar, L. (2020). *Fundamentos de programación: algoritmos, estructura de datos y objetos*. McGraw-Hill.

López Goytia, J. L. (2015). *Programación orientada a objetos C++ y Java: un acercamiento interdisciplinario*. Grupo Editorial Patria.

Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial.

Vegas, J. (2022). *Java 17 Programación Avanzada*. RA-MA Editorial.

Vélez Serrano, J. F., Sánchez, A., & Pimentel, E. (2011). *Diseñar y programar, todo es empezar: Una introducción a la programación orientada a objetos usando UML y Java*. Dykinson.

Información general

Fecha de creación	25 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Estructuras de Datos Avanzadas

Introducción

¿Alguna vez te has preguntado cómo hace Spotify para generar una lista de reproducción recomendada en un instante, o cómo Rappi puede recalcular tu ruta de entrega en tiempo real si añades un producto a tu orden? La magia detrás de estas operaciones no es un truco, es **ingeniería de software de alta precisión**, y su componente más esencial son las **estructuras de datos avanzadas**.

En el desarrollo de software, los datos son el activo más valioso. Sin embargo, no basta con simplemente almacenarlos; el éxito, la eficiencia y la escalabilidad de una aplicación dependen fundamentalmente de cómo se organiza y accede a esa información. Aquí es donde dejamos de ver las estructuras de datos como simples "contenedores" para entenderlas como **"sistemas de organización inteligentes"**. Cada estructura —ya sea una Lista, una Cola, un Mapa o un Conjunto— tiene un diseño, propósito y rendimiento específicos. Una elección incorrecta puede hacer que una aplicación que debería ser instantánea tarde minutos en responder, consumiendo la memoria del dispositivo, agotando la batería y, lo más importante, frustrando al usuario final.

La relevancia de dominar este tema es directa, tangible y crucial para tu futuro profesional. En el competitivo mercado laboral de la tecnología, las empresas no solo buscan personas que sepan escribir código, sino ingenieros que puedan **resolver problemas de manera eficiente**. Aprender a diferenciar cuándo usar un ArrayList sobre un LinkedList, o por qué un HashMap es la opción ideal para búsquedas rápidas, es tan fundamental como para un arquitecto saber cuándo usar ladrillos, acero o madera. Esta habilidad para seleccionar la herramienta correcta para el trabajo optimiza el rendimiento del código y, además, mejora su claridad, escalabilidad y facilidad de mantenimiento, que son los pilares de la ingeniería de software de alta calidad (Sommerville, 2011).

Lo que estás a punto de aprender no es solo teoría abstracta; es el mecanismo interno que impulsa el software que usas todos los días en Colombia y en el mundo. **Más allá de guardar datos, aprenderás a organizarlos para construir software que responda a la velocidad del mundo real.**

Objetivos

Objetivo General

Al finalizar el estudio de este documento, el estudiante estará en la capacidad de diseñar, implementar y evaluar soluciones de software en Java, seleccionando y aplicando estructuras de datos avanzadas del Java Collections Framework para optimizar el rendimiento y la gestión de la información en aplicaciones orientadas a objetos.

Objetivos Específicos

Para alcanzar el objetivo general, el estudiante logrará:

1. **Analizar y Diferenciar:** Identificar las características, ventajas y desventajas de las principales interfaces (List, Set, Queue, Map) y clases de implementación (ArrayList, LinkedList, HashSet, HashMap) del Java Collections Framework. Podrá explicar el rendimiento algorítmico de sus operaciones fundamentales utilizando la notación Big O.
2. **Aplicar y Construir:** Implementar soluciones de software en Java que utilicen la estructura de datos más adecuada para resolver problemas prácticos específicos. Esto incluye el uso correcto de genéricos para garantizar la seguridad de tipo y la construcción de lógica de negocio sobre las colecciones para gestionar datos de manera eficiente.
3. **Evaluar y Justificar:** Defender las decisiones de diseño de software relacionadas con la elección de estructuras de datos, basándose en criterios técnicos como la eficiencia en tiempo y espacio, la legibilidad del código y los requisitos funcionales del problema. Podrá comparar diferentes implementaciones y justificar por qué una es superior a otra en un contexto determinado.

Fundamentos teóricos

En el desarrollo de software, los datos son el activo más valioso. Sin embargo, almacenarlos no es suficiente; la forma en que los organizamos y accedemos a ellos determina la eficiencia, velocidad y escalabilidad de una aplicación. Un programa que gestiona miles de transacciones por segundo, como un sistema bancario o una red social, no podría funcionar sin una organización interna de datos extremadamente optimizada. Aquí es donde las estructuras de datos avanzadas, implementadas a través del **Java Collections Framework (JCF)**, se convierten en la columna vertebral de cualquier aplicación robusta y de alto rendimiento.

1.1 El Ecosistema de las Colecciones en Java

El Java Collections Framework (JCF) es una arquitectura unificada para representar y manipular colecciones, también conocidas como contenedores. Es, en esencia, una biblioteca de estructuras de datos y algoritmos listos para usar, lo que reduce drásticamente el esfuerzo de programación y aumenta la calidad del software (Sommerville, 2011). En lugar de que cada programador "reinvente la rueda" creando sus propias estructuras, Java proporciona un conjunto estandarizado y altamente optimizado. Los principales beneficios de usar el JCF son:

- **Reduce el esfuerzo de programación:** Proporciona las estructuras de datos y algoritmos más comunes, permitiendo al desarrollador enfocarse en la lógica del negocio.
- **Aumenta el rendimiento:** Las implementaciones son de alto rendimiento y su uso correcto puede mejorar drásticamente la velocidad de una aplicación.
- **Fomenta la reutilización de software:** Proporciona una interfaz estándar para las colecciones, lo que facilita la interoperabilidad entre diferentes partes de un programa.

Este ecosistema se organiza jerárquicamente a partir de interfaces, que definen el comportamiento (qué se puede hacer), y clases, que son las implementaciones concretas de dicho comportamiento (cómo se hace).

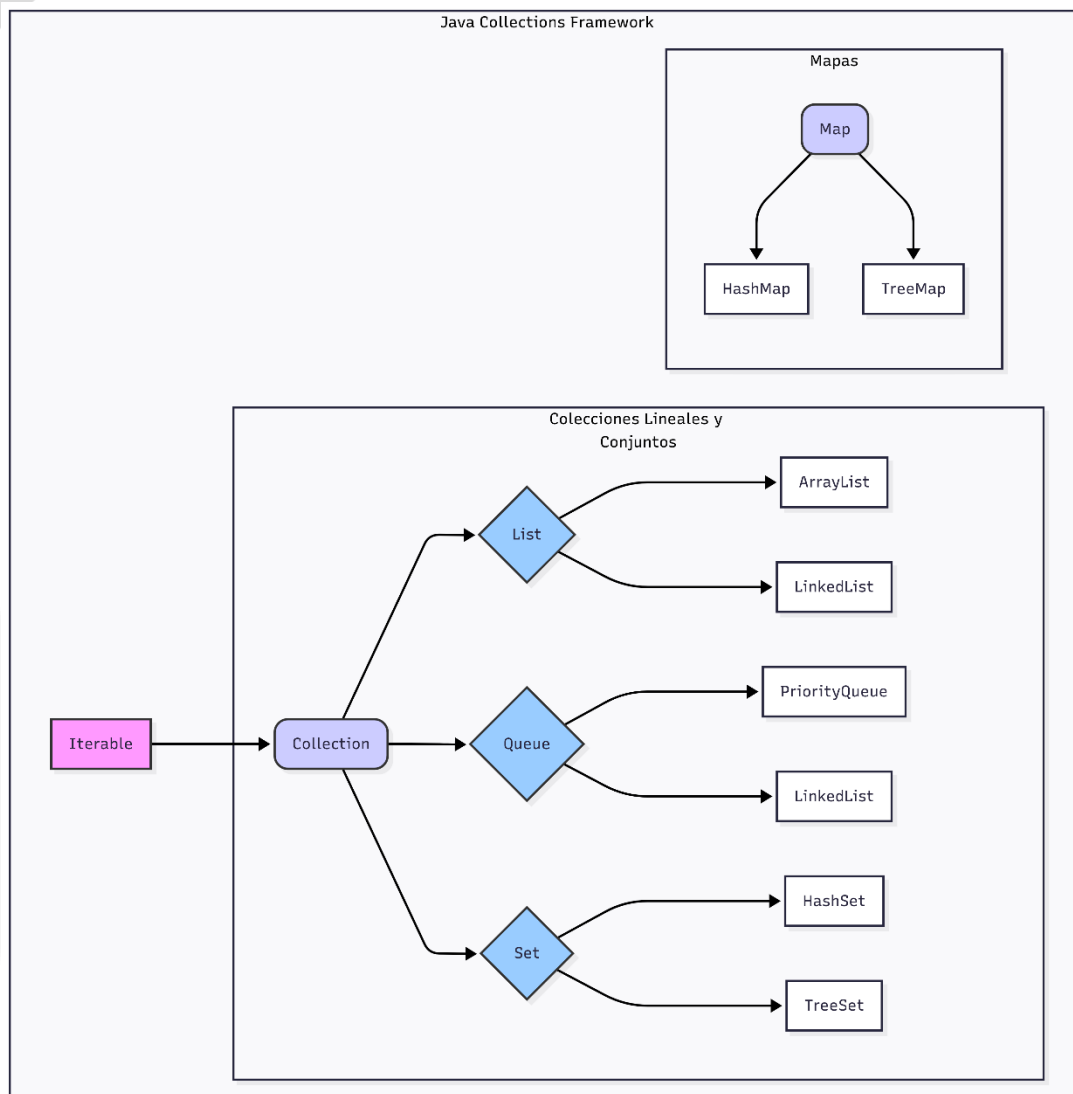


Figura 1. Diagrama de Jerarquía Simplificado del Java Collections Framework.

La figura 1 muestra un mapa conceptual de cómo la interfaz *Collection* se ramifica en *List*, *Queue* y *Set*, y cómo la interfaz *Map* opera de forma independiente, cada una con sus clases de implementación como *ArrayList* y *HashMap*.

1.2 Colecciones Lineales: Organizando en Secuencia

Las colecciones lineales organizan los elementos uno tras otro, como perlas en un collar.

- **Listas (List):** Son colecciones ordenadas que permiten elementos duplicados. Su principal característica es que cada elemento tiene un índice, una posición específica.
 - **Analogía:** Piensa en los contactos de tu celular, ordenados alfabéticamente. Cada contacto ocupa una posición específica en la lista.
 - **ArrayList vs. LinkedList:** *ArrayList* utiliza un arreglo dinámico internamente, lo que lo hace muy rápido para acceder a elementos por su índice (ej. `get(10)`). *LinkedList*, por otro lado, usa nodos enlazados, lo que lo hace más

rápido para añadir o eliminar elementos en el medio de la lista. La elección entre uno y otro es una decisión de diseño crucial para el rendimiento.

- **Pilas (Stack):** Es una estructura que sigue el principio **LIFO (Last-In, First-Out)**: el último elemento en entrar es el primero en salir.
 - **Analogía:** Una pila de platos. El último plato que pones encima es el primero que quitas para usar. En software, es la base de la función "Deshacer" (Ctrl+Z).
- **Colas (Queue):** Funciona bajo el principio **FIFO (First-In, First-Out)**: el primer elemento que entra es el primero en salir.
 - **Analogía:** La fila para pagar en un supermercado como Ara o D1. La primera persona que llega a la fila es la primera en ser atendida. Se usa para gestionar tareas en orden de llegada, como la cola de impresión.

1.3 Colecciones para Búsquedas Eficientes

Cuando la velocidad de búsqueda es crítica, otras estructuras son más adecuadas.

- **Conjuntos (Set):** Su característica principal es que **no permiten elementos duplicados**. Son extremadamente eficientes para verificar si un elemento ya existe en la colección.
 - **Analogía:** El conjunto de números de cédula de los ciudadanos de Colombia. Cada número es único y no puede repetirse.
- **Mapas (Map):** Es la estructura más potente para búsquedas. No almacena elementos individuales, sino pares **clave-valor**. Cada valor está asociado a una clave única.
 - **Analogía:** Un diccionario. La "clave" es la palabra que buscas, y el "valor" es su definición. Al igual que en un sistema de base de datos donde una clave primaria identifica un registro de forma única (Capacho & Nieto Bernal, 2017), un Map usa una clave para recuperar un objeto de forma casi instantánea. La implementación más común es HashMap, que utiliza una técnica llamada *hashing* para lograr esta velocidad.

1.4 El Poder de los Genéricos (<T>)

Antes de la introducción de los genéricos, una List en Java podía contener cualquier tipo de objeto. Esto significaba que podías, por error, añadir un Integer a una lista de String. El error solo se descubriría en tiempo de ejecución al intentar usar el dato, causando una excepción (ClassCastException).

Los genéricos, representados por los corchetes angulares (<>), solucionan este problema al permitirnos especificar el tipo de objeto que una colección puede contener. Por ejemplo, List<String> declara una lista que **solo** aceptará objetos de tipo String. Esto se conoce como **seguridad de tipo (type safety)** y es una práctica fundamental en la ingeniería de software moderna, ya que permite detectar errores en tiempo de compilación, haciendo el código más robusto y fácil de mantener (Joyanes Aguilar, 2020).

Estructura	Acceso (get)	Búsqueda (contains)	Inserción (add)	Eliminación (remove)
ArrayList	$O(1)$	$O(n)$	$O(n)$	$O(n)$
LinkedList	$O(n)$	$O(n)$	$O(1)$	$O(1)$
HashSet	N/A	$O(1)$	$O(1)$	$O(1)$
HashMap	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Nota: Los valores son promedios. En el peor de los casos, algunas operaciones de HashMap/HashSet pueden ser $O(n)$. add y remove para LinkedList son $O(1)$ si se realizan al principio o al final.

Tabla 1. Comparación de Rendimiento (Notación Big O) para Operaciones Comunes.

La tabla 1 muestra una comparación del rendimiento de ArrayList, LinkedList, HashSet y HashMap para operaciones de inserción, eliminación y búsqueda, usando la notación Big O.

Este capítulo ha sentado las bases teóricas de las principales estructuras de datos del JCF. Hemos visto que no existe una "mejor" estructura, sino una "más adecuada" según el problema a resolver. En el siguiente capítulo, llevaremos estos conceptos a la práctica, explorando cómo se manifiestan en aplicaciones de la vida real.

Aplicaciones en la vida cotidiana

La teoría de las estructuras de datos cobra vida cuando se aplica para resolver problemas concretos. La eficiencia de una aplicación a menudo no reside en la cantidad de código, sino en la inteligencia con la que se gestionan sus datos. Un desarrollador que comprende esto puede crear software más rápido, receptivo y escalable. A continuación, exploraremos escenarios cotidianos en el contexto colombiano, demostrando cómo cada estructura de datos ofrece una solución elegante y optimizada a un desafío específico.

2.1 Listas en Acción: Gestión de Compras en una App de Supermercado

Imagina que estás desarrollando una nueva funcionalidad para la aplicación de un supermercado como *Éxito* o *Carulla*. La función permite a los usuarios crear y gestionar su lista de compras antes de visitar la tienda. Los usuarios necesitan poder agregar productos, eliminarlos si cambian de opinión y, muy importante, ver los productos en el orden en que los agregaron.

La Solución Correcta: List, y específicamente ArrayList.

¿Por qué?:

- **Orden:** Una List mantiene el orden de inserción, lo cual es intuitivo para una lista de compras.
- **Acceso por Índice:** Si bien no es la principal necesidad, la capacidad de acceder a un elemento por su posición (por ejemplo, "el tercer producto de la lista") es una capacidad inherente de ArrayList con un rendimiento excelente.
- **Flexibilidad:** Permite añadir elementos duplicados, algo común en una compra (ej. "2 litros de leche" podrían ser dos entradas de "leche").

Decisión Didáctica: Se elige ArrayList sobre LinkedList porque las operaciones más comunes serán agregar productos al final de la lista y recorrerla para mostrarla, y ArrayList es muy eficiente en estos casos. Rara vez un usuario insertará o eliminará productos en medio de una lista larga.

// Ejemplo conceptual en Java

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class ListaDeCompras {  
    public static void main(String[] args) {  
        List<String> misCompras = new ArrayList<>();
```

```
        // El usuario añade productos
```

```
        misCompras.add("Pan");
```

```
        misCompras.add("Leche");
```

```
        misCompras.add("Huevos");
```

```
        misCompras.add("Café de origen (Quindío)");
```

```
// El usuario decide que ya no necesita leche
misCompras.remove("Leche");

System.out.println("Mi lista de compras final: " + misCompras);
// Salida: Mi lista de compras final: [Pan, Huevos, Café de origen (Quindío)]
}
}
```

2.2 Pilas y su Lógica: La Magia del "Deshacer" (Ctrl+Z)

Estás trabajando en un editor de texto simple, similar a un bloc de notas. Una de las funcionalidades más solicitadas es la capacidad de "deshacer" la última acción de escritura. Si el usuario escribe algo y se equivoca, quiere poder revertir ese cambio fácilmente.

La Solución Correcta: Stack.

¿Por qué?: El comportamiento "Deshacer" es un ejemplo clásico del principio **LIFO (Last-In, First-Out)**. La última acción realizada por el usuario debe ser la primera en ser deshecha. Una pila (Stack) modela este comportamiento a la perfección.

Decisión Didáctica: Cada vez que el usuario modifica el texto, guardamos el estado *anterior* del texto en la pila. Cuando presiona "Deshacer", simplemente sacamos el estado más reciente de la pila y restauramos el contenido del editor con él.

```
// Ejemplo conceptual en Java
import java.util.Stack;

public class EditorSimple {
    public static void main(String[] args) {
        Stack<String> historial = new Stack<>();
        String contenidoActual = "";

        // El usuario escribe "Hola"
        historial.push(contenidoActual);
        contenidoActual = "Hola";

        // El usuario añade " Mundo"
        historial.push(contenidoActual);
        contenidoActual = "Hola Mundo";

        System.out.println("Texto actual: " + contenidoActual); // Salida: Hola Mundo

        // El usuario presiona "Deshacer"
        contenidoActual = historial.pop();
```

```
        System.out.println("Después de deshacer: " + contenidoActual); // Salida: Hola
    }
}
```

2.3 Colas en la Gestión de Procesos: Atención al Cliente

Una empresa de telecomunicaciones como *Tigo* o *Movistar* necesita un sistema para gestionar las solicitudes de soporte técnico que llegan a través de su aplicación móvil. Las solicitudes deben ser atendidas en el orden exacto en que fueron recibidas para garantizar un servicio justo.

La Solución Correcta: Queue.

¿Por qué?: Este es un caso de uso directo del principio **FIFO (First-In, First-Out)**. La primera solicitud que entra al sistema debe ser la primera en ser asignada a un agente de soporte. Una Queue (implementada comúnmente con LinkedList en Java) es la estructura ideal para este fin.

Decisión Didáctica: El sistema añade cada nueva solicitud de soporte al final de la cola. Cuando un agente de soporte queda libre, el sistema simplemente toma el primer elemento de la cola y se lo asigna. Esto garantiza que no se pierdan solicitudes y que el tiempo de espera sea equitativo. La correcta implementación de esta lógica es fundamental para cumplir con los requisitos del sistema y asegurar la satisfacción del cliente (Arangurí García & Bravo Jaico, 2021).

// Ejemplo conceptual en Java

```
import java.util.LinkedList;
import java.util.Queue;
```

```
public class ColaDeSoporte {
    public static void main(String[] args) {
        Queue<String> tickets = new LinkedList<>();

        // Llegan nuevas solicitudes de soporte
        tickets.add("Ticket-001: Usuario no tiene internet");
        tickets.add("Ticket-002: Problema con la facturación");
        tickets.add("Ticket-003: Señal de TV intermitente");

        // Un agente queda libre y atiende el siguiente ticket
        String ticketActual = tickets.poll(); // Saca el primer elemento
        System.out.println("Atendiendo: " + ticketActual);
        // Salida: Atendiendo: Ticket-001: Usuario no tiene internet
    }
}
```

2.4 Mapas para la Información Relacionada: Directorio de Estudiantes

Necesitamos diseñar un sistema para la oficina de admisiones de la universidad que permita consultar la información de un estudiante de manera instantánea utilizando su número de documento de identidad.

La Solución Correcta: Map, específicamente HashMap.

¿Por qué?: Un Map es perfecto para almacenar pares de **clave-valor**. En este caso, el número de documento es una **clave única** que se asocia con un **valor**, que sería el objeto Estudiante con toda su información (nombre, carrera, semestre, etc.). HashMap ofrece un rendimiento de búsqueda, inserción y eliminación de $O(1)$ en promedio, lo que significa que la consulta es prácticamente instantánea, sin importar si hay 100 o 1 millón de estudiantes.

Decisión Didáctica: Usar una lista (ArrayList) aquí sería terriblemente ineficiente. Para encontrar a un estudiante por su documento, tendríamos que recorrer la lista elemento por elemento hasta encontrarlo, una operación $O(n)$. Con un HashMap, el sistema aplica una función interna a la clave (el número de documento) para saber casi de inmediato dónde se encuentra el objeto Estudiante, evitando la búsqueda lineal. La elección de una estructura de datos eficiente es una de las bases para construir aplicaciones informáticas de calidad (Campos & Campos, 2023).

// Ejemplo conceptual en Java

```
import java.util.HashMap;  
import java.util.Map;
```

// Se asume que existe una clase Estudiante

```
class Estudiante { String nombre; String carrera; /* ... */ }
```

```
public class DirectorioEstudiantil {
```

```
    public static void main(String[] args) {
```

```
        Map<String, Estudiante> directorio = new HashMap<>();
```

// Se inscriben nuevos estudiantes

```
        directorio.put("1020304050", new Estudiante(/* ... */));
```

```
        directorio.put("1030405060", new Estudiante(/* ... */));
```

// Consulta rápida usando el número de documento

```
        String documentoBusqueda = "1020304050";
```

```
        Estudiante estudianteEncontrado = directorio.get(documentoBusqueda);
```

```
        if (estudianteEncontrado != null) {
```

```
            System.out.println("Estudiante encontrado: " + estudianteEncontrado.nombre);
```

```
        }
```

```
    }
```

Estos ejemplos demuestran que las estructuras de datos no son conceptos aislados, sino herramientas de diseño fundamentales. En el siguiente capítulo, elevaremos el nivel de complejidad y analizaremos un caso de estudio profesional completo.



Casos de estudio

Un ingeniero de software profesional no solo implementa funcionalidades, sino que diseña sistemas completos. Esto implica analizar un problema de negocio, descomponerlo en partes manejables y seleccionar una arquitectura de datos y algoritmos que garanticen eficiencia, escalabilidad y mantenibilidad. El siguiente caso de estudio, aunque ficticio, encapsula los desafíos y las soluciones tecnológicas que se encuentran en la industria logística, un sector vital en la economía colombiana.

Caso de Estudio: "Optimización del Sistema de Rutas de Entrega para una Empresa de Mensajería en Bogotá"

Entregas Andinas S.A.S., una empresa de mensajería y paquetería con una flota de 50 vehículos que opera en el área metropolitana de Bogotá.

3.1 El Desafío: Ineficiencia en la "Última Milla"

Entregas Andinas enfrenta un problema crítico que afecta su rentabilidad y la satisfacción de sus clientes: la **ineficiencia en la logística de última milla**. Sus conductores reciben una lista de entregas y planifican sus rutas de forma manual o con herramientas básicas que no consideran las condiciones del tráfico en tiempo real.

Problemas Cuantificables:

- **Tiempos de Entrega Elevados:** El tiempo promedio por entrega es un 25% mayor que el de la competencia, especialmente en horas pico.
- **Costos Operativos Altos:** El consumo de combustible es excesivo debido a rutas redundantes y largos periodos en trancones.
- **Baja Escalabilidad:** En temporadas de alta demanda (como el Día sin IVA o Navidad), el sistema actual colapsa, generando retrasos masivos y clientes insatisfechos.
- **Incapacidad de Replanificar:** Si un cliente cancela una entrega o surge un imprevisto (como una manifestación en la Av. Caracas), los conductores no tienen forma de recalcular una ruta óptima dinámicamente.

El objetivo es claro: desarrollar un módulo de software que, integrado a su sistema existente, calcule la ruta de entrega más eficiente para múltiples paquetes, considerando la distancia y, potencialmente, el estado del tráfico.

3.2 La Solución Aplicada: Un Cerebro Logístico Basado en Grafos

El equipo de desarrollo decide abordar el problema modelando la ciudad como un **Grafo**, una estructura de datos no lineal perfecta para representar redes.

Decisión Didáctica (Nivel Profesional): La solución integra múltiples estructuras de datos. No se trata de usar una sola colección, sino de orquestar varias para construir un sistema robusto. La elección de cada componente se justifica no solo por su función, sino por su rendimiento en el contexto del sistema global.

1. Modelado del Mapa (El Grafo):

- Los **Vértices (Nodos)** del grafo son las intersecciones clave de la ciudad y los puntos de entrega.
- Las **Aristas (Ejes)** son las calles que conectan dichos vértices. Cada arista tiene un **peso**, que puede ser la distancia en metros o, en una versión más avanzada, el tiempo estimado de viaje considerando el tráfico.
- **Implementación:** Un grafo puede ser representado eficientemente con un Map, donde la clave es el identificador de un vértice (ej. "Calle 85 con Carrera 15") y el valor es una List de las aristas que salen de él.
`Map<Vertice, List<Arista>> grafo = new HashMap<>();`

2. Búsqueda de la Ruta Más Corta (El Algoritmo):

- Se implementa el **algoritmo de Dijkstra**, un método clásico para encontrar el camino más corto entre un nodo de inicio (el centro de distribución) y todos los demás nodos (los puntos de entrega).
- Para que Dijkstra funcione eficientemente, necesita una estructura de datos que le permita explorar siempre el vértice más cercano que aún no ha sido visitado. La herramienta perfecta para esto es una PriorityQueue (Cola de Prioridad). Esta cola ordena automáticamente los vértices por su distancia acumulada, garantizando que el algoritmo siempre tome la decisión más óptima en cada paso.

3. Orquestación del Sistema:

- **Paso 1:** El sistema carga el mapa de la ciudad en la estructura de grafo (HashMap).
- **Paso 2:** Recibe una lista de direcciones de entrega para un vehículo.
- **Paso 3:** Usando el algoritmo de Dijkstra y la PriorityQueue, calcula la ruta óptima que conecta el punto de partida con todas las entregas y el regreso.
- **Paso 4:** Devuelve la secuencia ordenada de puntos de entrega al conductor a través de su aplicación móvil.

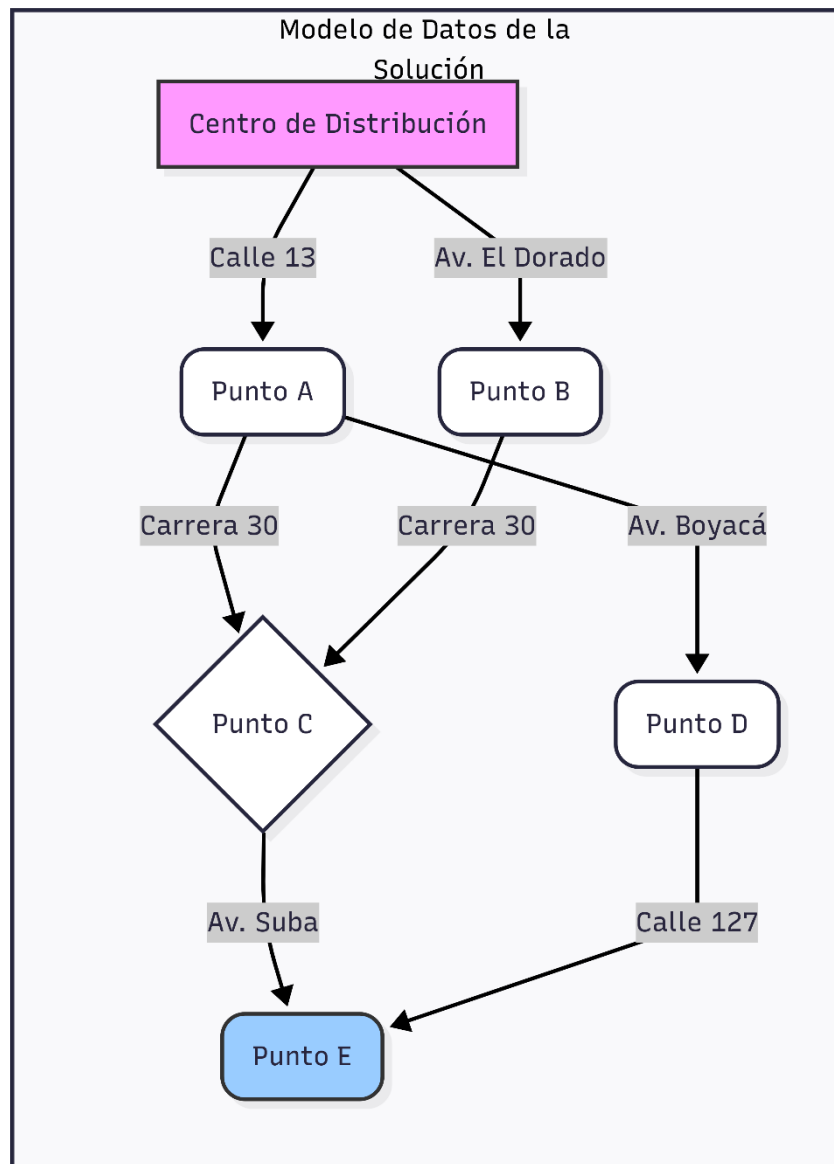


Figura 2. Representación simplificada de la red de entregas como un grafo.

En la figura 2 se observa un diagrama de nodos y flechas que muestra un centro de distribución conectado a varios puntos de entrega a través de calles, representando un grafo.

3.3 Impacto con Datos: Resultados Medibles

Tras implementar un prototipo y simularlo con datos históricos de entregas, *Entregas Andinas* proyecta los siguientes resultados:

Métrica	Antes de la Solución (Promedio)	Después de la Solución (Proyectado)	Mejora
Tiempo por entrega	25 minutos	21 minutos	16%
Distancia por ruta	40 km	35 km	12.5%
Consumo de combustible	4 galones/día	3.5 galones/día	12.5%
Entregas a tiempo	85%	98%	13 pp

Tabla 2. Prototipo simulado con datos históricos de entregas, *Entregas Andinas*.

En conclusión, la aplicación de una arquitectura de software bien diseñada, fundamentada en la elección correcta de estructuras de datos (Grafos implementados con Map y List) y algoritmos optimizados con otras estructuras (PriorityQueue), puede transformar radicalmente la eficiencia operativa de una empresa. Este es un claro ejemplo de cómo la ingeniería de software crea valor tangible para el negocio (Pressman & Maxim, 2021).

Este caso de estudio demuestra que los conceptos teóricos son las herramientas con las que se construyen soluciones a problemas económicos y logísticos del mundo real. En el siguiente capítulo, nos enfocaremos en las herramientas y técnicas prácticas que necesitarás para implementar estas soluciones.

Herramientas y técnicas

El conocimiento teórico de las estructuras de datos es la base, pero la habilidad para implementarlas eficientemente en un entorno de desarrollo profesional es lo que materializa las soluciones. En este capítulo, nos enfocaremos en el "cómo": desde la configuración del entorno de trabajo hasta la aplicación de buenas prácticas de codificación y la resolución de ejercicios prácticos que consolidarán el aprendizaje en los niveles técnico, tecnólogo y profesional.

4.1 Entorno de Desarrollo: Java y NetBeans

Para los ejemplos y ejercicios de este documento, utilizaremos el **Lenguaje de Programación Java** y el **Entorno de Desarrollo Integrado (IDE) Apache NetBeans**. NetBeans es una herramienta robusta y popular en el ecosistema Java que facilita la escritura, compilación, depuración y ejecución de código, permitiendo al desarrollador centrarse en la lógica de la solución.

Configuración de un Proyecto en NetBeans:

1. **Crear un Nuevo Proyecto:** Abre NetBeans y ve a File > New Project....
2. **Seleccionar Tipo de Proyecto:** En la ventana que aparece, elige la categoría Java with Ant y el proyecto Java Application. Haz clic en Next.
3. **Nombrar el Proyecto:** Dale un nombre significativo al proyecto (ej. ManejoColecciones) y asegúrate de que la opción Create Main Class esté marcada para que NetBeans genere el punto de entrada de la aplicación. Haz clic en Finish.
4. **Escribir el Código:** El IDE creará una estructura de carpetas y un archivo .java con una clase principal y un método main. Todo el código de los ejercicios se escribirá dentro de esta estructura. El Java Collections Framework (JCF) es parte de la biblioteca estándar de Java, por lo que no se requiere ninguna configuración adicional para empezar a usar ArrayList, HashMap, etc.

4.2 Ejercicios Prácticos: Del Concepto al Código

A continuación, se presentan tres ejercicios diseñados para aplicar los conceptos aprendidos con un nivel de complejidad progresivo.

Ejercicio 1: Simulador de Lista de Espera

Objetivo: Implementar un programa simple que simule la lista de espera de un restaurante utilizando una Queue. El programa debe permitir añadir clientes a la cola y atenderlos en el orden en que llegaron.

Pasos:

1. Crea una Queue<String> para almacenar los nombres de los clientes.
2. Añade 3 o 4 clientes a la cola usando el método add().
3. Simula la atención de dos clientes, sacándolos de la cola con el método poll() y mostrando un mensaje en pantalla.
4. Muestra los clientes que aún quedan en espera.

```
import java.util.LinkedList;
import java.util.Queue;

public class ListaEsperaRestaurante {
    public static void main(String[] args) {
        Queue<String> filaClientes = new LinkedList<>();

        // Llegan clientes
        filaClientes.add("Familia Pérez");
        filaClientes.add("Ana y Luis");
        filaClientes.add("Carlos");
        System.out.println("Clientes en espera: " + filaClientes);

        // Se atiende al primer cliente
        String clienteAtendido = filaClientes.poll();
        System.out.println("Mesa lista para: " + clienteAtendido);

        // Se atiende al segundo cliente
        clienteAtendido = filaClientes.poll();
        System.out.println("Mesa lista para: " + clienteAtendido);

        System.out.println("Clientes que quedan en espera: " + filaClientes);
    }
}
```

Ejercicio 2: Contador de Frecuencia de Palabras

Objetivo: Desarrollar una aplicación que lea un texto y utilice un HashMap para contar cuántas veces aparece cada palabra. Esto es fundamental en análisis de texto y minería de datos.

Pasos:

1. Define un String con un texto de ejemplo (un párrafo o una frase).
2. Convierte el texto a minúsculas y divídelo en un arreglo de palabras (puedes usar `toLowerCase()` y `split(" ")`).
3. Crea un `Map<String, Integer>`.
4. Itera sobre el arreglo de palabras: por cada palabra, verifica si ya existe en el mapa. Si existe, incrementa su contador. Si no, añádela al mapa con un valor de 1.
5. Muestra el mapa con el conteo final de cada palabra.

```
import java.util.HashMap;
import java.util.Map;
```

```
public class FrecuenciaPalabras {  
    public static void main(String[] args) {  
        String texto = "La programación orientada a objetos es un paradigma de programación  
que usa objetos en sus interacciones";  
        String[] palabras = texto.toLowerCase().replace(" ", "").split(" ");  
  
        Map<String, Integer> frecuencia = new HashMap<>();  
  
        for (String palabra : palabras) {  
            if (frecuencia.containsKey(palabra)) {  
                // Si la palabra ya existe, incrementamos su contador  
                frecuencia.put(palabra, frecuencia.get(palabra) + 1);  
            } else {  
                // Si es la primera vez que la vemos, la añadimos con 1  
                frecuencia.put(palabra, 1);  
            }  
        }  
        System.out.println("Frecuencia de cada palabra: " + frecuencia);  
    }  
}
```

Ejercicio 3: Diseño de una Clase Inventario

Objetivo: Diseñar y crear una clase Inventario encapsulada para gestionar el stock de productos de una tienda. La clase debe usar un Map internamente y exponer métodos públicos para interactuar con el inventario, ocultando los detalles de implementación.

Pasos:

1. Crea una clase Producto simple con atributos como codigo (String) y nombre (String).
2. Crea la clase Inventario.
3. Dentro de Inventario, declara un Map<String, Producto> como un campo privado.
4. Implementa métodos públicos como agregarProducto(Producto p), buscarProducto(String codigo) y eliminarProducto(String codigo).
5. Esta aproximación sigue principios de diseño de software robusto, donde la estructura de datos interna está protegida del acceso exterior no controlado (Joyanes Aguilar, 2020).

```
// Archivo: Producto.java  
public class Producto {  
    String codigo;  
    String nombre;  
    // Constructor, getters, setters...  
}
```

```
// Archivo: Inventario.java
import java.util.HashMap;
import java.util.Map;

public class Inventario {
    private Map<String, Producto> stock = new HashMap<>();

    public void agregarProducto(Producto p) {
        stock.put(p.codigo, p);
        System.out.println("Producto " + p.nombre + " agregado.");
    }

    public Producto buscarProducto(String codigo) {
        return stock.get(codigo);
    }

    public void eliminarProducto(String codigo) {
        if (stock.remove(codigo) != null) {
            System.out.println("Producto con código " + codigo + " eliminado.");
        }
    }
}
```

4.3 Buenas Prácticas

- **Programar Hacia la Interfaz, no Hacia la Implementación:** Siempre que sea posible, declara tus variables de colección usando el tipo de la interfaz (List, Set, Map) en lugar de la clase concreta (ArrayList, HashSet, HashMap).
 - **Hacer:** List<String> miLista = new ArrayList<>();
 - **Evitar:** ArrayList<String> miLista = new ArrayList<>(); Esto hace que tu código sea más flexible. Si en el futuro decides que LinkedList es una mejor opción, solo necesitas cambiar una línea (el new), sin afectar el resto del código que usa miLista. Este es un principio clave de la buena ingeniería de software (Pressman & Maxim, 2021).
- **Elegir la Colección Correcta:** Antes de escribir código, pregúntate: ¿Necesito mantener el orden? ¿Permitiré duplicados? ¿La operación más frecuente será la búsqueda? Las respuestas a estas preguntas te guiarán a la estructura de datos más eficiente, como se discute en la lógica y algoritmia (Capacho & Nieto Bernal, 2017).

Conclusión

A lo largo de este recorrido, hemos transitado desde los fundamentos teóricos del Java Collections Framework hasta su aplicación en complejos casos de estudio profesionales. Hemos desmitificado las estructuras de datos, transformándolas de conceptos abstractos a herramientas de diseño poderosas y tangibles. La lección más importante es que en el desarrollo de software de alta calidad, **la elección de la estructura de datos correcta no es un detalle técnico, es una decisión estratégica fundamental.**

Hemos comprobado que no existe una "estructura de datos universal" o "la mejor". La eficiencia de un ArrayList para accesos directos, la velocidad de un LinkedList para inserciones y eliminaciones, la garantía de unicidad de un HashSet y la potencia de búsqueda instantánea de un HashMap son características que responden a necesidades distintas. El verdadero dominio emerge de la capacidad del ingeniero para **analizar un problema, identificar sus patrones de acceso a datos y seleccionar la herramienta que ofrezca el rendimiento óptimo.**

Desde una simple lista de compras hasta un sofisticado sistema de optimización de rutas para una empresa de logística en Bogotá, el principio es el mismo: las estructuras de datos son el esqueleto invisible que soporta la lógica del negocio. Un sistema rápido, escalable y robusto es, casi siempre, el resultado de una arquitectura de datos bien pensada. Como futuros profesionales de la ingeniería, esta habilidad para modelar y organizar la información será uno de sus activos más valiosos.

El viaje del aprendizaje no termina aquí. Este documento ha sentado las bases, pero el campo de las estructuras de datos y algoritmos es vasto y fascinante. Te invitamos a seguir explorando, a experimentar con tus propios proyectos y a medir el impacto de tus decisiones. No te conformes con que el código "funcione"; busca siempre que sea elegante, eficiente y profesional.

Para continuar tu reflexión, te dejamos dos preguntas abiertas:

1. En el desarrollo de un videojuego multijugador masivo en línea (MMO), ¿qué estructura de datos considerarías ideal para gestionar la posición de miles de jugadores en un mapa en tiempo real y por qué?
2. ¿Cómo impacta la elección de una estructura de datos en la escalabilidad y los costos de una aplicación desplegada en la nube (por ejemplo, en AWS o Azure), donde se paga por el tiempo de cómputo y el uso de memoria?

Referencias

Arangurí García, M. Y., & Bravo Jaico, J. L. (2021). *Metodología integral de casos de prueba sustentado en un modelo de verificación de requisitos para desarrollo de software comercial*. D - Universidad Señor de Sipán.

Campos, G. E. C., & Campos, C. A. C. (2023). *Fundamentos de programación y las bases de la construcción de aplicaciones informáticas*. Ediciones de la U.

Capacho, J., & Nieto Bernal, W. (2017). *Lógica y algoritmia*. Ediciones de la U.

Joyanes Aguilar, L. (2020). *Java a fondo: programación orientada a objetos, interfaces gráficas y aplicaciones web*. Alfaomega Grupo Editor.

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software: un enfoque práctico* (9.^a ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación.

Información general

Fecha de creación	29 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Patrones de Diseño Orientados a Objetos

Introducción

En el mundo del desarrollo de software, escribir código que funcione es solo el primer paso. El verdadero desafío, el que distingue a un programador funcional de un arquitecto de software competente, reside en construir soluciones que no solo resuelvan un problema hoy, sino que puedan crecer, adaptarse y evolucionar con el tiempo sin colapsar bajo su propio peso. Aquí es donde entran en juego los Patrones de Diseño.

Un patrón de diseño no es un fragmento de código que se pueda copiar y pegar, ni un algoritmo específico con pasos definidos. Es, en esencia, una solución general y reutilizable a un problema común dentro de un contexto particular en el diseño de software. Imagina que eres un arquitecto que necesita construir un puente. No partes de cero; te basas en principios y estructuras probadas a lo largo de los años —puentes de arco, colgantes, atirantados—, cada uno ideal para un tipo de terreno, distancia y carga. Los patrones de diseño son esos "planos" conceptuales para el software: estrategias probadas que ofrecen un vocabulario común para que los desarrolladores discutan y resuelvan problemas recurrentes de manera elegante y eficiente.

La justificación para aprender a utilizarlos es directa: el software es inherentemente complejo y costoso de mantener. Un sistema mal diseñado, aunque funcional al principio, se convierte rápidamente en una maraña de dependencias y código "espagueti" que hace que cualquier cambio sea una tarea titánica y arriesgada. Los patrones de diseño nos guían para construir componentes desacoplados, flexibles y cohesivos, reduciendo la complejidad y promoviendo la reutilización. Al dominar estos conceptos, dejas de ser un simple constructor de "muros de código" y te conviertes en un diseñador de sistemas resilientes. Estudiarlos es invertir en la calidad y longevidad de tu trabajo profesional.

Por ello, recuerda siempre que aprender patrones de diseño no es aprender a escribir código, es aprender a pensar como un arquitecto de software.

Objetivos

Objetivo General

Al finalizar el estudio de este documento, el estudiante estará en la capacidad de analizar problemas comunes que surgen durante el ciclo de vida del desarrollo de software y, a partir de dicho análisis, podrá seleccionar, implementar y justificar el uso de patrones de diseño apropiados de las categorías creacional, estructural y de comportamiento. Esto le permitirá tomar decisiones de diseño informadas para construir aplicaciones en Java que sean robustas, mantenibles y reutilizables.

Objetivos Específicos / Resultados de Aprendizaje Esperados

Para alcanzar el objetivo general, se espera que el estudiante logre lo siguiente:

- **1. Comprender los Fundamentos:**
 - **Identificar** qué es un patrón de diseño, diferenciándolo de un algoritmo o una librería.
 - **Describir** el propósito y los beneficios de utilizar patrones, como el aumento de la flexibilidad y la mantenibilidad del código.
 - **Clasificar** los patrones de diseño en sus tres categorías principales (Creacionales, Estructurales y de Comportamiento), explicando la finalidad de cada una.
- **2. Aplicar los Patrones:**
 - **Implementar** en Java los patrones de diseño creacionales estudiados (Singleton, Factory, Builder) para gestionar la creación de objetos de manera controlada.
 - **Construir** soluciones que utilicen patrones estructurales (Adapter, Composite, Decorator) para organizar clases y objetos en estructuras más grandes y flexibles.
 - **Aplicar** patrones de comportamiento (Observer, Strategy) para gestionar la comunicación y la asignación de responsabilidades entre objetos de forma eficaz.
- **3. Analizar y Justificar:**
 - **Analizar** un requisito o problema de software determinado y **comparar** las posibles soluciones que ofrecen diferentes patrones de diseño.
 - **Seleccionar** el patrón más adecuado para resolver dicho problema, basándose en criterios técnicos como el acoplamiento, la cohesión y la escalabilidad.
 - **Justificar** verbalmente o por escrito la elección de un patrón de diseño específico sobre otras alternativas, argumentando las ventajas de su implementación en ese contexto particular.

Fundamentos teóricos

Para construir un edificio robusto, un arquitecto no improvisa; utiliza planos y principios estructurales probados. De manera análoga, para construir software de calidad, un desarrollador no debería empezar de cero ante cada problema. Este capítulo establece los "planos conceptuales" que nos guiarán: los fundamentos de los patrones de diseño.

1.1. ¿Qué es un Patrón de Diseño?

En su nivel más básico, un patrón de diseño es una **descripción de la comunicación entre objetos y clases, personalizada para resolver un problema de diseño general en un contexto particular** (Sommerville, 2011).

Es crucial entender que no es código terminado, sino una plantilla o una guía. Todo patrón se define por tres componentes esenciales:

- **Contexto:** La situación recurrente en la que se presenta un problema de diseño. Por ejemplo, la necesidad de asegurar que una clase tenga una única instancia en todo el sistema.
- **Problema:** El conjunto de fuerzas o restricciones que actúan en ese contexto. Siguiendo el ejemplo, el problema es cómo garantizar el acceso global a esa única instancia sin permitir que otras partes del código la creen accidentalmente.
- **Solución:** Un modelo de diseño genérico, compuesto por un conjunto de clases y objetos y sus relaciones, que resuelve el problema de manera efectiva. La solución del ejemplo sería una clase que se encarga de su propia creación y garantiza que no se pueda instanciar más de una vez.

Esta estructura (Contexto-Problema-Solución) es lo que hace que un patrón sea una herramienta de pensamiento y no solo una receta. Nos obliga a analizar la situación antes de programar.

1.2. Propósito y Orígenes: La "Gang of Four" (GoF)

El concepto de patrones de diseño no se originó en la informática, sino en la arquitectura de edificios con el trabajo de Christopher Alexander en los años 70. Sin embargo, su popularización en el mundo del software se debe al icónico libro *Design Patterns: Elements of Reusable Object-Oriented Software*, publicado en 1994 por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, conocidos afectuosamente como la **"Gang of Four" (GoF)**.

Este libro no inventó los patrones, sino que los "descubrió", documentando 23 soluciones de diseño recurrentes y probadas que habían observado en diversos

proyectos de software. Al darles un nombre y un formato estandarizado, crearon un vocabulario compartido invaluable. El propósito de este catálogo no era imponer reglas, sino proporcionar un lenguaje común que permitiera a los desarrolladores comunicarse sobre problemas de diseño complejos de manera mucho más eficiente (Pressman & Maxim, 2021). Decir "usemos un Singleton aquí" es infinitamente más rápido y preciso que describir toda la estructura de la solución desde cero.

1.3. Principios de Diseño S.O.L.I.D. como Base

Los patrones de diseño no existen en el vacío. Son la manifestación práctica de principios de diseño de software más profundos. Los más importantes son los principios **S.O.L.I.D.**, un acrónimo que representa cinco directrices para crear software comprensible, flexible y mantenible. Entenderlos es clave para valorar por qué los patrones están diseñados como están.

Resumen de los Principios S.O.L.I.D.

Principio	Nombre	Descripción Breve	Analogía Cotidiana
S	Single Responsibility (Responsabilidad Única)	Una clase debe tener una, y solo una, razón para cambiar.	Una navaja suiza es útil, pero un chef prefiere un cuchillo específico para cada tarea (picar, filetear, etc.). Cada herramienta tiene una única responsabilidad.
O	Open/Closed (Abierto/Cerrado)	Las entidades de software (clases, módulos) deben estar abiertas para su extensión, pero cerradas para su modificación.	Un celular permite instalar nuevas apps (abierto a extensión) sin necesidad de modificar su sistema operativo interno (cerrado a modificación).
L	Liskov Substitution (Sustitución de Liskov)	Los objetos de una clase derivada deben poder sustituir a los objetos de la clase base sin alterar la corrección del programa.	Un hijo (clase derivada) puede tomar el lugar de su padre (clase base) en una tarea si sabe hacerla igual o mejor, sin causar problemas.
I	Interface Segregation (Segregación de la Interfaz)	Ningún cliente debe ser forzado a depender de métodos que no utiliza. Es mejor tener muchas interfaces específicas que una sola general.	En un restaurante, no te dan un manual con todas las recetas (interfaz general), solo el menú (interfaz específica) con lo que puedes pedir.
D	Dependency Inversion (Inversión de la Dependencia)	Los módulos de alto nivel no deben depender de los de bajo nivel. Ambos deben depender de abstracciones.	El control remoto de un TV (alto nivel) no depende del circuito interno del TV (bajo nivel). Ambos dependen de una interfaz estándar (los botones y las señales infrarrojas).

Tabla 1 Resumen de los principios S.O.L.I.D.

La tabla que resume los cinco principios S.O.L.I.D. del diseño de software con su nombre, descripción y una analogía.

Los patrones de diseño son, en muchos casos, implementaciones exitosas y estandarizadas de uno o más de estos principios.

1.4. Clasificación de los Patrones de Diseño

La GoF organizó los 23 patrones originales en tres categorías principales según su propósito. Esta clasificación sigue siendo el estándar para entender y organizar el vasto catálogo de patrones que existe hoy en día (Sommerville, 2011).

- **Patrones Creacionales:** Se centran en el proceso de **creación de objetos**. Su objetivo es abstraer la lógica de instanciación para que el sistema sea independiente de cómo se crean, componen y representan sus objetos. Patrones como *Factory Method*, *Singleton* o *Builder* pertenecen a esta categoría.
- **Patrones Estructurales:** Describen cómo **componer clases y objetos para formar estructuras más grandes** y complejas. Se ocupan de las relaciones entre entidades, facilitando que funcionen juntas de manera eficiente. *Adapter*, *Composite* y *Decorator* son ejemplos clave.
- **Patrones de Comportamiento:** Se dedican a la **comunicación entre objetos** y a la asignación de responsabilidades. Definen cómo los objetos interactúan y colaboran para realizar tareas que un solo objeto no podría hacer por sí solo. *Observer* y *Strategy* son ejemplos representativos.

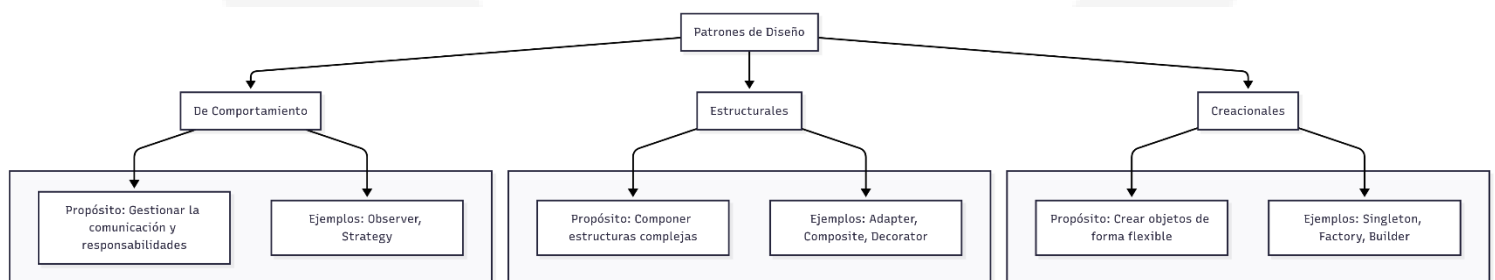


Figura 1 Clasificación de los Patrones de Diseño GoF.

Mapa conceptual que muestra las tres categorías de patrones de diseño (Creacionales, Estructurales y de Comportamiento) con su propósito y ejemplos.

1.5. Beneficios y Desventajas del Uso de Patrones

Como toda herramienta, los patrones de diseño deben usarse con criterio. Su aplicación indiscriminada puede llevar a una complejidad innecesaria (Pressman & Maxim, 2021).

Beneficios:

- **Soluciones Probadas:** Proporcionan soluciones que han sido validadas en innumerables sistemas.

- **Vocabulario Común:** Mejoran la comunicación dentro de los equipos de desarrollo.
- **Reutilización:** Fomentan el diseño de sistemas más modulares y reutilizables.
- **Mantenibilidad:** Un código basado en patrones es a menudo más fácil de entender y mantener.

Desventajas:

- **Complejidad Innecesaria:** Aplicar un patrón a un problema simple puede "sobre-diseñar" la solución, haciéndola más complicada de lo necesario.
- **Curva de Aprendizaje:** Requiere tiempo y práctica aprender a identificar cuándo y cómo aplicar cada patrón correctamente.
- **Malinterpretación:** Un patrón implementado incorrectamente puede ser peor que no usar ninguno.

Este capítulo ha sentado las bases de qué son los patrones, de dónde vienen, en qué principios se apoyan y cómo se clasifican. Con estos fundamentos teóricos, estamos listos para explorar cómo se manifiestan en el mundo real.

Aplicaciones en la vida cotidiana

Los patrones de diseño pueden parecer teóricos al principio, pero en realidad, son soluciones inspiradas en cómo resolvemos problemas en el mundo real. Una vez que aprendes a identificarlos, comienzas a verlos en todas partes: desde cómo pides un café hasta cómo funcionan las aplicaciones en tu celular. Este capítulo tiene como objetivo construir un puente entre la teoría abstracta y la experiencia tangible y cotidiana.

2.1. Analogías Cotidianas para Entender los Patrones

La mejor manera de internalizar un patrón es a través de una analogía simple. A continuación, exploraremos algunos de los patrones más comunes utilizando ejemplos de la vida diaria en el contexto colombiano.

- **Singleton (La Instancia Única): El Gerente del Banco de la República** En Colombia, solo hay un Gerente General del Banco de la República a la vez, y esta figura tiene la responsabilidad única de, por ejemplo, firmar la emisión de nuevos billetes. Cualquier entidad del gobierno que necesite una directriz monetaria oficial debe acudir a esta única instancia. El patrón **Singleton** funciona exactamente así: garantiza que una clase tenga una sola instancia y proporciona un punto de acceso global a ella. Es útil para gestionar recursos compartidos como configuraciones de aplicación, conexiones a bases de datos o, como en la analogía, una autoridad central.
- **Factory (La Fábrica): El Menú del Día o "Corrientazo"** Cuando vas a un restaurante y pides el "menú del día", no le dices al chef cómo preparar la sopa o cómo cocinar el pollo. Simplemente haces un pedido genérico ("quiero el almuerzo de hoy") y la cocina (la "fábrica") se encarga de crear y devolverte el plato específico que corresponde a ese día. El patrón **Factory** opera de forma similar. Le pides a una clase "fábrica" que te cree un objeto de un tipo general (ej. "Reporte"), y la fábrica decide qué objeto concreto devolver (un ReportePDF, ReporteExcel, etc.) según los parámetros o la configuración, sin que tú necesites conocer los detalles de la creación de cada uno.
- **Adapter (El Adaptador): El Enchufe para Viajar a Europa** Imagina que compras un portátil en Colombia con un enchufe de dos clavijas planas, pero viajas a España, donde las tomas de corriente son de dos clavijas redondas. Tu cargador es perfectamente funcional, pero su interfaz (las clavijas) es incompatible con la toma de corriente. La solución es simple: usas un **adaptador**. Este dispositivo se conecta a tu cargador por un lado y a la toma

europaea por el otro, "traduciendo" una interfaz para que sea compatible con la otra. El patrón **Adapter** hace precisamente esto en software: permite que dos clases con interfaces incompatibles colaboren, creando una clase intermedia que actúa como traductora.

- **Decorator (El Decorador): Personalizando tu Café en Juan Valdez**
Cuando pides un café en Juan Valdez, partes de una base: un tinto o un latte. A partir de ahí, puedes "decorarlo" añadiéndole funcionalidades (o sabores) extra: un poco de crema batida, un chorrito de arequipe, canela en polvo, etc. Cada adición envuelve a la anterior, añadiendo su propio sabor sin alterar el café base. El patrón **Decorator** permite añadir nuevas funcionalidades a un objeto dinámicamente, "envolviéndolo" en objetos decoradores. Es una alternativa flexible a la herencia para extender funcionalidades.
- **Observer (El Observador): Suscriptores de un Periódico como El Tiempo**
Cuando te suscribes a la versión digital de un periódico como El Tiempo, te conviertes en un "observador". El periódico es el "sujeto observado". Cada vez que el periódico publica una noticia importante (un cambio de estado), notifica automáticamente a todos sus suscriptores (los observadores) enviándoles un correo o una notificación push. Los suscriptores no tienen que estar preguntando al periódico "¿hay algo nuevo?" constantemente. El patrón **Observer** define esta relación de dependencia uno-a-muchos, donde los cambios en un objeto notifican y actualizan automáticamente a todos los que dependen de él.

2.2. Patrones en el Software que Usamos a Diario

Más allá de las analogías, los patrones de diseño son el esqueleto de muchas de las aplicaciones y tecnologías que utilizamos a diario. Identificar su uso en sistemas reales nos ayuda a comprender su valor práctico.

- **Observer en las Redes Sociales:** Cada vez que publicas una foto en Instagram o un estado en Facebook, tus seguidores reciben una notificación. Tú eres el "sujeto" y tus seguidores son los "observadores". El patrón **Observer** es la base del sistema de notificaciones de prácticamente todas las plataformas sociales modernas.
- **Factory en Frameworks de Desarrollo:** En entornos de desarrollo web como Angular o React, a menudo no creas los componentes directamente con la palabra clave new. En su lugar, utilizas métodos de un framework (una "fábrica") que se encargan de instanciar, configurar e inyectar las dependencias necesarias. Este uso del patrón **Factory** desacopla tu código

del proceso de creación de objetos, haciéndolo más modular y fácil de probar (Pressman & Maxim, 2021).

- **Decorator en las Librerías de Java I/O:** Una de las implementaciones más clásicas del patrón **Decorator** se encuentra en el paquete java.io de Java. Para leer un archivo de texto con un búfer de memoria (para mayor eficiencia), no usas una sola clase, sino que "decoras" una con otra:

```
// El FileInputStream es el objeto base  
// El BufferedReader es el "decorador" que añade la funcionalidad de búfer  
BufferedReader reader = new BufferedReader(new FileReader("miarchivo.txt"));
```

Aquí, `BufferedReader` envuelve a `FileReader` para añadirle la capacidad de leer en búfer, sin que `FileReader` tenga que saber nada al respecto. Esta composición de objetos es mucho más flexible que tener una clase monolítica para cada posible combinación de lectura de archivos.

Como vemos, los patrones de diseño no son solo diagramas UML en un libro de texto; son soluciones vivas que potencian la funcionalidad del software que da forma a nuestro mundo digital.

Casos de estudio

Después de entender la teoría y ver los patrones en la vida diaria, es hora de aplicarlos donde realmente marcan la diferencia: en el campo profesional. En esta sección, analizaremos desafíos de software comunes en empresas, desglosando el problema, la solución basada en patrones y el impacto medible de haber tomado esa decisión de diseño. Estos casos demuestran cómo los patrones no son un ejercicio académico, sino una herramienta estratégica para el desarrollo de software.

3.1. Patrones Creacionales: Sistema de Configuración y Reportes

Una *fintech* colombiana está desarrollando una aplicación de microcréditos. La aplicación necesita acceder a datos sensibles (claves de API para consultar centrales de riesgo, credenciales de la base de datos) que deben cargarse una sola vez al iniciar el sistema y estar disponibles globalmente. Además, la gerencia solicita un módulo que pueda generar reportes de cartera en diferentes formatos (PDF, Excel, CSV).

- **Desafío 1: Gestión de la Configuración Global**
 - **Problema:** Se debe garantizar que exista **una única instancia** del objeto de configuración en toda la aplicación. Múltiples instancias podrían llevar a inconsistencias, lecturas de datos obsoletos y un consumo de memoria innecesario. Crear un objeto global es una mala práctica que genera acoplamiento.
 - **Solución Aplicada:** Se implementó el patrón **Singleton**. Se creó una clase AppConfig que se autogestiona para asegurar que solo se pueda crear un objeto de sí misma. Proporciona un método estático getInstance() que devuelve siempre la misma instancia a cualquier parte del código que la necesite.
 - **Impacto:**
 - **Consistencia de Datos:** Se eliminó el riesgo de configuraciones duplicadas o desactualizadas.
 - **Rendimiento:** Se optimizó el uso de memoria al evitar la carga redundante del archivo de configuración.
 - **Seguridad:** Se centralizó el punto de acceso a datos sensibles, facilitando su control y auditoría.

- **Desafío 2: Creación Flexible de Reportes**

- **Problema:** El código cliente que solicita un reporte no debería estar acoplado a las clases concretas que lo generan (GeneradorPDF, GeneradorExcel). Si en el futuro se necesita un nuevo formato (ej. XML), habría que modificar el código cliente, violando el principio Abierto/Cerrado.
- **Solución Aplicada:** Se utilizó el patrón **Factory Method**. Se diseñó una interfaz Reporte y una clase creadora abstracta GeneradorReporte con un método crearReporte(). Luego, se crearon subclases concretas (GeneradorReportePDF, GeneradorReporteExcel) que implementan este método para devolver el tipo de objeto específico. El cliente solo necesita saber qué "fábrica" usar, no cómo se construye el producto final.
- **Impacto:**
 - **Extensibilidad:** Añadir nuevos formatos de reporte ahora solo requiere crear una nueva clase de "fábrica" y de "reporte", sin tocar el código existente. El sistema es mucho más fácil de mantener y escalar (Sommerville, 2011).
 - **Desacoplamiento:** El código de alto nivel (el que pide el reporte) no depende de los detalles de bajo nivel (las clases que lo generan).

3.2. Patrón Estructural: Integración de Pasarelas de Pago

Un popular *e-commerce* de moda en Colombia, que procesa pagos a través de la API de PayU, decide expandir sus opciones e integrar Wompi para llegar a más clientes. Sin embargo, las dos APIs son completamente diferentes: los nombres de los métodos, los parámetros requeridos y los formatos de respuesta son incompatibles.

- **Desafío: Unificar Interfaces Incompatibles**

- **Problema:** El sistema de carrito de compras está fuertemente acoplado a la forma en que funciona la API de PayU. Integrar Wompi directamente requeriría llenar el código con condicionales (if (esPayU) { ... } else if (esWompi) { ... }), lo que lo haría frágil, difícil de leer y de mantener.
- **Solución Aplicada:** Se implementó el patrón **Adapter**. Se creó una interfaz unificada IPasarelaPago con métodos genéricos como iniciarPago(monto, descripcion) y verificarEstado(idTransaccion). La clase existente que usaba PayU se refactorizó para implementar esta interfaz. Para Wompi, se creó una nueva clase WompiAdapter que también implementa IPasarelaPago. Internamente, los métodos de este adaptador "traducen" la llamada genérica a los métodos específicos que requiere la API de Wompi.

○ **Impacto:**

- **Código Limpio y Cohesivo:** El sistema de carrito de compras ahora interactúa con la interfaz IPasarelaPago sin saber qué motor de pagos se está utilizando por debajo.
- **Facilidad de Integración Futura:** Añadir una tercera pasarela de pago (ej. MercadoPago) en el futuro se vuelve una tarea trivial: solo se necesita crear un nuevo adaptador, sin modificar el núcleo del sistema.
- **Reutilización:** Se pudo reutilizar todo el código existente de la lógica de negocio sin modificaciones mayores (Pressman & Maxim, 2021).

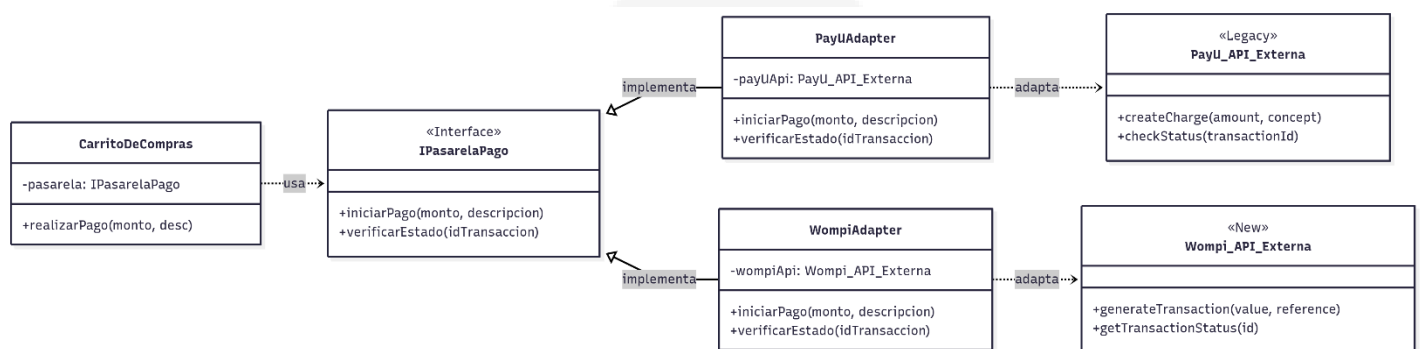


Figura 2 Diagrama UML del Patrón Adapter para Pasarelas de Pago

Diagrama de clases UML que muestra cómo la clase CarritoDeCompras depende de la interfaz IPasarelaPago, la cual es implementada por dos adaptadores (PayUAdapter y WompiAdapter) que traducen las llamadas a dos APIs externas e incompatibles.

3.3. Patrón de Comportamiento: Módulo de Promociones Dinámicas

Una cadena de supermercados (como Éxito o Jumbo) necesita implementar un nuevo módulo de promociones en su software de cajas registradoras (POS). Las promociones cambian constantemente: un día puede ser "20% de descuento en lácteos", otro día "Pague 2, lleve 3 en galletería", y durante el fin de semana "10% de descuento adicional si paga con la tarjeta del supermercado".

- **Desafío: Gestionar Algoritmos Intercambiables**

- **Problema:** Implementar cada lógica de descuento con una serie de if-else o switch dentro de la clase CarritoDeCompras la haría crecer descontroladamente. Cada nueva promoción requeriría modificar, probar y redespargar esta clase crítica, aumentando el riesgo de introducir errores.

- **Solución Aplicada:** Se utilizó el patrón **Strategy**. Se definió una interfaz `IEstrategiaDescuento` con un único método: `aplicarDescuento(total)`. Luego, cada tipo de promoción se encapsuló en su propia clase que implementa esta interfaz (ej. `DescuentoPorcentual`, `DescuentoPagueLleve`, `DescuentoTarjetaFidelidad`). La clase `CarritoDeCompras` tiene un objeto de tipo `IEstrategiaDescuento` y simplemente delega el cálculo del descuento a la estrategia que se le haya configurado en tiempo de ejecución.
- **Impacto:**
 - **Flexibilidad Máxima:** El departamento de mercadeo puede idear nuevas y complejas promociones, y el equipo de desarrollo solo necesita crear una nueva clase de estrategia e inyectarla en el sistema, a menudo sin detener la operación.
 - **Mantenibilidad y Pruebas:** Cada algoritmo de descuento está aislado en su propia clase, lo que permite probarlo de forma independiente y asegurar su correcto funcionamiento sin afectar el resto del sistema.
 - **Adherencia a Principios S.O.L.I.D.:** La solución respeta el principio Abierto/Cerrado (el carrito está cerrado a modificaciones pero abierto a nuevas estrategias) y el de Responsabilidad Única (el carrito no es responsable de cómo se calcula un descuento).

Herramientas y técnicas

Hemos explorado el "qué", el "porqué" y el "dónde" de los patrones de diseño. Ahora, nos adentraremos en el "cómo". Este capítulo es una guía práctica para traducir los diagramas UML y los conceptos teóricos en código Java funcional. Nos enfocaremos en la implementación, proporcionando ejercicios prácticos que te permitirán construir, paso a paso, soluciones elegantes y robustas utilizando el IDE NetBeans, una herramienta estándar en el ecosistema de desarrollo Java.

4.1. Implementación en Java con el IDE NetBeans

Para los ejercicios que siguen, necesitarás un entorno de desarrollo configurado.

- **Requisitos:**
 1. **JDK (Java Development Kit):** Asegúrate de tener instalada una versión reciente del JDK (versión 8 o superior).
 2. **Apache NetBeans:** Descarga e instala la última versión del IDE desde su sitio web oficial. Es gratuito y ofrece todas las herramientas necesarias para el desarrollo en Java.
- **Estructura del Proyecto:** Para mantener el código organizado, te recomendamos crear un único proyecto en NetBeans llamado `PatronesDeDiseno`. Dentro de este proyecto, crea tres paquetes principales (packages) para separar los ejercicios según la categoría del patrón:
 - `com.cun.patrones.creacionales`
 - `com.cun.patrones.estructurales`
 - `com.cun.patrones.comportamiento`

Esta estructura no solo es una buena práctica, sino que te ayudará a localizar y reutilizar tu código fácilmente. La ingeniería de software de calidad comienza con un proyecto bien organizado (Pressman & Maxim, 2021).

4.2. Ejercicio Práctico 1 (Estructural): Constructor de Pizzas

Una pizzería necesita un sistema para crear pizzas. Una pizza es un objeto complejo: puede tener diferentes tipos de masa (delgada, gruesa), salsa (tomate, BBQ), queso (mozzarella, parmesano) y múltiples ingredientes (jamón, champiñones, piña). Crear un objeto Pizza con un constructor que acepte todas estas combinaciones sería inviable y propenso a errores (`new Pizza("gruesa", "tomate", "mozzarella", true, true, false...)`).

- **Solución:** El patrón **Builder** es ideal para construir objetos complejos paso a paso. Separa la construcción del objeto de su representación final.
- **Guía de Implementación:**
 1. **Crea la clase Pizza (El Producto):** Dentro del paquete `creacionales`, crea una clase `Pizza`. Esta clase solo tendrá atributos (masa, salsa,

queso, etc.) y sus respectivos getters. Su constructor debe ser privado o de paquete para forzar su creación a través del Builder.

2. Crea la clase **PizzaBuilder** (El Constructor):

- Esta clase debe ser una clase anidada estática dentro de Pizza (ej. `public static class PizzaBuilder`).
- Tendrá los mismos atributos que la clase Pizza.
- Crea métodos para configurar cada atributo (ej. `public PizzaBuilder conMasa(String masa)`). Estos métodos deben devolver `this` (la propia instancia del Builder) para permitir el encadenamiento de llamadas.
- Crea un método final `build()` que creará y devolverá la instancia de Pizza usando los atributos configurados.

3. Implementación en el main:

// Ejemplo de uso

```
public static void main(String[] args) {  
    Pizza pizzaHawaiana = new Pizza.PizzaBuilder()  
        .conMasa("delgada")  
        .conSalsa("tomate")  
        .conQueso("mozzarella")  
        .conIngrediente("jamón")  
        .conIngrediente("piña")  
        .build();  
  
    System.out.println("Pizza creada: " + pizzaHawaiana);  
}
```

4.3. Ejercicio Práctico 2 (Creacional): Sistema de Archivos Simplificado

Se necesita modelar la estructura de un sistema de archivos, que contiene tanto archivos individuales como carpetas (directorios) que, a su vez, pueden contener más archivos y carpetas. Queremos tratar a un archivo simple y a una carpeta de la misma manera; por ejemplo, para calcular el tamaño total.

- **Solución:** El patrón **Composite** nos permite componer objetos en estructuras de árbol para representar jerarquías de parte-todo. Permite que los clientes traten a los objetos individuales y a las composiciones de objetos de manera uniforme.
- **Guía de Implementación:**
 1. **Crea la interfaz `ComponenteFS` (El Componente):** Dentro del paquete estructurales, crea una interfaz llamada `ComponenteFS`.

Debe declarar los métodos comunes, como String getNombre() y int getTamano().

2. **Crea la clase Archivo (La Hoja - Leaf):** Esta clase implementa ComponenteFS. Representa un objeto individual. Implementará los métodos getNombre() y getTamano() devolviendo sus valores propios.
3. **Crea la clase Carpeta (El Compuesto - Composite):**
 - Esta clase también implementa ComponenteFS.
 - Tendrá una lista de ComponenteFS (ej. private List<ComponenteFS> hijos = new ArrayList<>();).
 - Implementará métodos para gestionar sus hijos: void agregar(ComponenteFS componente) y void remover(ComponenteFS componente).
 - La implementación de getTamano() será recursiva: iterará sobre su lista de hijos, llamará a getTamano() en cada uno y sumará los resultados.
4. **Implementación en el main:**

// Ejemplo de uso

```
public static void main(String[] args) {  
    Carpeta raiz = new Carpeta("C:");  
    Carpeta documentos = new Carpeta("Documentos");  
    Archivo foto = new Archivo("vacaciones.jpg", 2048);  
    Archivo reporte = new Archivo("reporte.docx", 512);  
  
    documentos.agregar(reporte);  
    raiz.agregar(documentos);  
    raiz.agregar(foto);  
  
    // Calculamos el tamaño total desde la raíz, tratando todo uniformemente.  
    System.out.println("Tamaño total del disco: " + raiz.getTamano() + " KB");  
}
```

4.4. Ejercicio Práctico 3 (Comportamiento): Estación Meteorológica

Una estación meteorológica (el "sujeto") mide la temperatura y la humedad. Múltiples dispositivos (los "observadores") necesitan mostrar estos datos: una pantalla en la estación, una app en un celular y un registro en un sitio web. Cuando los datos del clima cambian, todos los dispositivos deben actualizarse automáticamente sin que la estación sepa los detalles de cada uno.

- **Solución:** El patrón **Observer** establece una dependencia de uno-a-muchos. Cuando el estado del sujeto cambia, todos sus observadores dependientes son notificados y actualizados.

- **Guía de Implementación (Java ya lo facilita):** Java proveía las clases `java.util.Observable` y la interfaz `java.util.Observer`, aunque ahora se prefieren implementaciones propias o librerías reactivas para mayor flexibilidad. Para este ejercicio, crearemos nuestra propia implementación simple para entender el mecanismo.
 1. **Crea las interfaces Sujeto y Observador:** En el paquete `comportamiento`, crea dos interfaces.
 - Sujeto: con métodos `registrarObservador(o)`, `removerObservador(o)` y `notificarObservadores()`.
 - Observador: con un único método `actualizar(temperatura, humedad)`.
 2. **Crea la clase `EstacionMeteorologica` (El Sujeto Concreto):**
 - Implementa la interfaz Sujeto.
 - Tendrá una lista de Observador y los atributos temperatura y humedad.
 - Cuando sus datos cambien (a través de un método `setMediciones()`), debe llamar a `notificarObservadores()`. El método `notificar...` recorrerá la lista y llamará a `actualizar()` en cada observador.
 3. **Crea clases de Display (Los Observadores Concretos):**
 - Crea `DisplayPantalla` y `DisplayCelular`, ambas implementando Observador.
 - El método `actualizar()` de cada una simplemente mostrará los nuevos datos del clima en la consola, simulando la actualización de la pantalla.
 4. **Implementación en el main:**

```
// Ejemplo de uso
public static void main(String[] args) {
    EstacionMeteorologica estacion = new EstacionMeteorologica();
    DisplayPantalla pantalla = new DisplayPantalla();
    DisplayCelular celular = new DisplayCelular();

    estacion.registrarObservador(pantalla);
    estacion.registrarObservador(celular);

    // Simulamos un cambio en el clima
    estacion.setMediciones(25.5f, 90.0f);
    // Salida esperada: La pantalla y el celular muestran los nuevos datos.
}
```

Estos ejercicios prácticos son la base para consolidar tu comprensión. La verdadera maestría en patrones de diseño se logra a través de la práctica y la reflexión constante sobre cómo mejorar la estructura del código que escribes (Sommerville, 2011).



Conclusión

A lo largo de este recorrido, hemos transitado desde los fundamentos teóricos de los patrones de diseño hasta su aplicación práctica en escenarios profesionales. Hemos descubierto que un patrón no es un simple bloque de código, sino una estrategia probada, un "plano" conceptual que nos permite construir software de una manera más inteligente y sostenible. Al dominar su uso, no solo resolvemos problemas técnicos, sino que también mejoramos nuestra capacidad para diseñar, comunicar y colaborar en equipos de desarrollo.

La verdadera esencia de los patrones de diseño radica en que nos enseñan a favorecer la flexibilidad sobre la rigidez, la composición sobre la herencia y el bajo acoplamiento sobre las dependencias monolíticas. Son la manifestación práctica de los principios S.O.L.I.D. y nos proporcionan un camino claro para escribir código que no solo funciona hoy, sino que está preparado para los cambios del mañana. Implementar patrones como Singleton, Factory, Adapter, Strategy u Observer no es una meta en sí misma; el objetivo es crear software de alta calidad que sea robusto, mantenible y escalable.

Ahora el desafío es tuyo. Te invito a no ver este documento como un punto final, sino como el comienzo de una nueva forma de pensar el diseño de software. La próxima vez que te enfrentes a un problema de programación, haz una pausa antes de escribir la primera línea de código y pregúntate: "¿Existe una solución probada para esto?". **El catálogo de patrones es un vocabulario compartido; úsalo para comunicarte mejor y construir mejor software.** La práctica constante te llevará de conocer la definición de un patrón a internalizar la sabiduría de diseño que contiene.

Para finalizar, te dejo dos preguntas abiertas que te animo a reflexionar:

1. *Mirando hacia atrás, ¿en qué proyecto académico o personal anterior podrías haber aplicado uno de los patrones de diseño estudiados para mejorar significativamente tu solución? ¿Qué problema específico habría resuelto?*
2. *Pensando en tu futuro profesional, ¿qué patrón de diseño consideras que será más útil para el tipo de aplicaciones o sector en el que te interesa trabajar (ej. desarrollo web, aplicaciones móviles, videojuegos, etc.) y por qué?*

Referencias.

Aedo, I., Montero, S., & Díaz, P. (2011). *Patrones de diseño aplicados al desarrollo de Objetos Digitales Educativos (ODE)*. Ministerio de Educación, Cultura y Deporte de España. <https://www-digitaliapublishing-com.zproxy.cun.edu.co/a/14712>

Montalvo Fránquiz, M. (2010). *Determinación de patrones que contribuyan a la formación de un equipo de proyecto de software, utilizando técnicas del modelo predictivo de la minería de datos*. D - Instituto Superior Politécnico José Antonio Echeverría. CUJAE. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/85833>

Montero, S. (2012). *Patrones de diseño aplicados al desarrollo de Objetos Digitales Educativos (ODE)*. Ministerio de Educación y Formación Profesional de España. <https://elibro-net.zproxy.cun.edu.co/es/lc/bibliocun/titulos/49278>

Pressman, R. S., & Maxim, B. R. (2021). *Ingeniería de software* (9.^a ed.). McGraw-Hill Interamericana. <https://www-ebooks7-24-com.zproxy.cun.edu.co/?il=16414>

Sommerville, I. (2011). *Ingeniería de software* (9.^a ed.). Pearson Educación. <https://www-ebooks7-24-com.zproxy.cun.edu.co/?il=3313>

Información general

Fecha de creación	29 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Desarrollo de Interfaces Gráficas con Orientación a Objetos

Introducción

Imagina que has desarrollado un motor increíblemente potente y eficiente, una verdadera obra de ingeniería. Sin embargo, para que alguien pueda usarlo, debe conectar cables manualmente, verter combustible con un embudo y accionar palancas en una secuencia precisa. ¿Sería útil en el día a día? Probablemente no. Le faltaría un chasis, un volante, pedales y un tablero de instrumentos; en resumen, una interfaz que permita a un conductor interactuar con esa potencia de forma simple y segura. En el desarrollo de software, ocurre exactamente lo mismo. Un programa, por más brillante que sea su lógica interna, necesita una "cara" amigable y funcional para ser verdaderamente útil y accesible: una **Interfaz Gráfica de Usuario (GUI)**.

Este documento te guiará en el fascinante proceso de construir esa "cara" para tus aplicaciones. Daremos un paso fundamental más allá de las aplicaciones de consola, donde la interacción se limita a texto secuencial, para adentrarnos en el mundo de las ventanas, los botones, los menús y los formularios interactivos. Descubrirás que los principios de la Programación Orientada a Objetos (POO) — encapsulamiento, herencia y polimorfismo— no son solo conceptos teóricos, sino herramientas increíblemente poderosas que nos permiten construir interfaces complejas de manera ordenada, modular y reutilizable. Cada botón, cada ventana, cada panel, será un objeto con su propio estado y comportamiento, dialogando con otros objetos para crear una experiencia de usuario fluida.

La relevancia de aprender a desarrollar interfaces gráficas es innegable en el mercado laboral actual. Desde el software de punto de venta en una tienda en Tocancipá, hasta complejos sistemas de gestión empresarial o herramientas de software especializado, la capacidad de crear una interacción visual efectiva es una habilidad esencial para cualquier desarrollador. No se trata solo de hacer que los programas "se vean bonitos", sino de construir puentes lógicos entre la complejidad del código y la necesidad de simplicidad del usuario final.

Lo que aprenderás aquí no solo sirve para aprobar una asignatura, sino para dar vida a tus ideas. **Aprenderás a construir aplicaciones que no solo funcionan, sino que también dialogan con el usuario, transformando código en experiencias interactivas.**

Objetivos

El propósito fundamental de este documento es dotar al estudiante de las competencias conceptuales y técnicas necesarias para diseñar, desarrollar e implementar interfaces gráficas de usuario funcionales y robustas en el lenguaje de programación Java, aplicando de manera coherente los paradigmas de la Programación Orientada a Objetos.

Al finalizar el estudio de este material, el estudiante estará en capacidad de:

- **Analizar** los requerimientos de una aplicación para proponer un diseño de interfaz de usuario que sea intuitivo y resuelva las necesidades de interacción del usuario final.
- **Diferenciar** las arquitecturas y componentes de las principales librerías gráficas de Java (Swing y JavaFX), seleccionando la más adecuada según las especificaciones del proyecto.
- **Construir** ventanas interactivas y visualmente organizadas, utilizando contenedores, componentes y administradores de diseño (Layout Managers) para crear estructuras adaptables.
- **Implementar** el modelo de delegación de eventos para capturar y responder a las acciones del usuario (clics, pulsaciones de teclas, etc.), conectando los elementos visuales con la lógica subyacente del programa.
- **Vincular** la capa de presentación (la Vista) con la lógica de negocio (el Modelo), aplicando patrones de diseño como el MVC (Modelo-Vista-Controlador) para garantizar un código desacoplado, mantenible y escalable.

En esencia, este módulo busca responder a la pregunta "¿Para qué aprendo esto?" demostrando cómo la POO se materializa en productos de software tangibles con los que las personas pueden interactuar de forma directa y efectiva.

Fundamentos teóricos

En el mundo del software, la primera impresión es fundamental. Un programa puede tener la lógica más robusta y el algoritmo más eficiente, pero si el usuario no puede interactuar con él de manera intuitiva y agradable, su valor se ve drásticamente reducido. Aquí es donde entran en juego las Interfaces Gráficas de Usuario (GUI). Este capítulo sienta las bases teóricas que te permitirán comprender cómo la Programación Orientada a Objetos (POO) no solo facilita, sino que potencia la creación de estas ventanas interactivas. Pasaremos de un modelo de ejecución secuencial, donde el programa tiene el control total, a un universo dinámico dirigido por las acciones del usuario.

1.1. Programación Dirigida por Eventos: El Usuario Toma el Control

La programación tradicional, como la que se utiliza en aplicaciones de consola, sigue un flujo predecible y lineal. El programa pide un dato, el usuario lo ingresa; el programa procesa, muestra un resultado y termina. En cambio, la programación de interfaces gráficas opera bajo un paradigma completamente diferente: la **Programación Dirigida por Eventos** (Event-Driven Programming).

En este modelo, el programa se encuentra en un estado de espera activa, listo para reaccionar a una multitud de posibles acciones del usuario. Estas acciones, conocidas como **eventos**, pueden ser un clic de ratón, la pulsación de una tecla, el movimiento del puntero o el cambio de tamaño de una ventana. El flujo del programa ya no es dictado por el programador, sino por el usuario. Como analogía, piensa en el personal de un restaurante de comida rápida: no preparan todos los platos en un orden fijo al inicio del día; en su lugar, **esperan y reaccionan** a los pedidos (eventos) que hacen los clientes a medida que llegan (Joyanes Aguilar, 2020).

1.2. La Anatomía de una GUI: Componentes, Contenedores y Layouts

Para construir una interfaz gráfica, trabajamos con bloques de construcción visuales. En la POO, cada uno de estos bloques es, naturalmente, un objeto con sus propias propiedades (atributos) y comportamientos (métodos). Estos elementos se organizan en tres categorías principales:

1. **Componentes (o Widgets):** Son los elementos interactivos individuales con los que el usuario interactúa directamente. Son los "átomos" de nuestra interfaz. Ejemplos claros son los botones (JButton), las etiquetas de texto (JLabel), los campos de entrada (JTextField) y las casillas de verificación (JCheckBox).

2. **Contenedores:** Son componentes especiales cuya función principal es albergar y organizar a otros componentes (incluidos otros contenedores). Actúan como los "marcos" o "cajas" donde colocamos nuestros elementos. El contenedor principal y más fundamental es la ventana (JFrame), pero también existen paneles (JPanel) que nos ayudan a agrupar componentes dentro de una ventana.
3. **Layout Managers (Administradores de Diseño):** Son objetos invisibles pero cruciales que definen la política de posicionamiento y dimensionamiento de los componentes dentro de un contenedor. En lugar de especificar coordenadas exactas (ej: "coloca este botón en la posición x=50, y=100"), lo cual es frágil y poco adaptable, un Layout Manager lo hace automáticamente. Por ejemplo, FlowLayout los coloca uno tras otro, mientras que BorderLayout los organiza en cinco regiones: norte, sur, este, oeste y centro. Usar Layout Managers garantiza que nuestras interfaces se vean bien en diferentes tamaños de pantalla y resoluciones, una tarea clave para un tecnólogo en desarrollo de software (Deitel & Deitel, 2012).

1.3. El Trío Dinámico: Fuente del Evento, Objeto Evento y Listener (Oyente)

El mecanismo que conecta las acciones del usuario con la respuesta de nuestro código se basa en una delegación de responsabilidades entre tres actores clave. Comprender esta relación es fundamental para el desarrollo de cualquier GUI.

- **Fuente del Evento (Event Source):** Es el componente de la GUI sobre el cual el usuario realiza la acción. Por ejemplo, un JButton es la fuente de un evento de clic.
- **Objeto Evento (Event Object):** Cuando la acción ocurre, la fuente crea un objeto que encapsula toda la información relevante sobre el evento. Para un clic, este objeto (ej:(ActionEvent)) contiene detalles como qué componente se presionó y cuándo.
- **Listener u Oyente (Event Listener):** Es un objeto que está "a la escucha", esperando que ocurra un evento en una fuente específica. Para que esto funcione, debemos registrar explícitamente el Listener en la Fuente. El Listener contiene el método que se ejecutará como respuesta al evento.

Este modelo se conoce como el **Modelo de Delegación de Eventos**. La fuente delega la tarea de manejar el evento al oyente.

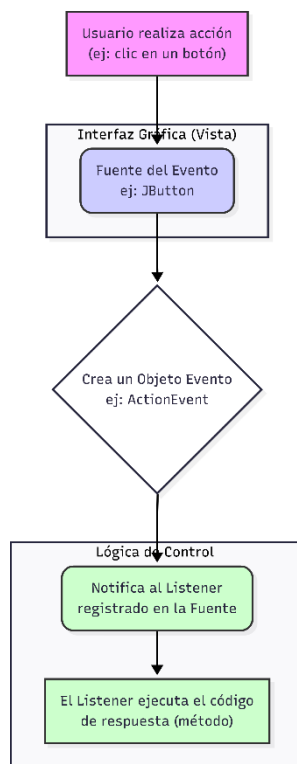


Figura 1 Diagrama del Modelo de Delegación de Eventos

Diagrama de flujo que muestra cómo una acción del usuario en un componente fuente genera un objeto evento que es enviado a un objeto listener para ejecutar una respuesta.

1.4. Librerías Gráficas en Java: De Swing a JavaFX

En el ecosistema de Java, existen principalmente dos bibliotecas para el desarrollo de GUIs de escritorio:

- **Swing:** Fue durante muchos años la librería estándar. Es robusta, multiplataforma y muy completa. Los nombres de sus clases suelen empezar con una "J" (e.g., JFrame, JButton). Una de sus ventajas es que los componentes son "ligeros", lo que significa que son dibujados enteramente por Java, dándoles una apariencia consistente en cualquier sistema operativo (Stallings, 2005).
- **JavaFX:** Es la evolución moderna de Swing. Fue diseñada para sucederla, ofreciendo un conjunto de herramientas más rico, soporte para propiedades, binding, efectos visuales y la capacidad de personalizar estilos de una manera mucho más sencilla y potente a través de CSS (Cascading Style Sheets). A nivel de un tecnólogo, es importante conocer ambas: Swing por su legado y la enorme cantidad de software existente, y JavaFX por ser la apuesta de futuro y ofrecer capacidades más avanzadas (Schildt, 2019).

1.5. Patrones de Diseño para GUI: Modelo-Vista-Controlador (MVC)

A medida que una aplicación crece, mezclar el código de la interfaz gráfica (la "vista") con la lógica de negocio (el "modelo") se convierte en una receta para el desastre. El mantenimiento se vuelve una pesadilla y la reutilización de código es casi imposible. Para resolver esto, se utiliza el patrón de diseño **Modelo-Vista-Controlador (MVC)**.

Este patrón propone separar la aplicación en tres componentes interconectados:

1. **Modelo (Model):** Representa los datos y la lógica de negocio de la aplicación. Es independiente de la interfaz de usuario. Por ejemplo, en una aplicación bancaria, el Modelo contendría las clases CuentaBancaria con métodos como depositar() y retirar().
2. **Vista (View):** Es la representación visual de los datos del Modelo. En nuestro caso, son las ventanas, botones y componentes de Swing o JavaFX. La Vista es responsable de mostrar la información al usuario.
3. **Controlador (Controller):** Actúa como intermediario entre el Modelo y la Vista. Recibe los eventos del usuario desde la Vista (como clics de botón) y los traduce en acciones a ejecutar sobre el Modelo. Una vez que el Modelo cambia, el Controlador puede notificar a la Vista para que se actualice (Sommerville, 2011).

Esta separación es clave para un desarrollo profesional, ya que permite que distintos desarrolladores trabajen en paralelo (unos en la Vista, otros en el Modelo) y facilita enormemente las pruebas y la evolución del software.

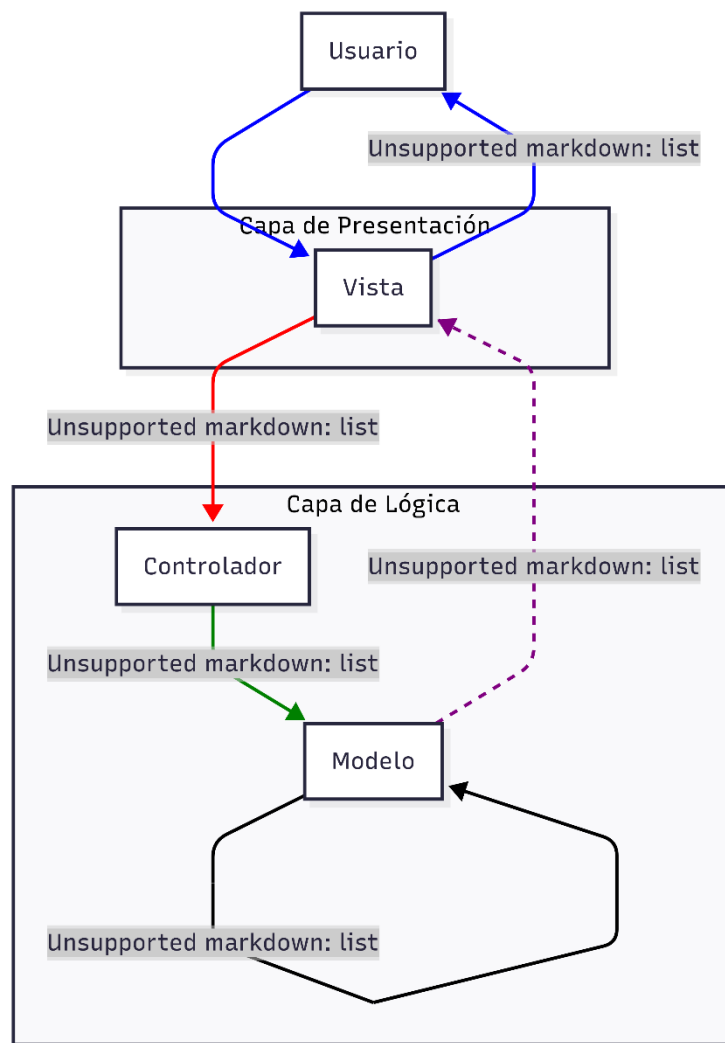


Figura 2 Diagrama del Patrón Modelo-Vista-Controlador (MVC)

Diagrama de bloques que ilustra la interacción entre el Usuario, la Vista, el Controlador y el Modelo en el patrón MVC.

Aplicaciones en la vida cotidiana

Los fundamentos teóricos del capítulo anterior son el cimiento, pero el verdadero aprendizaje ocurre cuando vemos esos conceptos en acción. Las Interfaces Gráficas de Usuario no son entidades abstractas; son el puente entre la lógica de un programa y las necesidades de una persona. Están en los teléfonos, en los supermercados, en los bancos y en las oficinas. Este capítulo está diseñado para "desarmar" digitalmente algunas de estas aplicaciones cotidianas, revelando cómo los componentes, contenedores y eventos que estudiamos se orquestan para crear experiencias funcionales en contextos muy familiares para cualquier persona en Colombia.

2.1. El Cajero Automático (ATM): Un Diálogo Guiado por Objetos

El cajero automático es, quizás, uno de los ejemplos más universales y claros de una aplicación de escritorio dirigida por eventos. Su éxito radica en que guía al usuario a través de un proceso complejo (transacciones financieras) de una manera simple y segura, paso a paso. Analicemos su interfaz desde la perspectiva de la POO:

- **La Ventana Principal (JFrame):** Toda la interacción ocurre dentro de una única ventana, un objeto JFrame que actúa como el contenedor principal. Esta ventana no se puede redimensionar y a menudo se ejecuta en pantalla completa para enfocar la atención del usuario.
- **Componentes de Interacción (JButton, JTextField):**
 - **Pantalla de Bienvenida:** Muestra una imagen y un texto (JLabel).
 - **Ingreso de Clave:** Aunque físicamente es un teclado numérico, en la lógica del software cada tecla es una **fuentes de eventos**. Al presionar un número, se genera un evento que un ActionListener captura para añadir el dígito a un campo de contraseña (un JPasswordField que enmascara los caracteres).
 - **Menú de Opciones:** La pantalla que nos pregunta "¿Qué desea hacer?" es un JPanel que contiene varios JButton ("Retiro", "Consulta de Saldo", "Transferencia"). Cada botón es un objeto independiente. Al hacer clic en "Retiro", solo el ActionListener asociado a ese botón específico se activa, llevando al usuario a la siguiente pantalla.
- **Flujo Dirigido por Eventos:** El cajero no hace nada hasta que el usuario actúa. La inserción de la tarjeta es el primer evento. El clic en un botón es el siguiente. El programa está en un bucle constante de "esperar-evento, procesar-evento". Esta arquitectura es robusta porque aísla las

funcionalidades. Un error en la lógica de "Transferencia" no afectará la funcionalidad de "Consulta de Saldo", ya que son manejados por listeners y clases distintas, un claro beneficio del encapsulamiento.

Esta interacción guiada y segura es un pilar de la experiencia de usuario (UX) y se logra gracias a la modularidad que nos brinda la POO (Sharp et al., 2019).

2.2. Punto de Venta (POS) en una Tienda: Eficiencia y Velocidad

Pensemos en la caja de una tienda de conveniencia como **Tiendas D1** o una panadería como **Tostao'**. La velocidad es crítica. El cajero no puede permitirse navegar por menús complejos. La GUI de un sistema de Punto de Venta (POS) está optimizada para la eficiencia y la reducción de errores.

- **Diseño Orientado a la Tarea:** La pantalla principal suele estar dividida por JPanels organizados con un GridLayout o GridBagLayout para maximizar el espacio. Un panel muestra los productos más vendidos como botones grandes con imágenes (JButton con ImageIcon). Otro panel contiene un resumen de la venta actual (una JTable). Un tercer panel tiene el teclado numérico y los botones de pago ("Efectivo", "Tarjeta", "Nequi").
- **Historia de Uso Cotidiano:**
 - *8:05 AM en Bogotá. Un cliente pide "un tinto y un pan de bono". La cajera, Ana, no necesita escribir nada. Simplemente presiona el botón con la imagen del café y luego el del pan de bono. Cada clic es un ActionEvent que un ActionListener central captura. Este listener invoca un método en el objeto "Pedido" (Modelo), que añade el producto y actualiza el total. La JTable que muestra la cuenta (Vista) se actualiza inmediatamente. El cliente paga con Nequi; Ana presiona el botón "Nequi", que abre una pequeña ventana de diálogo (JDialog) para confirmar la transacción. Todo el proceso toma menos de 10 segundos.*
- **Análisis Tecnológico:** La clave aquí es la **respuesta inmediata**. La separación del Modelo (la lógica del pedido, los precios) y la Vista (los botones) permite que la interfaz sea extremadamente rápida. Mientras la vista solo se encarga de mostrar botones y capturar clics, el modelo gestiona los cálculos en segundo plano. Este desacoplamiento es fundamental para el rendimiento en aplicaciones transaccionales de alto volumen.

2.3. Aplicaciones de Oficina: Gestión de Citas en una EPS

Las aplicaciones de escritorio siguen siendo vitales en entornos corporativos y de gestión, como en el sistema de agendamiento de una EPS en Colombia. Estas GUIs

manejan una mayor densidad de información y se enfocan en la organización y visualización de datos.

- **Componentes Complejos:** Estas interfaces utilizan componentes más avanzados. Por ejemplo, un calendario para seleccionar fechas (como el componente JCalendar de una librería externa), tablas (JTable) para mostrar listas de pacientes con barras de desplazamiento (JScrollPane), y formularios complejos con menús desplegables (JComboBox) para seleccionar especialistas o tipos de consulta.

- **Testimonio Ficticio (Recepcionista en una EPS de Cali):**

"Antes usábamos un cuaderno. Era un caos. Si un paciente cancelaba, tocaba tachar y buscar el hueco. Ahora, con el software, es diferente. Tengo la semana entera en la pantalla del calendario. Cuando alguien llama, busco al paciente por cédula en un campo de texto. Su historial aparece al instante. Hago clic en un día libre, selecciono la hora en un menú y listo, la cita queda agendada. El sistema me muestra alertas si hay un cruce de horarios. Me ahorra al menos una hora al día y hemos reducido los errores de agendamiento en un 90%. Para mí, esa pantalla es mi herramienta de trabajo principal."

- **Vinculación con la Lógica de Negocio:** Este ejemplo ilustra perfectamente la conexión entre la **Vista** (el formulario de citas) y el **Modelo** (las reglas de negocio: un médico no puede tener dos citas a la vez, una cita debe durar 20 minutos, etc.). Cuando el recepcionista presiona "Guardar Cita" (JButton), el Controlador toma los datos del formulario, los valida contra las reglas del Modelo y, si todo es correcto, los guarda en la base de datos. Si hay un error (como un cruce de horarios), el Modelo lo notifica al Controlador, que a su vez le pide a la Vista que muestre un mensaje de error (JOptionPane). Este flujo, basado en el patrón MVC, es lo que hace que la aplicación sea robusta y fiable (Domínguez Mínguez, 2022).

Casos de estudio

Mientras que las aplicaciones de la vida real nos muestran *qué* hacen las interfaces gráficas, un caso de estudio profesional nos revela *cómo* se construyen desde una perspectiva de ingeniería y gestión. Aquí es donde el rol del tecnólogo brilla, pues no solo se trata de escribir código, sino de tomar decisiones estratégicas, justificar el uso de una tecnología sobre otra y estructurar la solución de una manera que sea eficiente, mantenible y escalable. Este capítulo te sumergirá en un proyecto ficticio pero verosímil, desde la definición del problema hasta la medición del impacto, aplicando el rigor metodológico que demanda la industria.

3.1. Título del Caso: "SIGA-CUN: Sistema Integrado de Gestión Académica de Escritorio"

- **Cliente:** Una sede regional de la CUN con infraestructura de red intermitente.
- **Contexto:** La sede gestiona sus procesos de matrícula, inscripción de asignaturas y consulta de notas de forma manual, utilizando hojas de cálculo y documentos físicos. Esto genera inconsistencias, pérdida de datos y una gran sobrecarga administrativa al inicio de cada semestre.
- **Necesidad:** Requieren una aplicación de escritorio centralizada que pueda operar de manera local en la red de la oficina, sin depender de una conexión a internet constante, garantizando la seguridad y la rapidez en la consulta y gestión de la información estudiantil.

3.2. El Desafío: Requerimientos y Restricciones Clave

El equipo de desarrollo recibió un conjunto de requerimientos claros:

1. **Funcionalidad Principal:** El sistema debe permitir realizar CRUD (Crear, Leer, Actualizar, Borrar) para las entidades de Estudiantes, Docentes y Asignaturas. El módulo más crítico es el de matriculación, que debe permitir inscribir un estudiante en múltiples asignaturas, validando cupos y pre-requisitos.
2. **Operación Offline:** La principal restricción es que la aplicación debe ser 100% funcional dentro de la red local de la sede. La sincronización con un servidor central se podría hacer en lotes al final del día, pero la operación diaria no puede depender de internet.
3. **Seguridad:** Los datos de los estudiantes son sensibles. El acceso al sistema debe estar controlado por roles (Administrativo, Coordinador) y la base de datos debe residir en un servidor seguro dentro de la misma sede.
4. **Usabilidad:** El personal administrativo tiene distintos niveles de habilidad tecnológica. La interfaz debe ser extremadamente intuitiva, limpia y requerir una capacitación mínima.

3.3. La Solución Aplicada (Enfoque de Tecnólogo)

Tras analizar los requerimientos, el equipo de desarrollo tomó las siguientes decisiones estratégicas:

- **Selección de Tecnología:** Se optó por desarrollar una **aplicación de escritorio con Java y la librería Swing**.
 - **Justificación vs. Aplicación Web:** Aunque una aplicación web es más moderna, no cumplía el requisito de operación offline. Una aplicación de escritorio garantiza que la sede pueda seguir matriculando incluso si se cae el servicio de internet. Además, al ejecutarse en la JVM (Java Virtual Machine), la aplicación es multiplataforma por naturaleza, pudiendo correr en los equipos Windows o Linux de la institución.
- **Arquitectura de Software: Patrón Modelo-Vista-Controlador (MVC)**
 - Para garantizar la separación de responsabilidades y la mantenibilidad a futuro, se implementó rigurosamente el patrón MVC. Esta decisión es crucial desde el punto de vista tecnológico porque permite que el sistema evolucione. Por ejemplo, si en el futuro se decide crear una versión web, el **Modelo** (toda la lógica de negocio y acceso a datos) podría reutilizarse casi en su totalidad (Pressman & Maxim, 2021).

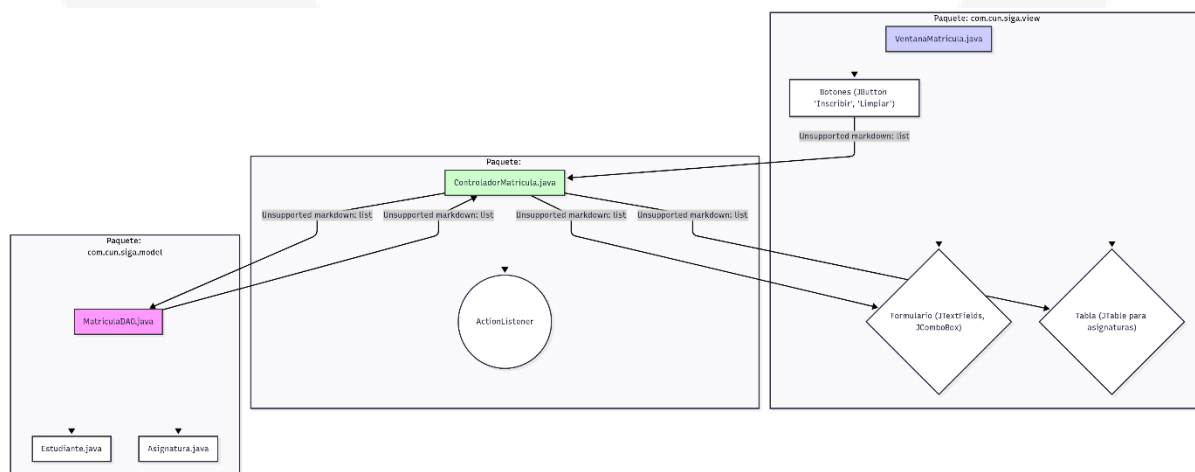


Figura 3 Arquitectura MVC aplicada al Módulo de Matriculación del SIGA-CUN

Diagrama de arquitectura MVC mostrando cómo la VentanaMatricula (Vista) se comunica con el ControladorMatricula, que a su vez manipula los objetos del Modelo como Estudiante y MatriculaDAO.

- **Implementación Orientada a Objetos:**
 - **Modelo:** Se crearon clases Estudiante, Docente y Asignatura que reflejaban directamente las entidades del mundo real (abstracción). Cada clase encapsulaba sus propios datos y métodos (ej:

estudiante.getNombre()). Se utilizó una clase MatriculaDAO (Data Access Object) para encapsular toda la lógica de interacción con la base de datos, haciendo el resto del código independiente de si se usaba MySQL, PostgreSQL u otra.

- **Vista:** Cada pantalla principal (ej: "Gestión de Estudiantes", "Proceso de Matrícula") se implementó como una clase que heredaba de JFrame o JPanel. Esto permitió reutilizar componentes comunes, como una barra de menú o un pie de página.
- **Controlador:** Se creó un Controlador específico para cada vista principal. El ControladorMatricula, por ejemplo, implementaba la interfaz ActionListener y se registraba con todos los botones de la VentanaMatricula. Su método actionPerformed contenía un switch o if-else para determinar qué botón se presionó y qué acción tomar.

3.4. Impacto y Resultados Cuantificables

Seis meses después de la implementación del **SIGA-CUN**, se realizó una evaluación de impacto con los siguientes resultados clave:

- **Reducción de Tiempo:** El tiempo promedio para completar una matrícula por estudiante, incluyendo la inscripción de todas sus asignaturas y la generación del recibo, **se redujo en un 40%**, pasando de 15 minutos a un promedio de 9 minutos.
- **Disminución de Errores:** Los errores de inscripción (cruces de horarios, inscripción en asignaturas sin cumplir pre-requisitos) **se eliminaron en un 98%**, ya que todas estas validaciones eran realizadas automáticamente por el Modelo antes de confirmar la operación.
- **Satisfacción del Personal:** Una encuesta interna mostró un **aumento del 75% en la satisfacción del personal administrativo** con sus herramientas de trabajo. La percepción de control, la reducción de estrés durante la temporada de matrículas y la fiabilidad de los datos fueron los puntos más valorados.

El éxito del proyecto validó la decisión de usar una arquitectura robusta y desacoplada, demostrando que una buena planificación y un diseño de software sólido son tan importantes como el código mismo (Larman, 2012).

Herramientas y técnicas

Saber qué construir es la mitad del trabajo; saber cómo construirlo eficientemente es la otra. Un desarrollador tecnológico no solo domina los conceptos, sino también las herramientas que le permiten materializar sus diseños de forma rápida y profesional. Este capítulo se centra en el "cómo". Exploraremos el entorno de desarrollo integrado (IDE) NetBeans y su potente diseñador visual de interfaces, que nos permite prototipar y construir GUIs de manera intuitiva. Luego, pondremos en práctica lo aprendido con ejercicios guiados que te llevarán desde la creación de tu primera ventana hasta la personalización de su apariencia, reforzando siempre las buenas prácticas que separan a un aficionado de un profesional.

4.1. El Entorno de Desarrollo: NetBeans y su Diseñador de GUI "Matisse"

Apache NetBeans es un IDE muy popular para el desarrollo en Java, y una de sus características más aclamadas es su editor de GUI, conocido internamente como "Matisse". Este editor es una herramienta de **Diseño Visual (WYSIWYG - What You See Is What You Get)**, que transforma la tarea de diseñar una ventana en un proceso similar a dibujar en un programa de diseño gráfico.

- **La Paleta de Componentes:** A un lado de la pantalla, NetBeans presenta una "Paleta" con todos los componentes de Swing (y JavaFX si está configurado). ¿Necesitas un botón? Simplemente lo arrastras desde la paleta y lo sueltas sobre tu JFrame o JPanel.
- **El Inspector de Propiedades:** Al seleccionar cualquier componente en el diseñador (un JButton, por ejemplo), el "Inspector" muestra todas sus propiedades: el texto (text), el color de fondo (background), el tamaño (preferredSize), etc. Puedes modificar estas propiedades directamente en el inspector, y el diseñador visual te mostrará el cambio en tiempo real.
- **Generación Automática de Código:** La verdadera magia de Matisse es que, mientras tú diseñas visualmente, NetBeans escribe el código Java correspondiente por ti. Genera todo el código de inicialización, posicionamiento (usando GroupLayout, su layout por defecto) y configuración de los componentes en un método llamado initComponents(). Este código está protegido (es de color gris y no editable) para evitar modificaciones accidentales que rompan el diseñador visual. Tú solo debes preocuparte por escribir el código que maneja los eventos.

Esta herramienta acelera drásticamente el desarrollo, pero es crucial entender el código que genera para poder depurar y optimizar la aplicación eficazmente (Farrell, 2019).

4.2. Ejercicio Práctico 1: Creando una Calculadora Básica con Swing

Este ejercicio es un rito de iniciación en el desarrollo de GUIs. Nos permitirá aplicar los conceptos de componentes, contenedores y, sobre todo, la gestión de eventos.

Guía Paso a Paso:

1. **Crear el Proyecto:** En NetBeans, crea un nuevo "Java with Ant" -> "Java Application".
2. **Crear el JFrame:** Haz clic derecho en tu paquete de fuentes -> "New" -> "JFrame Form". Nómbralo CalculadoraUI. NetBeans abrirá el diseñador visual.
3. **Diseñar la Interfaz:**
 - Arrastra un JTextField a la parte superior del JFrame. En sus propiedades, llámalo txtDisplay y borra su texto inicial.
 - Arrastra un JPanel debajo del campo de texto. En este panel irán los botones.
 - Configura el Layout Manager del JPanel a GridLayout. Un GridLayout de 4x4 es ideal para una calculadora estándar.
 - Arrastra 16 JButton dentro del JPanel. Edita sus propiedades de text para que muestren los números (0-9), los operadores (+, -, *, /), el igual (=) y un botón de limpiar (C).
4. **Implementar la Lógica (El ActionListener):**
 - Haz doble clic en el botón "1". NetBeans te llevará automáticamente al código y creará un método de evento para ti (jButton1ActionPerformed).
 - Dentro de este método, escribe el código para que el número "1" se añada al txtDisplay:

```
String textoActual = this.txtDisplay.getText();  
this.txtDisplay.setText(textoActual + "1");
```

- Repite este proceso para todos los botones de números y operadores. Para el botón "=", deberás implementar la lógica que evalúe la expresión matemática. Para un tecnólogo, un desafío interesante sería usar una librería como exp4j para evaluar la cadena de texto en lugar de implementar un parser manual (Ortega, 2020).

4.3. Ejercicio Práctico 2: Personalizando la Apariencia con UIManager

Una aplicación Swing por defecto puede verse algo anticuada. La clase UIManager nos permite cambiar el **"Look and Feel" (L&F)** de toda la aplicación con unas pocas líneas de código, para darle una apariencia más moderna o nativa del sistema operativo donde se ejecuta.

- **¿Cómo se implementa?** Justo al inicio de tu método main, antes de crear ninguna ventana, puedes insertar el siguiente código para intentar establecer el L&F del sistema operativo:

```
public static void main(String args[]) {  
    try {  
        // Establece el Look and Feel del sistema operativo actual (Windows, macOS,  
Linux GTK+)  
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());  
    } catch (ClassNotFoundException | InstantiationException |  
IllegalAccessException | UnsupportedLookAndFeelException ex) {  
  
java.util.logging.Logger.getLogger(CalculadoraUI.class.getName()).log(java.util.loggi  
ng.Level.SEVERE, null, ex);  
    }  
  
    /* Create and display the form */  
    java.awt.EventQueue.invokeLater(new Runnable() {  
        public void run() {  
            new CalculadoraUI().setVisible(true);  
        }  
    });  
}
```

Este simple cambio hace que la aplicación se sienta más integrada y profesional, una técnica de optimización de la experiencia de usuario muy valorada (Schildt, 2019).

4.4. Buenas Prácticas: Separando la Lógica del Diseño

El error más común al usar diseñadores visuales es poner toda la lógica del programa dentro de los métodos de evento de la ventana. Esto viola el principio de separación de responsabilidades y crea "código espagueti".

- **La Regla de Oro:** La clase de la Vista (tu `CalculadoraUI.java`) solo debe contener código relacionado con la apariencia y la manipulación de los componentes. **Nunca** debe contener la lógica de negocio.
- **Aplicación Práctica (Refactorizando la Calculadora):**
 1. Crea una nueva clase Java llamada `CalculadoraLogica.java` (el **Modelo**).
 2. Esta clase tendrá métodos como `agregarDigito(String digito)`, `agregarOperador(String op)` y `calcularResultado()`.
 3. En tu `CalculadoraUI.java` (la **Vista**), crea una instancia de `CalculadoraLogica`.

4. Ahora, el `actionPerformed` del botón "1" no manipulará directamente el `TextField`. En su lugar, llamará a `miCalculadoraLogica.agregarDigito("1");` y luego actualizará el `TextField` con el estado actual del modelo.

Esta separación, aunque requiere un poco más de código inicial, hace que el programa sea infinitamente más fácil de probar, depurar y extender en el futuro.



Conclusión

A lo largo de este recorrido, hemos transitado desde los fundamentos teóricos de la Programación Dirigida por Eventos hasta la construcción práctica de una aplicación con herramientas profesionales. Hemos desmitificado la "magia" detrás de las ventanas y botones, descubriendo que cada elemento de una interfaz gráfica es, en esencia, un objeto que colabora con otros dentro de un ecosistema bien definido. La Programación Orientada a Objetos se ha revelado no como un simple estilo de codificación, sino como el paradigma ideal para gestionar la complejidad inherente a las aplicaciones interactivas. Principios como el **encapsulamiento** nos permitieron crear componentes autónomos y seguros, la **herencia** facilitó la construcción de elementos especializados a partir de otros más generales, y el **polimorfismo**, a través de los listeners, nos dio la flexibilidad para que diferentes objetos respondan a un mismo tipo de evento de maneras únicas.

Hemos visto cómo esta teoría cobra vida en aplicaciones tan cotidianas como un cajero automático o el sistema de una tienda, demostrando que el software que usamos a diario se sostiene sobre estos pilares conceptuales. El caso de estudio del "SIGA-CUN" nos enseñó que las decisiones de un tecnólogo van más allá del código: implican analizar requerimientos, justificar tecnologías y, fundamentalmente, diseñar arquitecturas limpias como el **Modelo-Vista-Controlador (MVC)**, que garantizan que el software pueda crecer y adaptarse sin colapsar bajo su propio peso.

El verdadero aprendizaje, sin embargo, comienza ahora. La invitación es a que no te detengas en la calculadora de ejemplo. Mira a tu alrededor: la interfaz del microondas, la app de tu servicio de streaming, el software de la biblioteca. Intenta descomponerlos mentalmente. ¿Qué componentes usan? ¿Cómo manejarán los eventos? ¿Cómo estarán separados su Modelo y su Vista? Al aplicar este pensamiento crítico, transformarás tu manera de ver el software, pasando de ser un simple usuario a ser un arquitecto de experiencias. El desarrollo de interfaces gráficas es un campo que fusiona la lógica rigurosa con la empatía hacia el usuario, y dominarlo te abrirá innumerables puertas profesionales.

Para continuar tu viaje, te dejamos con dos preguntas abiertas que esperamos despierten tu curiosidad y te impulsen a investigar más allá de estas páginas:

1. En un mundo que se mueve cada vez más hacia las aplicaciones móviles y web, ¿qué nichos o industrias en el contexto colombiano consideras que seguirán dependiendo críticamente de las aplicaciones de escritorio robustas en los próximos cinco años y por qué?
2. Más allá de la funcionalidad, ¿cómo pueden los principios de diseño de experiencia de usuario (UX), como la simplicidad, la retroalimentación y la consistencia, ser implementados en el código de una aplicación Java Swing para crear no solo un programa que funcione, sino uno que sea un placer utilizar?

Referencias.

Deitel, P. J., & Deitel, H. M. (2012). *Java: How to Program*. Prentice Hall.

Domínguez Mínguez, T. (2022). *Tkinter. Desarrollo de interfaces gráficas en Python*. RA-MA S.A. Editorial y Publicaciones.

Farrell, J. (2019). *Java Programming*. Cengage Learning.

Joyanes Aguilar, L. (2020). *Fundamentos de Programación: Algoritmos, Estructuras de Datos y Objetos*. McGraw-Hill.

Larman, C. (2012). *Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development*. Prentice Hall.

Ortega, J. (2020). *Java para estudiantes*. IT Campus Academy.

Pressman, R. S., & Maxim, B. R. (2021). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.

Schildt, H. (2019). *Java: The Complete Reference*. McGraw-Hill Education.

Sharp, H., Preece, J., & Rogers, Y. (2019). *Interaction Design: Beyond Human-Computer Interaction*. John Wiley & Sons.

Sommerville, I. (2011). *Software Engineering*. Pearson.

Stallings, W. (2005). *Operating Systems: Internals and Design Principles*. Pearson/Prentice Hall.

Información general

Fecha de creación	29 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Persistencia de Datos en Aplicaciones Orientadas a Objetos

Introducción

Cuando desarrollamos una aplicación, ya sea un sistema de inventario, una red social o un videojuego, creamos objetos que viven y operan en la memoria RAM del computador. Estos objetos guardan información crucial: los datos de un cliente, los productos en un carrito de compras, o el puntaje de un jugador. Sin embargo, esta memoria es volátil. En el momento en que la aplicación se cierra o el sistema se apaga, toda esa información se desvanece como si nunca hubiera existido. Aquí es donde surge una pregunta fundamental para cualquier desarrollador de software: ¿cómo podemos hacer que la información de nuestros objetos perdure en el tiempo? La respuesta a esta pregunta es la **persistencia de datos**.

La persistencia de datos es el conjunto de principios y técnicas que permiten que el estado de un objeto sobreviva al proceso que lo creó. En términos simples, es el mecanismo para guardar la información de la memoria (que es temporal) en un medio de almacenamiento permanente, como un archivo en el disco duro o una base de datos. Esta capacidad es la que transforma un programa simple en una aplicación robusta y funcional. Sin persistencia, no existirían los perfiles de usuario, el guardado de progreso en los juegos, las transacciones bancarias en línea o el comercio electrónico. Cada vez que abriéramos una aplicación, empezaríamos desde cero.

Dominar la persistencia de datos es, por tanto, una habilidad no negociable en el desarrollo de software moderno. Nos permite construir sistemas que recuerdan, que aprenden del pasado y que ofrecen una experiencia continua y fiable al usuario. Este documento está diseñado para guiarte a través de los conceptos, herramientas y buenas prácticas para implementar soluciones de persistencia en tus aplicaciones orientadas a objetos. No se trata solo de guardar datos, sino de hacerlo de manera eficiente, segura y escalable. A lo largo de estos capítulos, descubrirás cómo darle memoria a tus aplicaciones, transformando programas efímeros en soluciones de software robustas y permanentes.

Objetivos

El propósito de este módulo es que, al finalizar, puedas diseñar e implementar soluciones de persistencia de datos en aplicaciones orientadas a objetos, seleccionando la estrategia más adecuada (archivos, bases de datos relacionales o no relacionales) según los requerimientos del problema. Este conocimiento es la base para construir aplicaciones completas y profesionales, capaces de gestionar información de manera efectiva a lo largo del tiempo.

Objetivos Específicos

Al completar el estudio de este documento, estarás en capacidad de:

- **Comprender** el "desajuste de impedancia objeto-relacional", identificando por qué es un desafío central al conectar la lógica de la programación orientada a objetos con las bases de datos tradicionales.
- **Implementar** las operaciones básicas de persistencia, conocidas como CRUD (Crear, Leer, Actualizar y Borrar), utilizando un framework de Mapeo Objeto-Relacional (ORM) en un proyecto Java.
- **Utilizar** formatos de intercambio de datos estándar como JSON y XML para la serialización y persistencia de objetos en archivos.
- **Modelar** diagramas de clases que representen entidades del mundo real y que puedan ser mapeados de manera coherente a un esquema de base de datos relacional.

Fundamentos teóricos

Para construir aplicaciones de software complejas y útiles, no basta con darles un comportamiento inteligente; también debemos dotarlas de memoria. Este capítulo explora los conceptos fundamentales que nos permiten almacenar y recuperar la información de nuestros objetos, asegurando que los datos sobrevivan más allá de una simple sesión de uso. Analizaremos desde las estrategias más clásicas hasta los desafíos que dieron origen a las herramientas modernas que usamos hoy en día.

3.1. ¿Qué es la Persistencia de Datos?

En la programación orientada a objetos (POO), todo gira en torno a los objetos. Un objeto es una entidad que tiene un **estado** (sus atributos o datos) y un **comportamiento** (sus métodos o las acciones que puede realizar). Por ejemplo, un objeto Estudiante tiene un estado (nombre: "Ana", código: "12345") y un comportamiento (matricularMateria(), calcularPromedio()).

3.2. Estrategias Clásicas de Persistencia

Dos de las estrategias más comunes son la serialización y el uso de archivos planos. La **serialización** convierte el estado de un objeto en una secuencia de bytes que puede ser almacenada. Por otro lado, los **archivos de texto plano (CSV)** o los **lenguajes de marcas (XML)** permiten estructurar la información de manera legible (Toharia Rabasco & Raya González, 2015).

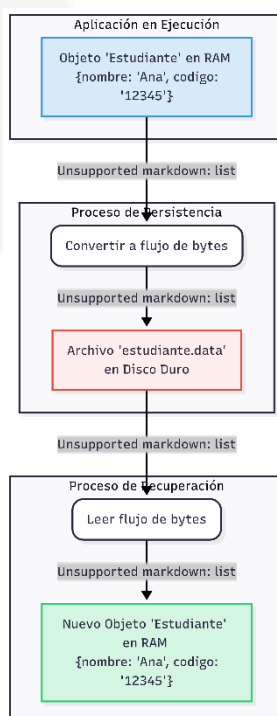


Figura 1 Diagrama de Flujo del Proceso de Serialización y Deserialización
Fragmento de código

Diagrama que muestra un objeto en RAM siendo serializado a un archivo en disco, y luego leído y deserializado de vuelta a un objeto en RAM.

3.3. El Mundo de las Bases de Datos

Cuando las aplicaciones necesitan gestionar grandes volúmenes de datos, entran en juego las bases de datos, sistemas especializados en almacenar y consultar datos de forma eficiente (Kroenke, 2003). Se dividen en dos grandes familias:

- **Bases de Datos Relacionales (SQL):** Organizan los datos en tablas, filas y columnas, utilizando el Lenguaje de Consulta Estructurado (SQL) para su manipulación (Martín Escofet, 2002).
- **Bases de Datos No Relacionales (NoSQL):** Ofrecen modelos de datos más flexibles, ideales para datos que no encajan bien en una estructura de tabla rígida.

3.4. El Desafío Central: Desajuste de Impedancia Objeto-Relacional

El problema fundamental al conectar la POO con las bases de datos relacionales es la diferencia entre el paradigma de objetos (con herencia, polimorfismo) y el paradigma relacional (plano y tabular). Este conflicto se conoce como el **desajuste de impedancia objeto-relacional**. La investigación en esta área ha buscado proponer modelos formales que faciliten esta "traducción" (Hernández González, 2009).

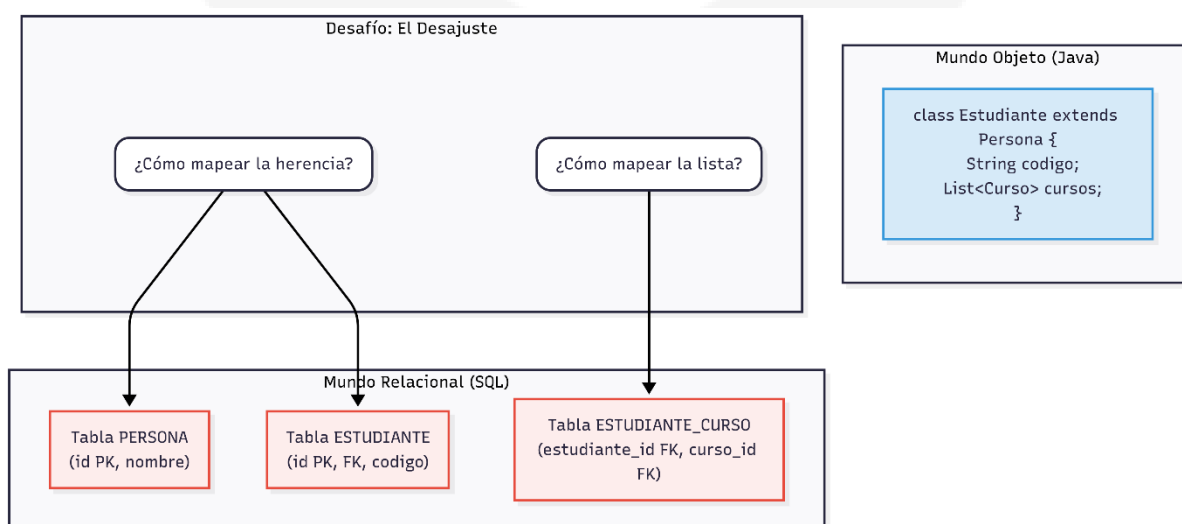


Figura 2 Mapa Conceptual del Desajuste de Impedancia Objeto-Relacional

Mapa conceptual que compara una estructura de clases con herencia y listas en Java con su compleja representación en múltiples tablas SQL.

3.5. Introducción a los ORM (Object-Relational Mapping)

Para resolver el problema del desajuste de impedancia, surgieron herramientas llamadas **ORM (Mapeo Objeto-Relacional)**. Un ORM es un software que actúa como un "traductor" automático entre el paradigma orientado a objetos de tu aplicación y el paradigma relacional de la base de datos. En lugar de escribir sentencias SQL a mano (Camuña Rodríguez, 2025), trabajas directamente con tus objetos, y el ORM se encarga de generar el código necesario para las operaciones CRUD (Crear, Leer, Actualizar, Borrar).

Aplicaciones en la vida cotidiana

La teoría de la persistencia de datos solo cobra sentido cuando la vemos en acción. Los conceptos como serialización, bases de datos SQL y NoSQL no son ideas abstractas, sino herramientas que resuelven problemas cotidianos para millones de personas. En este capítulo, exploraremos tres escenarios prácticos que ilustran cómo diferentes estrategias de persistencia se aplican en el software que usamos a diario. Desde guardar una simple configuración hasta gestionar complejas transacciones comerciales, veremos cómo la persistencia es el motor silencioso que hace que la tecnología funcione para nosotros.

2.1. Persistencia con Archivos: El Caso de la Configuración de una App

Imagina que descargas una aplicación de escritorio en tu computador, como un IDE de programación (NetBeans) o un videojuego. La primera vez que la abres, eliges el idioma en español, seleccionas el "modo oscuro" y ajustas el tamaño de la letra. Cierras la aplicación y, al día siguiente, la vuelves a abrir. Mágicamente, todas tus preferencias siguen ahí. ¿Cómo es posible?

- **El Problema:** La aplicación necesita "recordar" las elecciones del usuario entre sesiones. Estos datos de configuración son únicos para cada usuario pero no son complejos ni requieren ser consultados por miles de personas a la vez.
- **La Solución Cotidiana:** La estrategia más eficiente para este caso es la persistencia en archivos. La aplicación crea un objeto, por ejemplo `ConfiguracionUsuario`, que contiene atributos como `idioma="es"`, `tema="oscuro"`, `tamanoFuente=14`. Cuando cierras la aplicación, esta serializa el objeto `ConfiguracionUsuario` y lo guarda en un archivo local en tu computador, comúnmente en formato **JSON** (JavaScript Object Notation) o **XML** (eXtensible Markup Language). Estos lenguajes de marcas son fundamentales para los sistemas de gestión de información modernos (Toharia Rabasco & Raya González, 2015).
- **Análisis de Formatos:**
 - **JSON:** Es preferido por su simplicidad y legibilidad. Es nativo para JavaScript, lo que lo hace ideal para aplicaciones web, pero su simplicidad lo ha convertido en un estándar para archivos de configuración en muchos entornos.

JSON

```
{ "idioma": "es-CO", "tema": "oscuro", "tamanoFuente": 14 }
```

- **XML:** Es más verboso, pero también más potente para describir estructuras de datos complejas y metadatos, gracias a su sistema de

etiquetas y atributos. La validación de la estructura de un archivo XML contra un esquema (XSD) es una de sus grandes fortalezas en entornos empresariales (Kim-Park, 2011).

XML

```
<configuracionUsuario>  
  <idioma>es-CO</idioma>  
  <tema>oscuro</tema>  
  <tamanoFuente>14</tamanoFuente>  
</configuracionUsuario>
```

- **¿Por qué funciona tan bien?** Es una solución ligera, rápida y no requiere instalar un sistema de base de datos completo. Al iniciar, la aplicación simplemente lee este archivo, lo deserializa y reconstruye el objeto de configuración en memoria. Es el enfoque perfecto para datos de bajo volumen y específicos del entorno local.

2.2. El Carrito de Compras de un E-commerce Colombiano

Pensemos en una tienda online local, como "Cafés de mi Tierra", que vende café especial de diferentes regiones de Colombia.

- **El Problema:** El estado del carrito de compras debe persistir a través de diferentes sesiones y dispositivos. Además, el sistema debe controlar el inventario de forma concurrente para asegurar que el café que un cliente quiere comprar esté disponible.
- **La Solución con Base de Datos Relacional (SQL):** Un archivo JSON no es suficiente aquí. Necesitamos un sistema transaccional robusto que garantice la integridad de los datos. La solución ideal es una **base de datos relacional** (como PostgreSQL o SQL Server). El **diseño de la base de datos** es el pilar de la aplicación (Capacho & Nieto Bernal, 2017).
- **Interacciones y Transacciones:**
 1. **Añadir al Carrito:** Cuando un cliente añade un producto, se ejecuta una sentencia INSERT en una tabla CARRITO_ITEMS, asociando el id_usuario, id_producto y la cantidad.
 2. **Ver Carrito:** Se ejecuta una consulta SELECT con un JOIN entre las tablas CARRITO_ITEMS y PRODUCTOS para mostrar al usuario la descripción y el precio de los ítems que ha seleccionado.
 3. **Finalizar Compra (La Transacción Crítica):** Este es el momento más importante. El sistema debe ejecutar varias operaciones como un todo indivisible. Se inicia una **transacción** que:
 - Verifica el stock del producto en la tabla PRODUCTOS (SELECT ... FOR UPDATE para bloquear la fila y evitar que otro la compre al mismo tiempo).

- Si hay stock, crea un registro en la tabla PEDIDOS.
- Inserta los detalles del carrito en la tabla DETALLES_PEDIDO.
- Actualiza el inventario en la tabla PRODUCTOS (UPDATE PRODUCTOS SET stock = stock - cantidad ...).
- Elimina los ítems del CARRITO_ITEMS. Si cualquiera de estos pasos falla, la transacción completa se revierte (*rollback*), y la base de datos vuelve al estado anterior, evitando inconsistencias como vender un producto sin stock. El **lenguaje SQL** es la herramienta que permite estas operaciones complejas (Martín Escofet, 2002).

2.3. Gestión de Pacientes en un Centro de Salud en Bogotá

Una enfermera jefe en un centro de salud necesita acceder rápidamente al historial de un paciente.

- **El Problema:** Los datos son heterogéneos y no tienen una estructura fija. Un paciente puede tener una radiografía, otro puede tener 20 resultados de laboratorio, y un tercero puede tener notas de múltiples especialistas. Forzar esto en una estructura relacional rígida llevaría a tener tablas con cientos de columnas, la mayoría vacías para casi todos los pacientes, lo cual es ineficiente y difícil de mantener.
- **La Solución con Base de Datos No Relacional (NoSQL):** Este es un escenario perfecto para una **base de datos documental** como MongoDB. Cada paciente se almacena como un "documento" (en formato BSON, similar a JSON) que contiene toda su información.
- **Ventajas de la Flexibilidad:**
 - **Esquema Dinámico:** Si mañana se introduce un nuevo tipo de prueba genética, no es necesario alterar el diseño de la base de datos (ALTER TABLE). Simplemente, los nuevos pacientes tendrán un nuevo campo en su documento.
 - **Rendimiento en Lectura:** Toda la información de un paciente se puede recuperar en una sola operación de lectura, sin necesidad de realizar complejos JOINS entre 10 o 15 tablas diferentes, lo cual es crucial en una situación de urgencia.
 - **Escalabilidad Horizontal:** A medida que el hospital crece, se pueden añadir más servidores para distribuir la carga, una característica nativa de muchas bases de datos NoSQL.

Este enfoque flexible se alinea mucho mejor con la naturaleza semi-estructurada de los datos del mundo real en muchos dominios.

Casos de estudio

Aquí analizamos cómo se resuelven los problemas de persistencia de manera sistemática y escalable en un proyecto de software empresarial.

3.1. Caso de Estudio: Sistema de Gestión y Reservas para una Flota de Transporte Intermunicipal

- “Transportes Andinos S.A.S.” enfrentaba pérdidas económicas directas por su sistema manual. La sobreventa de tiquetes resultaba en compensaciones a clientes y daño a la reputación. La falta de datos centralizados impedía optimizar las rutas más rentables. La asignación de conductores y buses era un rompecabezas logístico que consumía horas de trabajo administrativo y era propenso a errores humanos costosos (ej. asignar un bus en mantenimiento a una ruta larga).
- **Solución Aplicada: Una Arquitectura por Capas** El equipo de desarrollo optó por una arquitectura de 3 capas, un estándar en la industria para organizar la complejidad:
 1. **Capa de Presentación:** Se desarrollaron dos interfaces: una aplicación web para que los clientes consulten rutas y compren tiquetes, y una aplicación de escritorio para los agentes de ventas en las terminales.
 2. **Capa de Lógica de Negocio:** Aquí reside el "cerebro" del sistema. Se crearon servicios como ReservaService y RutaService. Por ejemplo, ReservaService contenía el método crearReserva(datosReserva), que orquestaba todas las reglas de negocio: verificar la disponibilidad, validar el pago y confirmar la reserva.
 3. **Capa de Acceso a Datos (Persistencia):** Esta capa es la única que se comunica directamente con la base de datos. Se implementó usando el **patrón DAO (Data Access Object)**. Se crearon interfaces como IReservaDAO y sus implementaciones con Hibernate, como ReservaDAOImpl. Esto desacopla la lógica de negocio de la tecnología de persistencia. Mañana, si se decidiera cambiar de Hibernate a otra tecnología, solo habría que cambiar la implementación del DAO, sin tocar la capa de negocio. Este diseño se basa en **modelos de persistencia formales** que separan el qué del cómo (Hernández González, 2009).

Diseño de la Base de Datos y Mapeo: El equipo de bases de datos, siguiendo principios de **diseño de base de datos** (Capacho & Nieto Bernal, 2017), creó un esquema relacional normalizado en SQL Server (Wanumen Silva, 2018). Luego, los desarrolladores de la capa de persistencia usaron las anotaciones de JPA/Hibernate para mapear cada clase del dominio (Figura 3) a las tablas correspondientes.

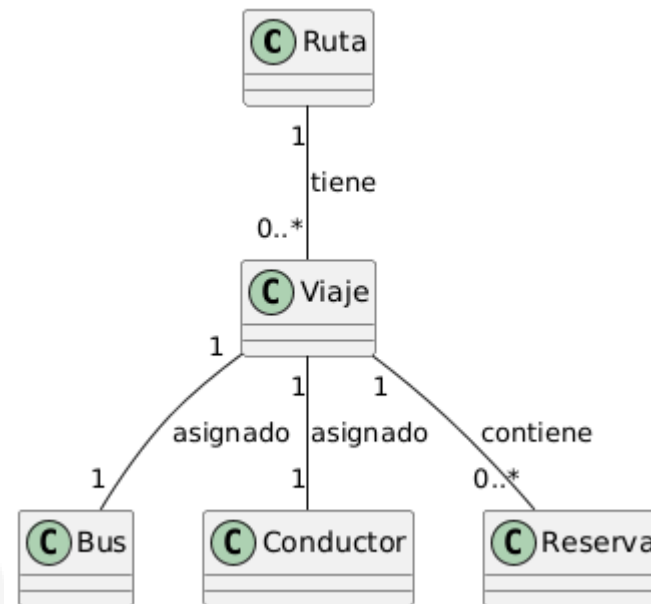


Figura 3 Diagrama de Clases UML Simplificado del Sistema de Reservas

Diagrama UML con las clases Ruta, Viaje, Bus, Conductor y Reserva, mostrando sus relaciones.

- **Desafíos Durante la Implementación:**
 - **Migración de Datos:** El mayor reto fue migrar los datos históricos desde hojas de cálculo a la nueva base de datos relacional. Se tuvieron que crear scripts para limpiar, transformar y cargar miles de registros, asegurando su consistencia.
 - **Gestión del Cambio:** La resistencia del personal acostumbrado al sistema manual fue un desafío. Se requirió un plan de capacitación intensivo para asegurar la adopción de la nueva herramienta.
- **Impacto Medible y Cualitativo:**
 - **Cuantitativo:** Además de la reducción del 95% en sobreventas, el sistema permitió un **análisis de rentabilidad por ruta**, lo que llevó a la optimización de horarios, resultando en un aumento del 15% en la ocupación promedio de los buses. La automatización de la asignación de vehículos redujo los costos administrativos en un 40%.
 - **Cualitativo:** La percepción de la marca mejoró notablemente. Los clientes ahora veían a "Transportes Andinos" como una empresa moderna y fiable. Internamente, la moral de los empleados aumentó al eliminar tareas manuales tediosas y propensas a conflictos.

Herramientas y técnicas

Este capítulo es una guía práctica para comenzar a trabajar con persistencia en Java.

4.1. Mapeo Objeto-Relacional con Hibernate (ORM)

Hibernate es el puente entre nuestro código Java y la base de datos relacional.

- **Configuración y SessionFactory:** La configuración en hibernate.cfg.xml es leída al inicio de la aplicación para crear un objeto SessionFactory. Este objeto es pesado y costoso de crear, por lo que se crea una sola vez y se reutiliza durante toda la vida de la aplicación.
- **EntityManager y la Sesión:** De la SessionFactory obtenemos un EntityManager (o Session), que representa una conversación con la base de datos. Es un objeto ligero que se crea y se destruye para cada unidad de trabajo.
- **Ejercicio Práctico Guiado (CRUD):**
 1. **La Entidad Estudiante:**

Java

```
@Entity @Table(name = "estudiantes")
public class Estudiante {
    @Id @Column(name = "codigo_estudiante")
    private String codigo;
    @Column(name = "nombre_completo")
    private String nombre;
    private String carrera;
    // Getters, Setters, Constructores...
}
```

2. El Código de Persistencia:

Java

```
// Suponiendo que tenemos una clase HibernateUtil para gestionar la
SessionFactory
EntityManager em = HibernateUtil.getEntityManager();

// CREATE
em.getTransaction().begin();
```

```
Estudiante nuevo = new Estudiante("202401", "Carlos Moncada", "Ingeniería de Software");  
em.persist(nuevo);  
em.getTransaction().commit();
```

```
// READ
```

```
Estudiante encontrado = em.find(Estudiante.class, "202401");  
System.out.println("Leído: " + encontrado.getNombre());
```

```
// UPDATE
```

```
em.getTransaction().begin();  
encontrado.setCarrera("Ingeniería de Sistemas");  
em.merge(encontrado);  
em.getTransaction().commit();
```

```
// DELETE
```

```
em.getTransaction().begin();  
em.remove(encontrado);  
em.getTransaction().commit();
```

```
em.close();
```

- Esta aproximación al **procesamiento de bases de datos** a través de un ORM es un pilar de la ingeniería de software moderna (Kroenke, 2003).

4.2. Consultas con HQL (Hibernate Query Language)

HQL es más potente que un simple find. Permite expresar consultas complejas de manera orientada a objetos.

- **HQL vs. SQL:**
 - SQL: `SELECT nombre_completo FROM estudiantes WHERE carrera = 'Ingeniería de Sistemas';`
 - HQL: `SELECT e.nombre FROM Estudiante e WHERE e.carrera = :carrera` La sintaxis es similar, pero HQL usa nombres de clases y propiedades, no de tablas y columnas.
- **Ejemplos de Consultas Avanzadas:**
 - **Consulta con JOIN:** Para obtener todas las reservas de un viaje específico.

Fragmento de código

```
SELECT r FROM Reserva r JOIN r.viaje v WHERE v.id = :viajeld
```

- **Consulta con Agregación:** Para contar cuántos estudiantes hay en cada carrera.

Fragmento de código

```
SELECT e.carrera, COUNT(e) FROM Estudiante e GROUP BY e.carrera
```

- Estas consultas demuestran el poder de los **lenguajes de definición y modificación de datos** adaptados al paradigma de objetos (Camuña Rodríguez, 2025).

4.3. Trabajo con Archivos JSON usando la librería Gson

La librería Gson de Google es una herramienta excelente para la serialización.

- **Serializando una Colección:** Un caso de uso muy común es guardar una lista de objetos.

Java

```
List<Producto> inventario = new ArrayList<>();  
inventario.add(new Producto("CAF-001", "Café Nariño", 35000.0));  
inventario.add(new Producto("PAN-002", "Mogolla Chicharrona", 5000.0));
```

```
Gson gson = new Gson();  
String jsonInventario = gson.toJson(inventario);  
// El resultado es un string que representa un array de objetos JSON  
// [{"id":"CAF-001",...},{id:"PAN-002",...}]
```

- **Deserializando una Colección:** Para convertir el string de vuelta a una lista de objetos, necesitamos usar un TypeToken para que Gson entienda la estructura genérica List<Producto>.

Java

```
Type tipoLista = new TypeToken<ArrayList<Producto>>().getType();  
List<Producto> inventarioRecuperado = gson.fromJson(jsonInventario, tipoLista);
```

Conclusión

Hemos completado un viaje exhaustivo a través del mundo de la persistencia de datos. Comenzamos con un problema fundamental y aparentemente simple: cómo evitar que la información de nuestras aplicaciones se pierda al cerrarlas. Esta pregunta nos llevó a explorar los cimientos teóricos, desde la serialización de objetos hasta la compleja pero poderosa arquitectura de las bases de datos. Vimos cómo el **desajuste de impedancia objeto-relacional** no es un problema menor, sino un desafío central en la ingeniería de software que ha impulsado la creación de herramientas sofisticadas como los ORM.

Luego, pusimos estos conceptos en el contexto del mundo real. Analizamos cómo un simple archivo **JSON** resuelve elegantemente la necesidad de guardar la configuración de una aplicación, cómo la estructura y transaccionalidad de una **base de datos SQL** son indispensables para la fiabilidad de un e-commerce, y cómo la flexibilidad de **NoSQL** se adapta perfectamente a la data heterogénea del sector salud. Profundizamos en un caso de estudio profesional, donde vimos que la persistencia no es solo una capa de código, sino una decisión de **arquitectura** con un impacto directo y medible en el éxito de un negocio. Finalmente, nos equipamos con las herramientas prácticas, aprendiendo a implementar operaciones **CRUD con Hibernate** y a manejar la serialización de datos con **Gson**.

La persistencia de datos es, en esencia, el arte de darle memoria al software. Es una habilidad que trasciende un lenguaje o un framework específico. Comprender los principios de cómo estructurar, guardar y recuperar datos de manera eficiente y segura te permitirá construir aplicaciones más robustas, escalables y valiosas. El camino del desarrollador de software es un aprendizaje continuo. Te invito a que veas los conceptos de este documento no como un punto final, sino como un sólido punto de partida. Experimenta, construye tus propios proyectos, enfrenta los desafíos de la gestión de datos y sigue aprendiendo, pues en el mundo digital, los datos son el activo más importante.

Preguntas para la Reflexión:

1. Imagina que debes diseñar la base de una red social para una comunidad de artistas en Colombia. Los usuarios publican "obras" que pueden ser imágenes, videos cortos o textos. Los usuarios pueden seguirse entre sí y comentar en las obras. ¿Qué estrategia de persistencia elegirías? ¿Una base de datos relacional, una documental (NoSQL) o quizás una base de datos de grafos? Justifica tu decisión pensando en las consultas principales que el sistema deberá realizar (ej. "mostrar las obras de los artistas que sigo").
2. La persistencia de datos implica guardar información sobre los usuarios. En el caso del sistema de salud que discutimos, se guarda información médica sensible. ¿Qué responsabilidades éticas y legales (como la Ley de Habeas Data en

Colombia) crees que un desarrollador de software debe considerar al diseñar la capa de persistencia de una aplicación que maneja datos personales?



Referencias

Camuña Rodríguez, J. F. (2025). *Lenguajes de definición y modificación de datos SQL. IFCT0310* (1.ª ed.). IC Editorial.

Capacho, J. R., & Nieto Bernal, W. (2017). *Diseño de base de datos*. Universidad del Norte.

Hernández González, A. (2009). Un modelo de persistencia para el diseño de la base de datos basado en un modelo de objetos formal. *Revista Investigación Operacional*, 23(3). Editorial Universitaria.

Kim-Park, D. S. (2011). Aplicación de un oráculo de prueba automatizado a la evaluación de salidas de programas basados en XML. *Revista Española de Innovación, Calidad e Ingeniería del Software*, 7(1), 6-21. Asociación de Técnicos de Informática (ATI).

Kroenke, D. M. (2003). *Procesamiento de bases de datos: fundamentos, diseño e implementación*. Pearson Educación.

Martín Escofet, C. (2002). El lenguaje SQL. *UOC Papers*.

Medina Serrano, S. (2015). *SQL Server 2014: soluciones prácticas de administración*. RA-MA Editorial.

Toharia Rabasco, J. P. & Raya González, L. (2015). *Lenguajes de marcas y sistemas de gestión de la información*. RA-MA Editorial.

Wanumen Silva, L. F. (2018). *Bases de datos en SQL server*. Ecoe Ediciones.

Información general

Fecha de creación	6 - 10 - 2025	Dirección de operaciones
-------------------	---------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Conexión y Consumo de APIs

Introducción

Imagina que estás en un restaurante concurrido. Tú (el cliente) tienes una necesidad clara: comer. La cocina tiene la capacidad de satisfacerla. Pero no entras directamente a la cocina a revolver las ollas y tomar los ingredientes. Sería un caos ineficiente y peligroso. En su lugar, interactúas con un intermediario experto: el mesero. Le entregas un pedido claro y estructurado (una **solicitud**) y él se encarga de traducirlo al lenguaje de la cocina, gestionar el proceso y, finalmente, traerte el plato terminado (una **respuesta**).

En el vasto y dinámico universo del software, una **API (Interfaz de Programación de Aplicaciones)** cumple exactamente ese rol de mesero. Es un contrato formal, un conjunto de reglas y herramientas que permite que dos piezas de software completamente distintas se comuniquen y colaboren entre sí. Una aplicación no necesita conocer la complejidad interna de otra —en qué lenguaje fue escrita, cómo está estructurada su base de datos o en qué servidor se ejecuta—; solo necesita saber "hablar" el idioma definido por la API.

Esta capacidad de interconexión estandarizada ha provocado una revolución silenciosa. Hemos pasado de construir aplicaciones como fortalezas aisladas (monolitos) a diseñar ecosistemas vibrantes y modulares (microservicios). Piensa en las aplicaciones que usas a diario en Colombia: cuando pides un servicio en Rappi, la app no ha construido su propio sistema de mapas, ni su propia pasarela de pagos, ni su propio sistema de notificaciones. En su lugar, consume la API de Google Maps para las rutas, la de Wompi o PayU para las transacciones y la de Firebase para los mensajes. Rappi es, en esencia, una brillante orquestación de múltiples APIs. Este modelo, conocido como la **"Economía de APIs"**, es el motor del desarrollo tecnológico actual.

Dominar la conexión y el consumo de APIs se ha convertido, por tanto, en una competencia esencial y diferenciadora para cualquier ingeniero de software. Las empresas ya no solo buscan programadores que puedan construir algoritmos, sino arquitectos de soluciones que puedan integrar servicios existentes para crear valor de forma rápida y escalable (Pressman & Maxim, 2020). Por ello, lo que aprenderás aquí va más allá de la sintaxis y los comandos. Aprender a consumir APIs no es solo una habilidad técnica; **es obtener la llave para conectar tus proyectos con el universo digital.**

Objetivos

Objetivo General

Al finalizar el estudio de este documento, el estudiante de la Corporación Unificada Nacional de Educación Superior estará en capacidad de diseñar, consumir y gestionar conexiones con APIs RESTful de forma segura y eficiente, integrándolas como componentes modulares dentro de aplicaciones orientadas a objetos construidas en Java, para construir soluciones de software robustas y escalables.

Objetivos Específicos

Para alcanzar este objetivo general, el estudiante deberá ser capaz de:

1. **Identificar los Fundamentos:** Describir y diferenciar los conceptos teóricos clave que sustentan la comunicación web, incluyendo los verbos y códigos de estado del protocolo HTTP, los principios de la arquitectura REST y la estructura del formato de datos JSON.
2. **Analizar en Contexto:** Evaluar el rol estratégico de las APIs en aplicaciones de la vida real y casos de estudio profesionales, explicando cómo su integración permite resolver problemas de negocio complejos y generar nuevas funcionalidades.
3. **Implementar la Comunicación:** Construir clientes HTTP en Java para ejecutar las cuatro operaciones básicas (CRUD) contra un endpoint de una API, enviando solicitudes y gestionando las respuestas del servidor de manera programática.
4. **Manejar los Datos:** Aplicar técnicas de serialización y deserialización para mapear eficientemente los datos entre el formato JSON de la API y los objetos del dominio de una aplicación Java, utilizando librerías estándar del mercado.
5. **Garantizar la Seguridad:** Comprender e implementar flujos de autenticación y autorización, como OAuth2, para consumir APIs protegidas y asegurar un manejo responsable de los datos y los permisos.

Fundamentos teóricos

Para construir un edificio sólido, primero debemos entender las propiedades de los materiales y los principios de la arquitectura. De igual manera, antes de consumir nuestra primera API, es crucial dominar los conceptos teóricos que gobiernan la comunicación entre aplicaciones en la web. Este capítulo establece esas bases fundamentales.

1.1 ¿Qué es una API? Conceptos Clave

Como vimos, una API es un intermediario, pero esa comunicación se rige por un vocabulario técnico preciso. Los componentes esenciales que debes dominar son:

- **Endpoint:** Es la URL específica donde la API expone un recurso. Piensa en ello como la "dirección" exacta a la que debes enviar tu solicitud. Por ejemplo, `https://api.banco.com.co/v1/clientes/12345` es un endpoint que apunta al cliente con el ID 12345.
- **Recurso (Resource):** Es cualquier objeto o pieza de información que la API puede proporcionar. En el ejemplo anterior, el "cliente" es el recurso. Otros ejemplos podrían ser un producto, una publicación o un video.
- **Verbos HTTP (Métodos):** Son las acciones que deseamos realizar sobre un recurso. Los más comunes, que forman el acrónimo CRUD (Create, Read, Update, Delete), son:
 - **GET:** Para leer o recuperar un recurso. Es una operación segura, no modifica datos.
 - **POST:** Para crear un nuevo recurso. Por ejemplo, registrar un nuevo cliente.
 - **PUT:** Para actualizar completamente un recurso existente.
 - **DELETE:** Para eliminar un recurso.
- **Códigos de Estado HTTP:** Son números de tres dígitos que la API devuelve en su respuesta para indicar el resultado de la solicitud. Permiten una gestión de errores robusta. Se agrupan en familias (Comer, 2015):
 - **2xx (Éxito):** 200 OK, 201 Created.
 - **4xx (Error del Cliente):** 400 Bad Request, 401 Unauthorized, 404 Not Found.
 - **5xx (Error del Servidor):** 500 Internal Server Error.

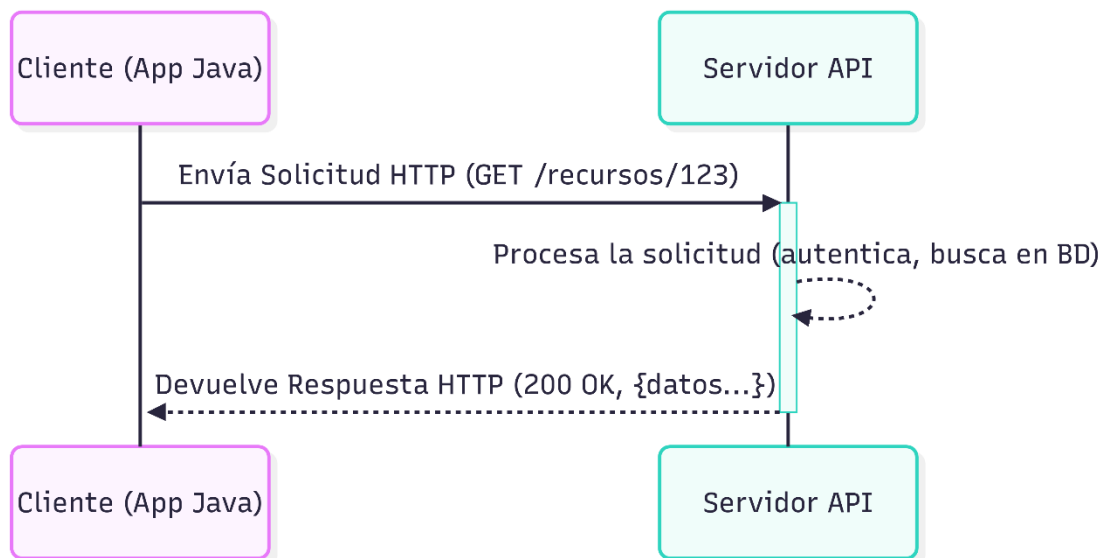


Figura 1: Diagrama del Ciclo de Vida de una Solicitud HTTP

Diagrama de secuencia que muestra a un Cliente enviando una solicitud HTTP a un Servidor API, el cual procesa la solicitud y devuelve una respuesta HTTP con un código de estado y un cuerpo de datos.

1.2 Arquitectura REST (Representational State Transfer)

No todas las APIs se diseñan de la misma manera. REST no es un protocolo ni un estándar, sino un conjunto de principios o restricciones arquitectónicas para diseñar aplicaciones en red. Las APIs que siguen estos principios se denominan **APIs RESTful** y son el estándar de facto en la web por su simplicidad y rendimiento (Pressman & Maxim, 2020).

Los principios clave son:

- **Arquitectura Cliente-Servidor:** Clara separación entre el cliente (quien consume los datos) y el servidor (quien los almacena y provee).
- **Sin Estado (Stateless):** Cada solicitud del cliente al servidor debe contener toda la información necesaria para ser entendida y procesada. El servidor no guarda ningún estado del cliente entre solicitudes.
- **Cacheable:** Las respuestas del servidor deben poder ser almacenadas en caché por el cliente para mejorar el rendimiento.
- **Interfaz Uniforme:** Es el principio fundamental de REST. Simplifica la arquitectura al requerir una forma estandarizada de comunicación, que

incluye la identificación de recursos vía URI (el endpoint), la manipulación de recursos a través de representaciones (como JSON) y el uso de mensajes autodescriptivos.

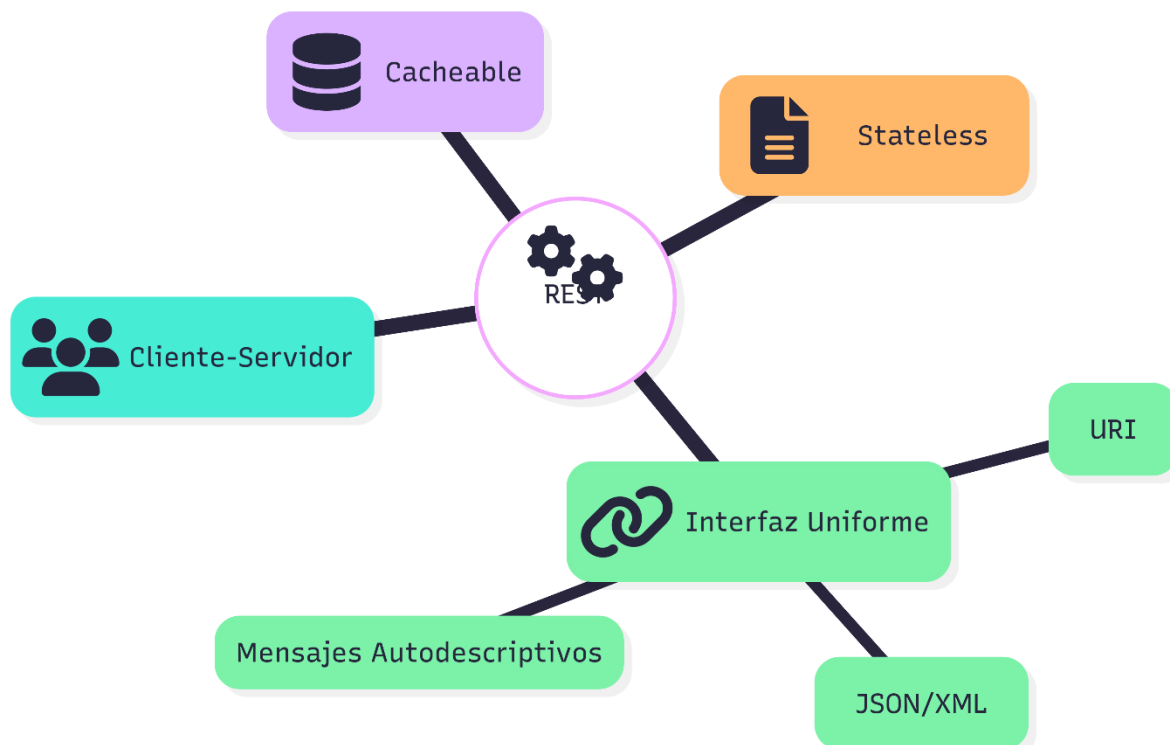


Figura 2: Mapa Conceptual de los Principios REST

Mapa conceptual que muestra el concepto central "REST" conectado a sus cuatro principios principales: Cliente-Servidor, Sin Estado, Cacheable e Interfaz Uniforme.

1.3 El Formato de Datos JSON

Para que dos sistemas intercambien datos, deben "hablar" un idioma común. **JSON (JavaScript Object Notation)** se ha convertido en el lenguaje preferido para las APIs RESTful debido a que es ligero, legible para los humanos y fácil de procesar para las máquinas. Su estructura se basa en pares de clave: valor, similar a los atributos de un objeto en programación, lo que facilita enormemente su mapeo (Yanev, 2025).

Un ejemplo de un objeto Estudiante en JSON:

```
JSON
{
  "id": "2025102001",
  "nombre": "Ana",
```

```
"apellido": "Rojas",  
"activo": true,  
"cursos": [ "POO2", "Bases de Datos" ]  
}
```

1.4 Solicitudes y Respuestas HTTP con Objetos

El verdadero poder de usar APIs en Programación Orientada a Objetos radica en la capacidad de convertir datos JSON en objetos de nuestro lenguaje (en este caso, Java) y viceversa.

- **Deserialización:** Es el proceso de tomar una cadena de texto en formato JSON recibida del servidor y convertirla en un objeto Java. Por ejemplo, el JSON anterior se convertiría en una instancia de una clase Estudiante en nuestro código.
- **Serialización:** Es el proceso inverso. Tomar un objeto Java (por ejemplo, uno que acabamos de crear) y convertirlo a una cadena de texto JSON para enviarlo al servidor en el cuerpo de una solicitud POST o PUT.

Este mapeo directo entre el formato de datos de la API y los objetos de nuestro programa es lo que permite una integración limpia y coherente, siguiendo los principios del encapsulamiento y la abstracción de la POO (Deitel & Deitel, 2018; Erl, 2008).

Aplicaciones en la vida cotidiana

La teoría del Capítulo 1 cobra vida cuando observamos el ecosistema digital que nos rodea. Las APIs no son un concepto abstracto confinado a los libros de texto; son la infraestructura que permite que las aplicaciones de nuestro celular, las plataformas de comercio electrónico y los servicios gubernamentales funcionen de manera integrada y eficiente. Este capítulo aterriza los fundamentos en casos de uso cotidianos, demostrando el valor práctico y transformador de saber consumir APIs.

2.1 APIs en las Redes Sociales: El Mundo Conectado

Cada vez que ves un "Iniciar sesión con Google" o "Continuar con Facebook" en una aplicación, estás presenciando una API de autenticación en acción. En lugar de crear un nuevo usuario y contraseña, la aplicación delega la tarea de verificar tu identidad a la API de un proveedor de confianza. De manera similar, cuando una aplicación de noticias muestra los últimos trinos sobre un tema o una tienda en línea te permite compartir un producto en tu muro de Facebook, no están "hackeando" la plataforma; están utilizando sus APIs públicas.

- **Caso Práctico:** Un portal de marketing digital en Colombia, como "Marketing 4 Ecommerce", podría desarrollar un *dashboard* para sus clientes. Este tablero, en lugar de obligar al usuario a entrar a cada red social por separado, consume la API de Twitter para mostrar métricas de trinos, la API de Instagram para analizar el rendimiento de las historias y la API de Facebook para rastrear el alcance de las publicaciones. El valor para el cliente es inmenso: una vista unificada y en tiempo real de toda su estrategia digital, posible gracias a la orquestación de múltiples APIs.

2.2 APIs en el E-commerce Colombiano: Facilitando las Transacciones

El comercio electrónico en Colombia ha crecido exponencialmente, y las APIs son su columna vertebral. Desde la validación de la dirección de envío hasta el procesamiento del pago, cada paso es gestionado por servicios especializados que se comunican vía API.

- **Caso Práctico:** Imagina una pequeña empresa de Medellín que vende café de especialidad a través de su sitio web.
 1. **Cálculo de Envío:** Cuando un cliente en Cali ingresa su dirección, el sitio web no tiene una base de datos con todos los costos de envío del país. En su lugar, envía los datos del paquete (peso, dimensiones) y la dirección de destino a la **API de un operador logístico** como Servientrega o Interrapidísimo. La API calcula el costo en tiempo real y lo devuelve para mostrarlo en el carrito de compras.

2. **Procesamiento del Pago:** Al momento de pagar, el cliente elige su método preferido (tarjeta de crédito, PSE, etc.). El sitio web no procesa directamente esta información sensible. Envía los detalles de la transacción a la **API de una pasarela de pagos** como Wompi (de Bancolombia) o PayU. Esta API se encarga de la comunicación segura con el banco, autoriza la transacción y devuelve una confirmación (o un rechazo) a la tienda.

Este desacoplamiento es crucial: la tienda de café se enfoca en lo que mejor sabe hacer (vender café), mientras que la logística y los pagos son manejados por expertos a través de APIs robustas y seguras. La integración de APIs de inteligencia artificial, por ejemplo, puede incluso personalizar la experiencia de compra, ofreciendo productos basados en el historial del cliente (Yanev, 2025).

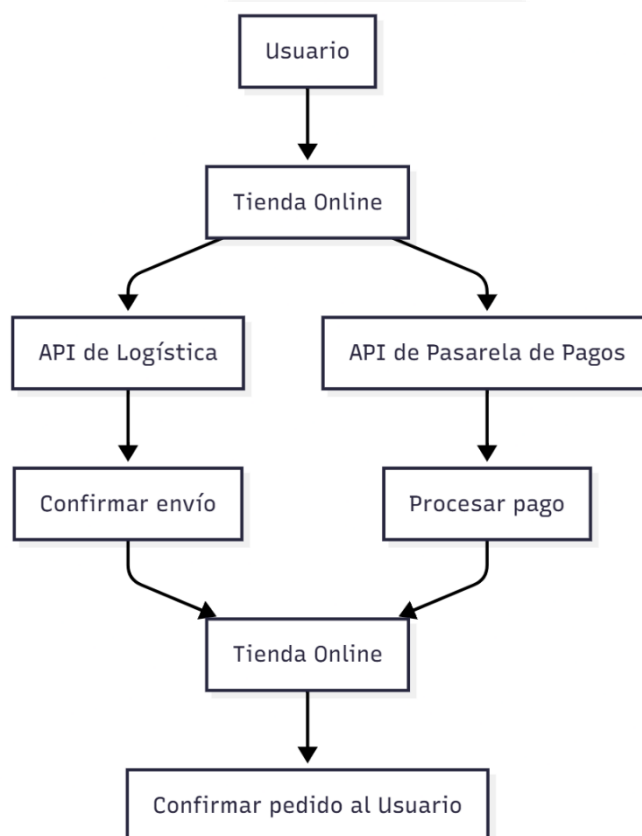


Figura 3: Diagrama de Flujo de una Compra Online con APIs

Diagrama de flujo que ilustra los pasos de una compra en línea. El flujo comienza con el Usuario, interactúa con la Tienda Online, que a su vez se comunica con una API de Logística para el envío y una API de Pasarela de Pagos para la transacción, antes de confirmar el pedido al Usuario.

2.3 APIs de Servicios Públicos y Datos Abiertos: Innovación Cívica

Los gobiernos y las entidades públicas también han adoptado las APIs como un mecanismo para fomentar la transparencia y la innovación. A través de portales de datos abiertos, exponen información de interés público para que desarrolladores, periodistas y ciudadanos puedan crear nuevas aplicaciones y servicios de valor.

- **Ejemplo Cercano: El portal de Datos Abiertos de Colombia (Datos.gov.co)** ofrece cientos de conjuntos de datos a través de APIs. Un estudiante de la CUN podría desarrollar una aplicación que:
 - Consuma la API de **Calidad del Aire** del Ministerio de Ambiente para mostrar un mapa en tiempo real de los niveles de contaminación en las principales ciudades.
 - Utilice la API del **SECOP (Sistema Electrónico de Contratación Pública)** para crear un sistema de alertas que notifique a las pequeñas empresas sobre nuevas licitaciones en su sector.
 - Integre la API de **Indicadores del Banco de la República** para crear una herramienta de finanzas personales que convierta precios a la TRM del día.

Estas aplicaciones no solo representan excelentes proyectos académicos, sino que también son ejemplos de cómo la tecnología, habilitada por las APIs, puede generar un impacto social positivo, mejorar la toma de decisiones y fortalecer la participación ciudadana. La correcta evaluación y testeo de estas APIs es fundamental para asegurar su fiabilidad y seguridad (Qin et al., 2025).

Casos de estudio

Mientras que el capítulo anterior mostró el "qué" y el "dónde" de las APIs, este se enfoca en el "cómo" y el "porqué" dentro de un contexto empresarial. Analizaremos un desafío de negocio específico, desglosaremos la solución técnica implementada y evaluaremos los resultados cuantitativos y cualitativos. Este enfoque integral es el que se utiliza en la industria para justificar y desarrollar proyectos de software.

3.1 Caso de Estudio: "CunExpress - Logística Inteligente"

Contexto: CunExpress es una empresa de mensajería y paquetería pyme con sede en Bogotá. Su operación se ha basado tradicionalmente en la asignación manual de rutas, donde un despachador, basándose en su experiencia, distribuye los paquetes del día entre los mensajeros.

- **Desafío:** La empresa enfrenta crecientes problemas de competitividad. Sus costos de combustible son altos, las entregas a menudo se retrasan debido al tráfico impredecible de Bogotá y a eventos inesperados (como manifestaciones o fuertes lluvias), y la satisfacción del cliente ha comenzado a disminuir. La gerencia necesita una solución tecnológica para optimizar la operación logística sin incurrir en el costo prohibitivo de desarrollar un sistema de enrutamiento desde cero.
- **Solución Aplicada:** Un equipo de tecnólogos de la CUN es contratado para desarrollar un módulo de software en Java que se integre al sistema de gestión de pedidos existente. La solución no busca reinventar la rueda, sino conectar inteligentemente servicios externos a través de sus APIs. La arquitectura propuesta se basa en el consumo de dos APIs clave:
 1. **API de Google Maps (Directions API):** Para el cálculo de rutas. Al inicio del día, el módulo de Java toma la lista de direcciones de entrega, la envía a la API de Google Maps y esta devuelve la ruta más eficiente que conecta todos los puntos, considerando el tráfico en tiempo real.
 2. **API de OpenWeatherMap (One Call API):** Para la predicción climática. Antes de despachar a un mensajero, el sistema consulta las coordenadas de la ruta en la API de OpenWeatherMap. Si se pronostica una lluvia intensa en una zona específica, el sistema puede recalcular la ruta para evitar áreas propensas a inundaciones o simplemente notificar al cliente de un posible retraso.

El núcleo de la solución es un conjunto de clases Java (GestorRutas, ClienteGoogleMaps, ClienteClima) que encapsulan la lógica de las llamadas a las APIs. Por ejemplo, el método obtenerRutaOptima() de GestorRutas se encarga de recolectar las direcciones, construir la solicitud para la API de Google Maps,

enviarla, y deserializar la respuesta JSON para convertirla en un objeto Ruta que el sistema puede entender y mostrar.

- Impacto y Datos:** Tras un piloto de tres meses, CunExpress midió los siguientes resultados:
 - Reducción del 15% en el tiempo promedio de entrega:** Gracias a la optimización de rutas y la evasión de trancones.
 - Disminución del 10% en el consumo de combustible:** Rutas más cortas y eficientes se traducen directamente en ahorro de costos.
 - Aumento del 20% en la satisfacción del cliente:** Medido a través de encuestas, gracias a entregas más puntuales y notificaciones proactivas sobre retrasos por clima.

Este caso demuestra cómo la integración de APIs es una estrategia de **apalancamiento tecnológico**. CunExpress, una pyme, logró acceder a la misma tecnología de optimización de rutas que usan gigantes como Amazon o Mercado Libre, pagando solo por el consumo de las APIs, lo cual es significativamente más económico y rápido que el desarrollo propio (Pressman & Maxim, 2020).

Característica	Antes de la Integración de APIs	Después de la Integración de APIs
Asignación de Ruta	Manual, basada en la experiencia del despachador.	Automática, optimizada con datos de tráfico en tiempo real.
Adaptabilidad	Nula. Las rutas no cambiaban ante imprevistos.	Alta. Capacidad de recalcular rutas por clima o tráfico.
Toma de Decisiones	Subjetiva y propensa a errores humanos.	Basada en datos, objetiva y consistente.
Comunicación Cliente	Reactiva. El cliente llama para saber de su paquete.	Proactiva. Notificaciones automáticas de posibles retrasos.
Costo Operativo	Alto (más combustible, más horas-hombre).	Reducido (ahorro en combustible y tiempo).

Tabla 1: Comparativa de Funcionalidades (Antes vs. Después de la API)

Tabla comparativa que muestra las mejoras en el proceso de asignación de rutas de una empresa de logística antes y después de integrar APIs, destacando cambios en eficiencia, adaptabilidad y comunicación.

Este enfoque, que consiste en componer sistemas complejos a partir de servicios más pequeños y especializados, es la esencia de las arquitecturas orientadas a servicios (SOA) y microservicios, un paradigma fundamental en el desarrollo de

software moderno (Erl, 2008). La habilidad para identificar, evaluar e integrar APIs de terceros es, por lo tanto, una competencia estratégica para cualquier profesional del software.



Herramientas y técnicas

La transición de la teoría a la práctica requiere el dominio de herramientas específicas. En el ecosistema de Java, existen librerías robustas y estandarizadas para manejar la comunicación por red y el procesamiento de datos. Este capítulo se enfoca en el "cómo": cómo escribir el código para realizar solicitudes HTTP, cómo manipular datos en formato JSON y cómo implementar flujos de autenticación para construir las soluciones que describimos en capítulos anteriores.

4.1 Cliente HTTP Nativo en Java

Desde la versión 11, Java incluye un cliente HTTP moderno y muy capaz en su biblioteca estándar (`java.net.http`), eliminando la necesidad de dependencias externas para tareas básicas. Este cliente es asíncrono por defecto y tiene una API fluida y fácil de usar.

Los tres componentes principales son:

- `HttpClient`: El objeto que envía las solicitudes y recibe las respuestas. Se crea una sola vez y se reutiliza.
- `HttpRequest`: Representa la solicitud que enviaremos. Se construye especificando la URI (el endpoint), el método (GET, POST, etc.) y las cabeceras.
- `HttpResponse`: Contiene la respuesta del servidor, incluyendo el código de estado, las cabeceras y el cuerpo (los datos).
-

Ejercicio Práctico: Realizar una solicitud GET

El siguiente código consume la API pública "JSONPlaceholder" para obtener los datos de un usuario de prueba.

Java

```
// Es necesario importar las clases del paquete java.net.http
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ConsumoAPI {

    public static void main(String[] args) throws Exception {
        // 1. Crear una instancia de HttpClient
        HttpClient client = HttpClient.newHttpClient();

        // 2. Crear una instancia de HttpRequest con la URI del endpoint
        HttpRequest request = HttpRequest.newBuilder()
```

```
.uri(URL.create("https://jsonplaceholder.typicode.com/users/1"))  
.GET() // Especificar el verbo HTTP  
.build();
```

```
// 3. Enviar la solicitud y recibir la respuesta  
// El segundo argumento, BodyHandlers.ofString(), indica que queremos leer el  
cuerpo como un String  
HttpResponse<String> response = client.send(request,  
HttpResponse.BodyHandlers.ofString());  
  
// 4. Imprimir los resultados  
System.out.println("Código de Estado: " + response.statusCode());  
System.out.println("Cuerpo de la Respuesta (JSON): " + response.body());  
}  
}
```

4.2 Manejo de JSON en Java con la Librería Gson

Como vimos, la respuesta de la API es una cadena de texto en formato JSON. Trabajar directamente con este texto es engorroso y propenso a errores. Aquí es donde entran en juego las librerías de mapeo objeto-JSON, como **Gson** de Google o **Jackson**. Estas herramientas automatizan la **serialización** (de objeto Java a JSON) y la **deserialización** (de JSON a objeto Java).

Para usar Gson, primero debes agregar la librería a tu proyecto en NetBeans. Luego, creas una clase en Java (un "POJO" - Plain Old Java Object) que tenga los mismos nombres de atributos que las claves del JSON.

Ejercicio Práctico: Deserializar la respuesta JSON a un Objeto

1. Crear la clase Usuario.java:

Java

```
public class Usuario {  
    private int id;  
    private String name;  
    private String username;  
    private String email;
```

// Es importante generar los getters y un método toString() para ver el resultado

```
public String getName() {  
    return name;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
@Override
public String toString() {
    return "Usuario{id=" + id + ", name=" + name + ", email=" + email + "}";
}
}
```

2. Modificar el main para usar Gson:

Java

```
import com.google.gson.Gson; // Importar Gson
// ... (otras importaciones)

public class ConsumoAPI {
    public static void main(String[] args) throws Exception {
        // ... (código para la solicitud HTTP)
        HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

        // 5. Usar Gson para deserializar el JSON
        if (response.statusCode() == 200) {
            Gson gson = new Gson();
            // response.body() contiene el String JSON
            Usuario usuario = gson.fromJson(response.body(), Usuario.class);

            System.out.println("Objeto Usuario deserializado: " + usuario);
            System.out.println("Nombre del usuario: " + usuario.getName());
        } else {
            System.out.println("Error al consultar la API.");
        }
    }
}
```

Este enfoque es la base de una programación limpia y orientada a objetos, pues nos permite trabajar con Usuario como un objeto de nuestro dominio, en lugar de manipular cadenas de texto (Deitel & Deitel, 2018).

4.3 Autenticación y Autorización con OAuth2

Muchas APIs no son públicas y requieren que la aplicación se identifique y tenga permisos para acceder a los datos. **OAuth2** es un estándar de la industria para la autorización delegada. En lugar de que el usuario le dé su contraseña a nuestra aplicación, le da un permiso (consentimiento) a través de un intermediario de confianza (como Google o Facebook).

El flujo más común ("Authorization Code") funciona así, conceptualmente:

1. Nuestra aplicación redirige al usuario a la página de inicio de sesión del proveedor (ej. Google).

2. El usuario inicia sesión y autoriza a nuestra aplicación a acceder a ciertos datos (ej. "ver tu correo electrónico").
3. El proveedor redirige de vuelta al usuario a nuestra aplicación, entregándole un "código de autorización" de un solo uso.
4. Nuestra aplicación, desde el servidor (backend), intercambia este código (junto con sus propias credenciales de cliente) por un **Token de Acceso**.
5. Este Token de Acceso es el que se incluye en las cabeceras de las futuras solicitudes a la API para autenticar la comunicación.

La implementación completa de este flujo es avanzada, pero como tecnólogo, es crucial entender el proceso para poder utilizar librerías que lo simplifiquen y para configurar correctamente las credenciales de cliente en las consolas de desarrollador de las APIs (Pressman & Maxim, 2020).

4.4 Taller de Integración: NetBeans y Base de Datos SQL

Unamos todo en un taller práctico. El objetivo es crear una aplicación simple en NetBeans que consuma una API pública, procese los datos y los guarde en una tabla de una base de datos local (ej. MySQL o PostgreSQL).

Pasos del Taller:

1. **Configurar el Proyecto:** Crea un nuevo proyecto Java en NetBeans. Agrega la librería de Gson y el conector JDBC para tu base de datos al *classpath* del proyecto.
2. **Diseñar la Base de Datos:** Crea una tabla simple en tu gestor de base de datos. Por ejemplo: CREATE TABLE posts (id INT PRIMARY KEY, userId INT, title VARCHAR(255), body TEXT);.
3. **Consumir la API:** Usando HttpClient, haz una solicitud GET a <https://jsonplaceholder.typicode.com/posts>. Este endpoint devuelve un array de 100 publicaciones.
4. **Deserializar el Array:** La respuesta será un JSON que empieza con [. Debes usar Gson para deserializar esta cadena en un array o una lista de objetos Post (Post[] o List<Post>). Para ello, primero debes crear la clase Post.java.
5. **Conectar y Guardar:** Utiliza JDBC para establecer una conexión con tu base de datos. Itera sobre la lista de objetos Post y, para cada uno, ejecuta una sentencia INSERT en la tabla posts. Es fundamental usar PreparedStatement para evitar inyección SQL (Yanev, 2025).
6. **Verificar:** Ejecuta una consulta SELECT * FROM posts en tu base de datos para confirmar que los 100 registros se guardaron correctamente.

Este ejercicio integra redes, procesamiento de datos y persistencia, simulando un ciclo de trabajo muy común en el desarrollo de software profesional.

Conclusión

Hemos viajado desde los fundamentos teóricos que definen a una API como el "mesero" del mundo digital, hasta su aplicación práctica en escenarios cotidianos y profesionales que impactan la economía y la sociedad en Colombia. Desglosamos la arquitectura REST, manipulamos el formato JSON y analizamos cómo gigantes del e-commerce y servicios públicos dependen de estas interfaces para funcionar. Finalmente, nos sumergimos en el código, utilizando Java y herramientas profesionales para construir un puente tangible entre nuestra aplicación y el vasto universo de servicios disponibles en la web.

La lección más importante de este recorrido es que el desarrollo de software moderno es un acto de **integración y colaboración**. La habilidad para consumir APIs ya no es una especialización, sino una competencia fundamental. Te permite construir aplicaciones más potentes, en menos tiempo y con menos recursos, al apalancarte en el trabajo de miles de desarrolladores expertos en sus respectivos dominios, ya sea en mapas, pagos, inteligencia artificial o análisis de datos. Has aprendido a no reinventar la rueda, sino a construir vehículos más rápidos y eficientes conectando las mejores ruedas disponibles.

El conocimiento adquirido aquí es una base sólida, pero el ecosistema de APIs evoluciona a una velocidad vertiginosa. Te invito a ser un explorador constante: busca nuevas APIs, lee su documentación, experimenta con ellas en pequeños proyectos personales y mantén viva tu curiosidad. La verdadera maestría no reside en memorizar una librería o un framework, sino en desarrollar la capacidad de entender, adaptar e integrar cualquier servicio que el futuro ponga a tu disposición.

Para finalizar, te dejo dos preguntas abiertas que te invitan a llevar esta reflexión más allá de lo técnico:

1. Más allá de la eficiencia técnica, ¿cuál crees que es la **responsabilidad ética** de un desarrollador al consumir APIs que manejan datos personales de los usuarios? ¿Dónde trazarías la línea entre la personalización de un servicio y la invasión de la privacidad?
2. Observando tu entorno local (tu barrio, tu ciudad, tu universidad), ¿qué **proceso manual o ineficiente** podrías mejorar o automatizar drásticamente mediante la integración de una o varias APIs que ya existen?

Referencias.

Comer, D. E. (2015). *Redes de computadoras e Internet*. Pearson Educación.
<https://www-ebooks7-24-com.zproxy.cun.edu.co/?il=3262>

Deitel, P. J., & Deitel, H. M. (2018). *Cómo programar en Java* (11ª ed.). Pearson Educación. <https://ebooks7-24-com.zproxy.cun.edu.co/?il=7333>

Erl, T. (2008). *SOA: Principios de los servicios de diseño*. Pearson Educación.
<https://www-ebooks7-24-com.zproxy.cun.edu.co/?il=3351>

Gómez, M. (2007). *Calidad en el desarrollo de software*. RA-MA. <https://www-digitaliapublishing-com.zproxy.cun.edu.co/a/17094>

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería de software: Un enfoque práctico* (9ª ed.). McGraw Hill Interamericana. <https://ebooks7-24-com.zproxy.cun.edu.co/?il=24119>

VV. AA. (2025). *Inteligencia Artificial. El futuro de la humanidad*. Aranzadi.
<https://www-digitaliapublishing-com.zproxy.cun.edu.co/a/188182>

Información general

Fecha de creación	08 - 09 - 2025	Dirección de operaciones
-------------------	----------------	--------------------------

Escuela	Ingenierías
Programa	Ingeniería de sistemas.

Identificadores del curso

Código de asignatura	INGS0806
Asignatura	Programación Orientada a Objetos 2
Temática	Pruebas y Depuración de Aplicaciones Orientadas a Objetos

Introducción

En el vertiginoso mundo del desarrollo de software, donde nuevas aplicaciones y servicios digitales surgen a diario, la velocidad de entrega a menudo parece ser el único factor de éxito. Sin embargo, la historia nos ha enseñado una lección crucial: el software que perdura, que gana la confianza de los usuarios y que aporta un valor real al negocio, no es necesariamente el que se construye más rápido, sino el que se construye **mejor**. Aquí es donde entra en juego el **aseguramiento de la calidad del software (SQA)**, un conjunto de disciplinas cuyo propósito es garantizar que el software no solo cumpla con sus requisitos funcionales, sino que también sea robusto, seguro y fácil de mantener.

Este documento se centra en las **Pruebas y Depuración de Aplicaciones Orientadas a Objetos**, dos de los pilares más importantes del SQA. ¿Por qué es esto tan relevante para ti como futuro profesional de la ingeniería en Colombia y Latinoamérica? Porque nuestra región está experimentando una transformación digital acelerada. Desde aplicaciones de *fintech* que manejan el dinero de miles de personas, hasta plataformas de comercio electrónico que gestionan inventarios complejos y sistemas de salud que administran datos críticos de pacientes, la demanda de software fiable nunca ha sido tan alta. Un solo *bug* o defecto en una de estas aplicaciones puede tener consecuencias económicas, de reputación e incluso de seguridad muy graves.

Aprender a probar y depurar de manera sistemática no es una habilidad opcional; es una responsabilidad ética y profesional. Es la diferencia entre entregar un producto frágil que genera problemas constantes y construir una solución de ingeniería sólida que inspira confianza. A lo largo de estas páginas, descubrirás las técnicas y herramientas que te permitirán validar tu propio código, encontrar y corregir errores de manera eficiente y, en última instancia, elevar la calidad de tu trabajo a un estándar profesional.

Lo que aprenderás aquí no es solo a encontrar errores, sino a construir software de tal manera que los errores tengan cada vez menos lugares donde esconderse.

Objetivos

Objetivo General

Al finalizar el estudio de este documento, el estudiante estará en la capacidad de formular y aplicar estrategias de pruebas y depuración siguiendo los principios de la Programación Orientada a Objetos para garantizar la fiabilidad, robustez y mantenibilidad de aplicaciones desarrolladas en Java, utilizando el entorno de desarrollo NetBeans y el framework JUnit.

Objetivos Específicos

Para alcanzar el objetivo general, el estudiante logrará:

1. Distinguir los conceptos teóricos fundamentales del aseguramiento de la calidad y su aplicación específica en el paradigma orientado a objetos, diferenciando entre verificación, validación, niveles de prueba y tipos de defectos.
2. Implementar pruebas unitarias para clases individuales en Java utilizando las anotaciones y aserciones del framework JUnit, con el fin de verificar el correcto funcionamiento del comportamiento encapsulado de los objetos.
3. Utilizar de manera sistemática las herramientas del depurador (*debugger*) del IDE NetBeans para analizar la ejecución de un programa, inspeccionar el estado de los objetos en memoria y diagnosticar la causa raíz de las fallas en el software.

Fundamentos teóricos

En el desarrollo de software moderno, la creación de código funcional es solo el punto de partida. La verdadera medida de la excelencia profesional radica en la capacidad de construir sistemas que no solo funcionen hoy, sino que sean fiables, mantenibles y robustos a lo largo del tiempo. Este compromiso con la calidad es lo que diferencia a un programador de un ingeniero de software. El aseguramiento de la calidad del software (SQA, por sus siglas en inglés) no es una fase final ni una tarea secundaria; es una mentalidad y un conjunto de prácticas integradas en todo el ciclo de vida del desarrollo. En la Programación Orientada a Objetos (POO), donde los sistemas se modelan como colecciones de objetos que colaboran entre sí, las pruebas adquieren una dimensión particular, centrándose en la correcta implementación y la interacción de estas unidades fundamentales (Pressman & Maxim, 2021).

1.1. Conceptos Clave del Aseguramiento de Calidad

Para construir software de alta calidad, primero debemos hablar un lenguaje común. Ciertos términos son cruciales para entender y aplicar correctamente las técnicas de prueba.

- **Verificación vs. Validación:** A menudo usados indistintamente, estos dos conceptos responden a preguntas fundamentalmente diferentes.
 - La **Verificación** se pregunta: "**¿Estamos construyendo el producto correctamente?**". Se enfoca en si el software cumple con sus especificaciones técnicas y de diseño. Es un proceso interno que revisa la arquitectura, el código y la lógica. Las actividades de verificación incluyen revisiones de código, inspecciones de diseño y pruebas unitarias. Por ejemplo, en una aplicación bancaria, la verificación se aseguraría de que el método `calcularInteres()` implementa la fórmula matemática especificada por el departamento financiero sin errores de cálculo (Sommerville, 2011).
 - La **Validación** se pregunta: "**¿Estamos construyendo el producto correcto?**". Se enfoca en si el software satisface las necesidades reales del cliente y los usuarios. Es un proceso externo que evalúa el comportamiento del producto final, a menudo con la participación de los stakeholders. Siguiendo el ejemplo bancario, la validación consistiría en que un usuario final pruebe la funcionalidad de "transferencia" y confirme que es intuitiva, fácil de usar y resuelve su necesidad de enviar dinero.
- **Defecto (Bug), Error y Falla:** Estos términos describen la cadena causal de un problema. Comprender esta secuencia es vital para la depuración.

1. **Error:** Es la acción humana. Un programador, por cansancio o falta de atención, escribe un operador incorrecto en una condición: `if (edad > 18)` en lugar de `if (edad >= 18)`.
 2. **Defecto (o Bug):** Es el resultado del error, latente en el código. La línea `if (edad > 18)` es el defecto. El sistema puede funcionar correctamente en miles de casos, pero el defecto está ahí, esperando ser activado.
 3. **Falla:** Es la manifestación visible del defecto. Cuando un usuario que cumple exactamente 18 años intenta registrarse, la condición `(18 > 18)` devuelve `false`, y el sistema le niega el acceso incorrectamente. Esa negación es la falla. El objetivo de las pruebas es encontrar defectos antes de que causen fallas en producción.
- **Calidad del Software:** La calidad no es un atributo único. Estándares formales como la norma **ISO/IEC 25010** la definen como un conjunto de características. Para un desarrollador orientado a objetos, las más relevantes son:
 - **Funcionalidad:** El software hace lo que se supone que debe hacer.
 - **Fiabilidad:** Opera sin fallas durante un tiempo específico.
 - **Usabilidad:** Es fácil de aprender y utilizar por los usuarios.
 - **Eficiencia:** Tiene un buen rendimiento en relación con los recursos que consume (CPU, memoria, red).
 - **Mantenibilidad:** Es la facilidad con la que el software puede ser modificado. Principios de POO como el **encapsulamiento** mejoran directamente la mantenibilidad, ya que los cambios internos en una clase no deberían afectar a las otras que la utilizan.
 - **Portabilidad:** Puede ser transferido de un entorno a otro (p. ej., de un sistema operativo Windows a Linux) con mínimo esfuerzo.
 - **El Ciclo de Vida de las Pruebas:** Las pruebas no son un evento único, sino un proceso disciplinado que, según modelos estándar, incluye fases como: **1) Planificación y control, 2) Análisis y diseño de pruebas, 3) Implementación y ejecución, 4) Evaluación de criterios de salida y reporte, y 5) Actividades de cierre** (Pressman & Maxim, 2021).

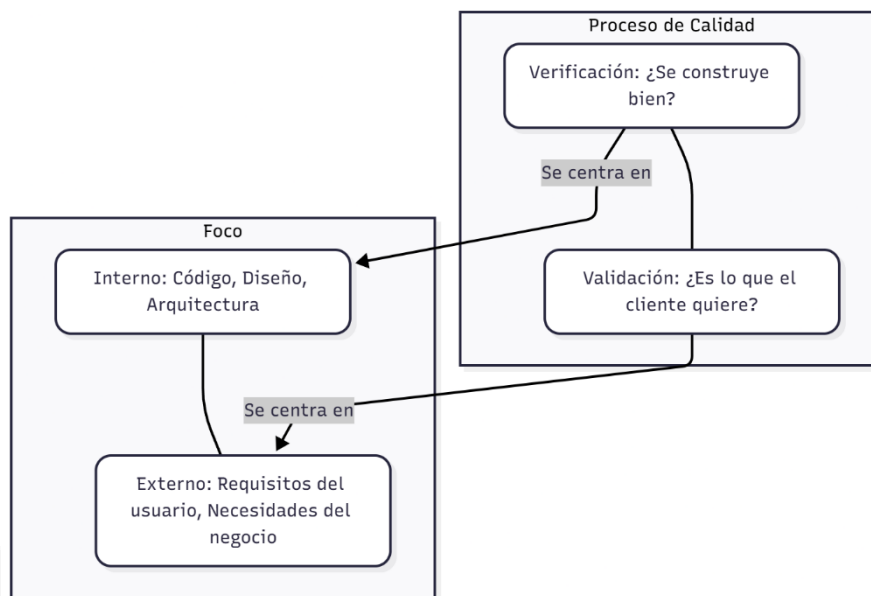


Figura 1: Diagrama de Venn comparando Verificación y Validación

Diagrama que muestra la Verificación como un proceso interno enfocado en el diseño y la Validación como un proceso externo enfocado en los requisitos del cliente.

1.2. El Paradigma de Pruebas Orientadas a Objetos

El paradigma orientado a objetos, con sus principios de encapsulamiento, herencia y polimorfismo, influye directamente en cómo se deben probar las aplicaciones.

- **Pruebas de Unidad (Unit Testing):** En POO, la unidad fundamental de trabajo y prueba es la **clase**. Las pruebas de unidad se centran en verificar que cada clase y sus métodos funcionan como se espera, de forma aislada. El **encapsulamiento** es un gran aliado aquí, pues permite tratar a la clase como una "caja negra", probando su comportamiento a través de su interfaz pública (los métodos públicos) sin preocuparse por los detalles de su implementación interna.
- **Pruebas de Integración (Integration Testing):** Los sistemas OO rara vez consisten en una sola clase; su poder reside en la **colaboración entre objetos**. Las pruebas de integración verifican que los objetos de diferentes clases trabajan juntos correctamente. Por ejemplo, ¿un objeto Factura interactúa correctamente con un objeto Cliente para obtener los datos de facturación? El **polimorfismo** puede introducir complejidad aquí, ya que un objeto puede interactuar con diferentes tipos de objetos a través de una misma interfaz, y todas las variantes deben ser probadas.
- **Pruebas del Sistema (System Testing):** Estas pruebas evalúan la aplicación en su conjunto. La **herencia** presenta un desafío particular: si se modifica una superclase, ¿cómo afecta esto a todas sus subclases? Es

necesario realizar pruebas de regresión para asegurar que los cambios en una parte de la jerarquía de clases no introduzcan fallas en otras.

1.3. Tipos de Pruebas y su Aplicación

Las estrategias de prueba se pueden clasificar según el nivel de conocimiento que se tenga sobre el funcionamiento interno del sistema.

- **Pruebas de Caja Negra (Black-Box Testing):** El probador no conoce la estructura interna del código. Las pruebas se diseñan basándose únicamente en los requisitos y especificaciones. Se centra en el "qué" hace el sistema, no en el "cómo". Ejemplo: Probar una pantalla de login introduciendo combinaciones de usuario y contraseña válidas e inválidas y verificando que el sistema responde según lo esperado (muestra un mensaje de error, redirige a la página principal, etc.).
- **Pruebas de Caja Blanca (White-Box Testing):** El probador tiene acceso al código fuente y diseña las pruebas para verificar las diferentes rutas lógicas, bucles y condiciones dentro del código. Se centra en la cobertura del código. Ejemplo: Para un método con una estructura if-else, se deben diseñar al menos dos casos de prueba: uno que haga que la condición del if sea verdadera y otro que la haga falsa, para asegurar que ambos bloques de código se ejecutan (Pressman & Maxim, 2021).
- **Pruebas de Regresión (Regression Testing):** Cada vez que se modifica el código (para corregir un bug o añadir una nueva funcionalidad), existe el riesgo de introducir nuevos defectos en funcionalidades que antes operaban correctamente. Las pruebas de regresión consisten en volver a ejecutar un conjunto de pruebas existentes para asegurar que los cambios no han "roto" nada. La automatización es clave para que esta práctica sea viable en entornos de desarrollo ágil (Sommerville, 2011).

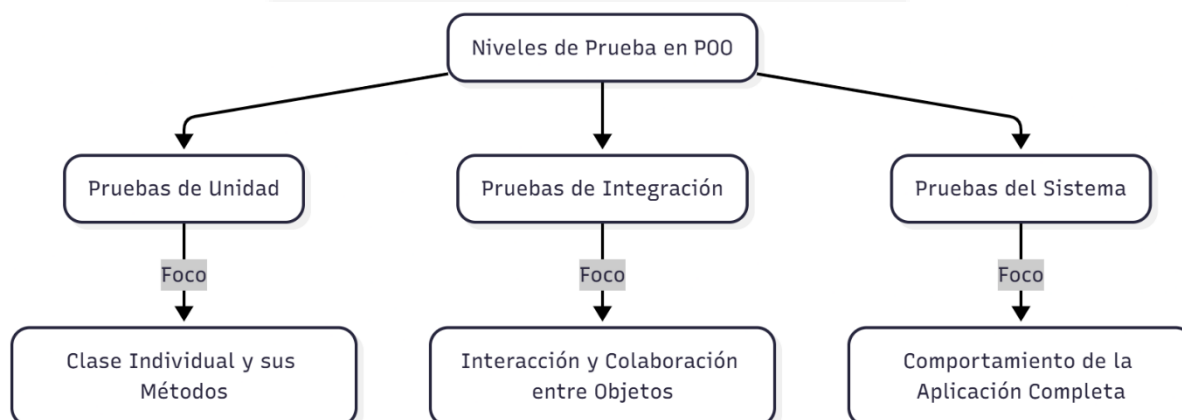


Figura 2: Mapa conceptual de los niveles de prueba en POO

Mapa conceptual que desglosa los niveles de prueba en POO (Unidad, Integración, Sistema) y su foco principal.

Este capítulo ha sentado las bases conceptuales. Entender estos fundamentos es indispensable antes de sumergirnos en la aplicación práctica de herramientas y técnicas, que exploraremos en el siguiente capítulo con JUnit.



Aplicaciones en la vida cotidiana

Una vez comprendidos los fundamentos teóricos, es hora de llevarlos a la práctica. La teoría de las pruebas de software es útil, pero su verdadero valor se materializa cuando se traduce en código que verifica activamente la calidad de otro código. Por años, los programadores novatos han recurrido a `System.out.println()` para verificar si sus métodos funcionan, una técnica manual, propensa a errores y completamente ineficiente. El desarrollo de software profesional exige un enfoque sistemático y automatizado. En el ecosistema de Java, la herramienta por excelencia para esta tarea es **JUnit**. No es solo una librería; es un framework que ha moldeado la forma en que los desarrolladores de Java piensan sobre la calidad y la verificación de su trabajo. Aprender a usar JUnit es una habilidad no negociable para cualquier programador de Java que aspire a la excelencia profesional (Deitel & Deitel, 2020).

2.1. JUnit: El Estándar para Pruebas Unitarias en Java

JUnit es parte de la familia de frameworks de prueba xUnit, que establecieron el estándar para las pruebas unitarias automatizadas en múltiples lenguajes de programación. Su éxito radica en su simplicidad y en su poderosa integración con los Entornos de Desarrollo Integrado (IDE) como NetBeans, lo que permite ejecutar cientos de pruebas con un solo clic.

La estructura de una prueba en JUnit 5 (la versión moderna) se basa en dos componentes principales: las **anotaciones** y las **aserciones**.

- **Anotaciones:** Son metadatos, marcadores que se anteponen a los métodos para indicar a JUnit cómo deben ser tratados. Las más comunes son:
 - **@Test:** Marca un método como un método de prueba ejecutable. Es la anotación fundamental; sin ella, el método es simplemente ignorado por el motor de pruebas.
 - **@BeforeEach:** Indica que un método debe ser ejecutado *antes* de cada método **@Test** en la misma clase. Es ideal para inicializar objetos y restablecer el estado del entorno, garantizando que cada prueba se ejecute de forma aislada y no se vea afectada por el resultado de la anterior.
 - **@AfterEach:** Indica que un método debe ser ejecutado *después* de cada método **@Test**. Se usa para tareas de limpieza, como cerrar conexiones a bases de datos o eliminar archivos temporales.
 - **@DisplayName("Un nombre claro y descriptivo para la prueba"):** Permite asignar un nombre legible en lenguaje natural a la prueba. Esto es increíblemente útil, ya que los reportes de prueba se vuelven mucho más fáciles de entender para todo el equipo.
 - **@Disabled:** Permite desactivar temporalmente un método de prueba o una clase de prueba. Es útil cuando una funcionalidad está en proceso de cambio y las pruebas fallan temporalmente.

- Aserciones (Assertions):** Son métodos estáticos, usualmente importados de la clase `org.junit.jupiter.api.Assertions`, que permiten verificar si una condición se cumple. Si la condición no se cumple, la aserción lanza una excepción (`AssertionFailedError`) y la prueba falla. El conjunto de aserciones de JUnit es una caja de herramientas muy completa:
 - `assertEquals(expected, actual)`: Verifica que dos valores son iguales. Es la aserción más común.
 - `assertTrue(condition) / assertFalse(condition)`: Verifica que una condición booleana es verdadera o falsa.
 - `assertNotNull(object) / assertNull(object)`: Verifica que una referencia a un objeto no es nula o sí lo es.
 - `assertThrows(ExpectedException.class, () -> { ... })`: Una aserción muy poderosa que verifica que un bloque de código lanza una excepción del tipo esperado. Es fundamental para probar el manejo de errores y validaciones.
 - `assertAll(...)`: Permite agrupar varias aserciones. Todas las aserciones se ejecutan, y todos los fallos se reportan juntos, a diferencia de las aserciones individuales que detienen la prueba en el primer fallo.

2.2. Caso Práctico Cotidiano: "Probando un Carrito de Compras Online"

Imaginemos que estamos desarrollando una pequeña plataforma de e-commerce para una tienda de café especial en Salento, Quindío. Dos de las clases más importantes son `Producto` y `Carrito`. La clase `Carrito` tendrá la lógica para añadir productos, calcular el total y gestionar su contenido. ¿Cómo nos aseguramos de que esta lógica funciona perfectamente bajo todas las circunstancias? Escribiendo pruebas unitarias exhaustivas.

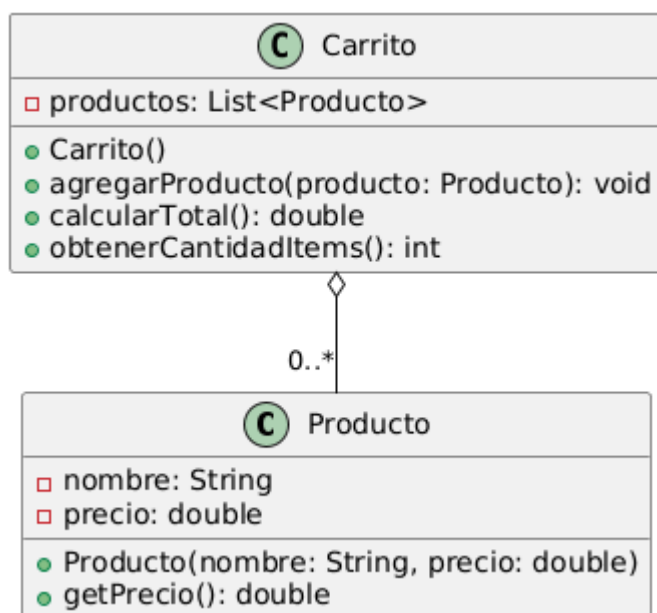


Figura 3: Diagrama de clases UML para el caso del "Carrito de Compras"

Diagrama UML que muestra una clase Carrito que contiene una lista de objetos de la clase Producto.

Código de las Clases a Probar (Java):

Java

// Archivo: Producto.java

package com.tiendacafe.modelo;

```
public class Producto {  
    private String nombre;  
    private double precio;  
  
    public Producto(String nombre, double precio) {  
        // Regla de negocio: Un producto no puede tener precio negativo.  
        if (precio < 0) {  
            throw new IllegalArgumentException("El precio no puede ser negativo.");  
        }  
        this.nombre = nombre;  
        this.precio = precio;  
    }  
  
    public double getPrecio() {  
        return precio;  
    }  
}
```

// Archivo: Carrito.java

package com.tiendacafe.modelo;

import java.util.ArrayList;

import java.util.List;

```
public class Carrito {  
    private final List<Producto> productos = new ArrayList<>();  
  
    public void agregarProducto(Producto producto) {  
        // Regla de negocio: No se pueden agregar productos nulos al carrito.  
        if (producto == null) {  
            throw new IllegalArgumentException("No se puede agregar un producto  
nulo.");  
        }  
        productos.add(producto);  
    }  
}
```

```
public int obtenerCantidadItems() {  
    return productos.size();  
}  
  
public double calcularTotal() {  
    return productos.stream().mapToDouble(Producto::getPrecio).sum();  
}  
}
```

Guía Práctica Paso a Paso: Configuración de JUnit en NetBeans

1. **Crear el Proyecto:** En NetBeans, ve a File > New Project.... Selecciona Java with Ant y Java Application. Nombra el proyecto "TiendaCafe".
2. **Crear las Clases:** Dentro del paquete Source Packages, crea un nuevo paquete llamado com.tiendacafe.modelo. Dentro de este, crea las clases Producto.java y Carrito.java y pega el código anterior.
3. **Crear la Clase de Prueba:**
 - En el explorador de proyectos, haz clic derecho sobre la clase Carrito.java.
 - Ve a Tools > Create/Update Tests.
 - En el diálogo que aparece, asegúrate de que el framework seleccionado es **JUnit 5**. NetBeans te preguntará si quieres añadir la librería de JUnit al proyecto. Haz clic en OK.
 - NetBeans creará automáticamente una clase CarritoTest.java en el paquete Test Packages, dentro de una estructura de paquetes paralela (com.tiendacafe.modelo).
4. **Escribir y Ejecutar las Pruebas:** Abre CarritoTest.java y reemplaza su contenido con el siguiente código. Este código no solo prueba el "camino feliz", sino también las validaciones y casos borde.

Código de la Clase de Prueba (Ejemplo Robusto en Java con JUnit 5):

Java

```
package com.tiendacafe.modelo;  
  
import org.junit.jupiter.api.*;  
import static org.junit.jupiter.api.Assertions.*;  
  
@DisplayName("Pruebas para la clase Carrito de Compras")  
public class CarritoTest {  
  
    private Carrito carrito;
```

@BeforeEach

```
void setUp() {
    // Se crea un nuevo carrito antes de cada prueba para asegurar el aislamiento
    carrito = new Carrito();
}
```

@Test

@DisplayName("Un carrito nuevo debe estar vacío y su total debe ser cero")

```
void testEstadoInicialDelCarrito() {
    assertAll(
        () -> assertEquals(0, carrito.obtenerCantidadItems(), "Un carrito nuevo no debería tener items."),
        () -> assertEquals(0.0, carrito.calcularTotal(), "El total de un carrito nuevo debería ser cero.")
    );
}
```

@Test

@DisplayName("Debe agregar un producto y actualizar la cantidad y el total")

```
void testAgregarUnProducto() {
    Producto producto = new Producto("Café Geisha", 45000.0);
    carrito.agregarProducto(producto);
    assertEquals(1, carrito.obtenerCantidadItems(), "El carrito debería tener 1 item después de añadir.");
    assertEquals(45000.0, carrito.calcularTotal(), "El total debe ser el precio del único producto.");
}
```

@Test

@DisplayName("Debe calcular el total correctamente con múltiples productos")

```
void testCalcularTotalMultiplesProductos() {
    carrito.agregarProducto(new Producto("Café Geisha", 45000.0));
    carrito.agregarProducto(new Producto("Bolsa de Panela Orgánica", 8000.0));
    assertEquals(2, carrito.obtenerCantidadItems());
    assertEquals(53000.0, carrito.calcularTotal(), "El total debería ser la suma de los precios de los productos.");
}
```

@Test

@DisplayName("Debe lanzar una excepción si se intenta agregar un producto nulo")

```
void testAgregarProductoNuloLanzaExcepcion() {
    // Verificamos que al ejecutar el método agregarProducto(null),
    // se lanza una excepción de tipo IllegalArgumentException.
}
```

```

    assertThrows(IllegalArgumentException.class, () -> {
        carrito.agregarProducto(null);
    });
}
}

```

5. Interpretar los Resultados:

- Haz clic derecho en el archivo CarritoTest.java y selecciona Test File (o Shift+F6).
- NetBeans abrirá la ventana Test Results. Si todas las pruebas pasan, verás una **barra verde** y un mensaje de éxito. Esto te da una alta confianza de que tu clase Carrito se comporta como esperas.
- Si una prueba falla, verás una **barra roja**. La ventana te mostrará exactamente qué prueba falló y la razón (p. ej., AssertionError: expected: <53000.0> but was: <52000.0>). Esta información es vital para depurar el error rápidamente.

Casos de estudio

Escribir pruebas unitarias es fundamental para construir una red de seguridad que nos alerte cuando algo se rompe. Sin embargo, cuando una prueba falla o, peor aún, un usuario reporta un comportamiento inesperado en producción, las pruebas solo nos dicen el "qué" (qué falló), pero no el "porqué". Descubrir la causa raíz de un defecto es el trabajo del **depurador** (*debugger*). La depuración es una de las habilidades más críticas y que más tiempo consume en el ciclo de vida del desarrollo. Un enfoque metódico puede reducir drásticamente el tiempo necesario para encontrar y solucionar un *bug*, diferenciando a un programador novato de uno profesional. El objetivo de la depuración es siempre encontrar la causa del error, no solo sus síntomas (Pressman & Maxim, 2021).

Dominar el depurador de un IDE como NetBeans es como tener superpoderes: permite pausar la ejecución del programa en cualquier punto, viajar en el tiempo a través de la pila de llamadas y espiar el estado interno de los objetos en memoria, algo imposible de hacer solo leyendo el código estático.

3.1. Caso Ficticio: "Depurando un Sistema de Gestión de Turnos para un Centro Médico en Colombia"

Una IPS (Institución Prestadora de Salud) en Bogotá utiliza una aplicación Java para gestionar las citas de sus pacientes. La aplicación ha funcionado bien, pero recientemente, después de una actualización para manejar los días festivos oficiales de Colombia, el personal de recepción ha reportado un problema grave.

El Desafío (El Bug): El sistema parece estar asignando turnos a los pacientes en días festivos oficiales, lo que causa confusión, sobrecarga de trabajo en los días siguientes y una mala experiencia para los pacientes que llegan al centro médico y lo encuentran cerrado. El defecto es intermitente; no ocurre con todos los festivos, lo que lo hace aún más difícil de rastrear. El equipo de desarrollo necesita encontrar la causa raíz y solucionarla con urgencia.

El Código Problemático (Fragmento): Dentro de una clase `GestorDeCitas.java`, existe un método que valida si una fecha es un día hábil. La lógica para los festivos se ha añadido recientemente.

Java

```
public class GestorDeCitas {  
  
    private List<java.util.Date> diasFestivos; // ¡Ojo! Usando la clase Date antigua.  
  
    public GestorDeCitas() {  
        // En el constructor se carga una lista de festivos.  
        this.diasFestivos = cargarFestivosDesdeServicio();  
    }  
}
```

```

    }

    public boolean esDiaHabil(java.time.LocalDate fecha) {
        // Lógica para fines de semana (sábado y domingo)
        DayOfWeek dia = fecha.getDayOfWeek();
        if (dia == DayOfWeek.SATURDAY || dia == DayOfWeek.SUNDAY) {
            return false;
        }

        // Nueva lógica para festivos
        for (java.util.Date festivo : diasFestivos) {
            // Intento de comparación... ¿será correcto?
            if (festivo.equals(fecha)) {
                return false;
            }
        }

        return true;
    }
    // ... otros métodos
}

```

Solución Aplicada: Estrategias de Depuración Sistemática en NetBeans

En lugar de añadir líneas de `System.out.println()` por todo el código —una técnica primitiva y poco eficiente—, el desarrollador a cargo decide usar el depurador de NetBeans de forma metódica.

Paso 1: Reproducir el *Bug* de Forma Fiable

El primer y más importante paso es aislar el problema. Intentar reproducir el *bug* manualmente a través de la interfaz gráfica es lento e inconsistente. La práctica profesional es escribir una prueba unitaria que falle específicamente por este motivo.

Java

@Test

@DisplayName("El 7 de agosto (Batalla de Boyacá) no debe ser un día hábil")

void testNoAsignarCitaEnFestivo() {

 GestorDeCitas gestor = new GestorDeCitas();

 LocalDate fechaFestiva = LocalDate.of(2024, 8, 7);

 // La aserción que esperamos que sea verdad (y que fallará)

 assertFalse(gestor.esDiaHabil(fechaFestiva), "El 7 de agosto debería ser reconocido como no hábil.");

}

Al ejecutar esta prueba, falla. La aserción esperaba false, pero el método esDiaHabil devolvió true. Ahora tenemos un escenario de prueba rápido y repetible que expone el defecto.

Paso 2: Uso Estratégico de Puntos de Ruptura (Breakpoints)

El desarrollador coloca un **punto de ruptura** en la primera línea del bucle for dentro del método esDiaHabil. Un punto de ruptura es una señal que le dice al depurador que pause la ejecución del programa justo en esa línea.

- **Cómo:** Se hace clic en el margen izquierdo del editor de código en NetBeans, junto al número de línea. Aparecerá un icono de un cuadrado rojo.
- **Ejecución:** En lugar de "Run File", se selecciona **"Debug Test File"** (o se presiona Ctrl+Shift+F5).
-

La ejecución de la prueba se inicia y se detiene exactamente donde se colocó el *breakpoint*. El IDE NetBeans cambia a la perspectiva de "Debug", mostrando varias ventanas nuevas y cruciales.

Paso 3: Inspeccionar Variables y el Estado de los Objetos

Con la ejecución pausada, la ventana **"Variables"** es nuestra principal herramienta de investigación. Muestra todas las variables locales (fecha, festivo) y los campos del objeto (this.diasFestivos) y sus valores actuales.

El desarrollador expande this.diasFestivos y ve una lista de objetos java.util.Date.

Luego, observa la variable local fecha y ve que es de tipo java.time.LocalDate.

Inmediatamente, formula una hipótesis: **"El problema es una incompatibilidad de tipos. Estoy comparando un objeto Date antiguo con un objeto LocalDate moderno, y su método .equals() nunca devolverá true"**.

Paso 4: Ejecución Paso a Paso (Stepping) para Confirmar la Hipótesis

Para confirmar la hipótesis, el desarrollador utiliza los controles de ejecución paso a paso:

- **Step Over (F8):** Ejecuta la línea actual y pasa a la siguiente. Lo usa para avanzar en cada iteración del bucle.
- **Step Into (F7):** Si la línea actual es una llamada a otro método, el depurador salta *dentro* de ese método. Es útil para explorar el código de otras clases.

El desarrollador presiona F8 repetidamente. Observa en la ventana de "Variables" cómo la variable festivo cambia en cada iteración. Cuando llega al festivo del 7 de agosto, se detiene en la línea if (festivo.equals(fecha)). Presiona F8 una vez más y ve que la ejecución salta la instrucción return false, confirmando que la condición equals fue falsa. La hipótesis está confirmada.

Paso 5: Analizar la Pila de Llamadas (Call Stack)

La ventana **"Call Stack"** muestra la secuencia de llamadas a métodos que llevaron al punto actual de ejecución (ej. CarritoTest.testNoAsignarCitaEnFestivo()) llamó a

GestorDeCitas.esDiaHabil()). Esto es invaluable en sistemas complejos para entender cómo se llegó a un estado de error.

Análisis Profundo

El *bug* técnico (comparación de tipos de fecha incorrectos) es un síntoma de un problema de diseño más profundo. La clase GestorDeCitas ha acumulado demasiadas responsabilidades, violando el **Principio de Responsabilidad Única (SRP)**. Este principio de diseño de software establece que un módulo o clase debe tener una y solo una razón para cambiar (Sommerville, 2011).

En nuestro caso, GestorDeCitas tiene al menos dos razones para cambiar:

1. Si cambian las reglas de negocio para agendar citas.
2. Si cambia la forma en que se obtienen o calculan los días festivos.

Una arquitectura de software robusta busca una alta **cohesión** (agrupar funcionalidades relacionadas) y un bajo **acoplamiento** (minimizar las dependencias entre módulos). La solución a largo plazo no es solo corregir la comparación de fechas, sino **refactorizar** el código, creando una nueva clase, por ejemplo, CalendarioFestivoColombiano, con la única responsabilidad de manejar la lógica de los días festivos. Esto haría el sistema más modular, fácil de entender y, sobre todo, más fácil de probar de forma aislada.

Impacto de la Solución

Al corregir la comparación de fechas (convirtiendo un tipo a otro para la comparación) y estandarizar el uso de java.time.LocalDate en todo el módulo, el desarrollador logró que la prueba pasara. Se implementó un parche de emergencia esa misma tarde.

- **Impacto Cuantificable:** Se estimó que cada cita mal asignada generaba entre 15 y 20 minutos de trabajo administrativo adicional (reprogramación, llamadas al paciente). Con un promedio de 10 citas mal asignadas por festivo, la solución **ahorró aproximadamente 3 horas de trabajo administrativo por cada día festivo** y mejoró significativamente la satisfacción del paciente.
- **Reducción del Tiempo de Resolución:** El uso del depurador permitió diagnosticar el problema en menos de 30 minutos, en comparación con las horas que podría haber tomado el método de "prueba y error" con System.out.println (Deitel & Deitel, 2020).

Herramientas y técnicas

Hasta ahora, hemos escrito y ejecutado pruebas de forma manual desde nuestro IDE. Esto es perfecto para verificar componentes individuales mientras los desarrollamos, pero en un proyecto profesional, con cientos o miles de clases, ejecutar cada prueba a mano es inviable y propenso a olvidos. El verdadero poder de las pruebas se desata a través de la **automatización**. La automatización nos permite integrar las pruebas en el proceso de construcción del software, garantizando que cada cambio, por pequeño que sea, sea verificado de forma sistemática. En las metodologías ágiles, donde los ciclos de desarrollo son cortos y las entregas frecuentes, las pruebas automatizadas son esenciales para proporcionar una retroalimentación rápida y mantener la calidad a lo largo del tiempo (Pressman & Maxim, 2021).

4.1. Más Allá de la Prueba Manual: El Desarrollo Guiado por Pruebas (TDD)

Una de las disciplinas más importantes que fomenta la automatización y un diseño de alta calidad es el **Desarrollo Guiado por Pruebas (Test-Driven Development - TDD)**. TDD invierte el proceso de desarrollo tradicional. En lugar de escribir el código de producción primero y las pruebas después (si hay tiempo), TDD sigue un ciclo corto y repetitivo conocido como **Rojo-Verde-Refactor**.

1. **Fase ROJA: Escribir una prueba que falla.** El ciclo comienza escribiendo una prueba unitaria para una funcionalidad que *aún no existe*. Como no hay código que la satisfaga, la prueba debe fallar al ejecutarse (la barra del IDE se muestra en rojo). Este paso es crucial porque nos obliga a pensar primero en los requisitos y en la interfaz de nuestro código: ¿Cómo se llamará el método? ¿Qué parámetros recibirá? ¿Qué debería devolver? La prueba actúa como la primera especificación ejecutable de nuestro diseño.
2. **Fase VERDE: Escribir el código mínimo para pasar la prueba.** El objetivo ahora es hacer que la barra de pruebas se ponga en verde lo más rápido posible. Se escribe la cantidad **mínima** de código de producción necesaria para que la prueba que acabamos de escribir pase. En esta fase, no buscamos la solución más elegante o eficiente, sino una que funcione y cumpla con el contrato definido por la prueba. A veces, esto implica incluso devolver un valor constante ("hardcodear") solo para pasar la prueba y continuar el ciclo.
3. **Fase REFACTOR: Mejorar el código.** Con la seguridad de que la prueba está pasando, ahora podemos **mejorar y limpiar** el código que acabamos de escribir. Esto puede implicar eliminar duplicación, mejorar nombres de variables, simplificar la lógica o extraer métodos. Durante esta fase, ejecutamos continuamente la suite de pruebas para asegurar que nuestras mejoras no han roto la funcionalidad existente. La red de seguridad de pruebas nos da la confianza para refactorizar sin miedo.

Este ciclo, que puede durar solo unos minutos, se repite para cada pequeña pieza de funcionalidad. El resultado es un código robusto, completamente cubierto por pruebas y con un diseño limpio y mantenible (Pressman & Maxim, 2021).

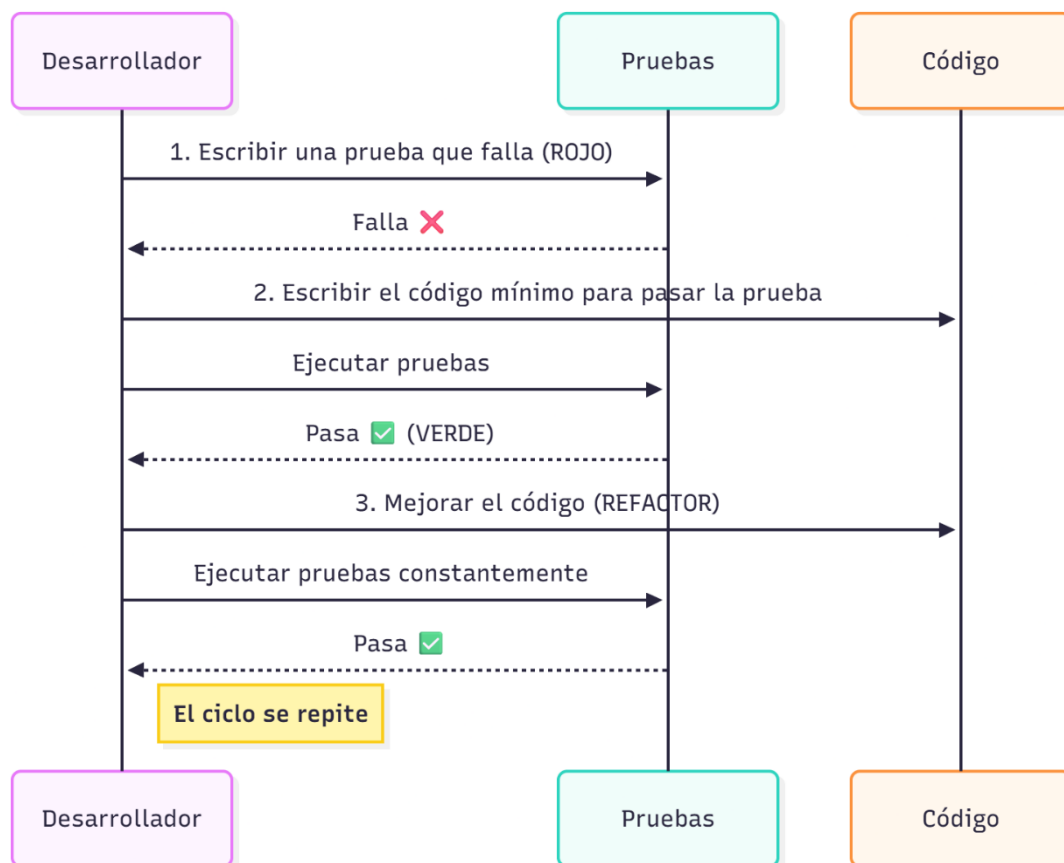


Figura 4: Diagrama de secuencia del ciclo TDD (Rojo-Verde-Refactor)

Diagrama que muestra los tres pasos del ciclo TDD: escribir una prueba fallida, escribir código para que pase y refactorizar el código, manteniendo las pruebas en verde.

4.2. Ejercicio Práctico: Refactorización Segura usando Pruebas de Regresión

La **refactorización** es el proceso de reestructurar el código existente, alterando su estructura interna sin cambiar su comportamiento externo. El objetivo es mejorar atributos no funcionales del software, como la legibilidad, la complejidad y la mantenibilidad. Sin una suite de pruebas automatizadas, la refactorización es una actividad de alto riesgo (Sommerville, 2011).

Imaginemos que en nuestro proyecto de tienda online tenemos un método que calcula un descuento, pero fue escrito de forma rápida y es difícil de leer y propenso a errores.

El Código Original (difícil de mantener):

Java

```
public class CalculadoraDescuento {  
    // Devuelve el % de descuento como un double (ej. 0.1 para 10%)  
    public double calcular(double totalCompra, String tipoCliente, int  
    aniosAntiguedad) {  
        if (tipoCliente.equals("VIP")) {  
            if (totalCompra > 100000) return 0.20;  
            else return 0.15;  
        } else if (tipoCliente.equals("REGULAR")) {  
            if (aniosAntiguedad > 5) return 0.10;  
            if (totalCompra > 200000) return 0.05;  
        }  
        return 0.0;  
    }  
}
```

Paso 1: Crear la Red de Seguridad (Pruebas de Regresión)

Antes de tocar una sola línea de ese método, escribimos una suite de pruebas que cubra todos los casos lógicos que conocemos.

Java

```
class CalculadoraDescuentoTest {  
    private CalculadoraDescuento calc = new CalculadoraDescuento();  
    @Test void testClienteVipConCompraAlta() { assertEquals(0.20,  
    calc.calcular(150000, "VIP", 2)); }  
    @Test void testClienteVipConCompraBaja() { assertEquals(0.15,  
    calc.calcular(50000, "VIP", 2)); }  
    @Test void testClienteRegularAntiguo() { assertEquals(0.10, calc.calcular(50000,  
    "REGULAR", 6)); }  
    @Test void testClienteRegularConCompraAlta() { assertEquals(0.05,  
    calc.calcular(250000, "REGULAR", 2)); }  
    @Test void testClienteSinDescuento() { assertEquals(0.0, calc.calcular(50000,  
    "NUEVO", 1)); }  
}
```

Ejecutamos las pruebas y confirmamos que todas pasan. Ahora tenemos una red de seguridad.

Paso 2: Refactorizar el Código para Mejorar la Claridad

Ahora mejoramos el método original, usando constantes para los "números mágicos" y simplificando la lógica anidada.

El Código Refactorizado (limpio y legible):

Java

```
public class CalculadoraDescuento {  
    private static final double DESC_VIP_ALTO = 0.20;  
    private static final double DESC_VIP_BAJO = 0.15;  
    private static final double DESC_REGULAR_ANTIGUEDAD = 0.10;  
    private static final double DESC_REGULAR_COMPRA = 0.05;  
  
    public double calcular(double totalCompra, String tipoCliente, int  
    aniosAntiguedad) {  
        if ("VIP".equals(tipoCliente)) {  
            return (totalCompra > 100000) ? DESC_VIP_ALTO : DESC_VIP_BAJO;  
        }  
        if ("REGULAR".equals(tipoCliente)) {  
            if (aniosAntiguedad > 5) return DESC_REGULAR_ANTIGUEDAD;  
            if (totalCompra > 200000) return DESC_REGULAR_COMPRA;  
        }  
        return 0.0;  
    }  
}
```

Paso 3: Volver a Ejecutar las Pruebas

Inmediatamente después de la refactorización, volvemos a ejecutar la suite de pruebas. Como todas las pruebas siguen pasando, tenemos una alta confianza de que nuestra mejora no introdujo ningún *bug*. Hemos mejorado la calidad interna del código sin alterar su comportamiento externo.

4.3. Conexión con Bases de Datos (SQL)

Un desafío común es probar métodos que dependen de una base de datos. Estas pruebas pueden ser lentas y frágiles. Probar contra una base de datos real introduce dependencias externas que hacen que las pruebas no sean deterministas. Dos técnicas profesionales para aislar nuestras pruebas de la base de datos son:

- **Uso de Dobles de Prueba (Mocks y Stubs):** Un "doble de prueba" es un objeto que reemplaza a un objeto real con fines de prueba.
 - **Stub:** Proporciona respuestas predefinidas a las llamadas realizadas durante la prueba. Por ejemplo, un `ClienteRepositoryStub` siempre devolvería el mismo objeto `Cliente` de prueba cuando se llama a su método `buscarPorId`.
 - **Mock:** Es un objeto que registra las llamadas que recibe. Permite verificar no solo el resultado de una operación, sino también las interacciones; por ejemplo, podemos verificar que el método `guardar`

fue llamado exactamente una vez. Librerías como **Mockito** facilitan enormemente la creación de estos objetos falsos en Java.

- **Bases de Datos en Memoria:** Para pruebas de integración, podemos usar una base de datos que se ejecuta completamente en la memoria RAM, como **H2**. Al inicio de las pruebas, se crea la base de datos, se insertan los datos de prueba y, al finalizar, se destruye. Esto es mucho más rápido y consistente que usar una base de datos de desarrollo compartida (Deitel & Deitel, 2020).



Conclusión

A lo largo de este recorrido, hemos transitado un camino fundamental en la formación de todo ingeniero de software: el que va de ser un simple constructor de código a convertirse en un **arquitecto de soluciones fiables**. Hemos desglosado la teoría detrás de la calidad, diferenciando los roles críticos de la verificación y la validación. Hemos aplicado pruebas unitarias para forjar una red de seguridad alrededor de nuestro código, aprendiendo a validar nuestros componentes con la precisión y la automatización que exige la industria moderna. Hemos aprendido a cazar defectos como detectives, usando las herramientas de un depurador no como una muleta, sino como un instrumento de análisis sistemático para entender el comportamiento de nuestro software a un nivel profundo. Finalmente, hemos explorado técnicas avanzadas como el Desarrollo Guiado por Pruebas (TDD), descubriendo que las pruebas no son una tarea posterior, sino una herramienta de diseño que nos guía hacia un código más limpio, robusto y mantenible.

El mensaje central es claro e inequívoco: en la Programación Orientada a Objetos, y en el desarrollo de software en general, **la calidad no es un accidente, es un diseño**. No es una fase que se añade al final del proyecto si queda tiempo, sino una disciplina rigurosa que se integra en cada línea de código que escribimos, en cada decisión de diseño que tomamos y en cada conversación que tenemos con nuestro equipo y nuestros clientes. Las pruebas unitarias no son un trabajo extra, son la documentación viva y ejecutable de nuestro sistema. La depuración no es una frustración, es un proceso científico de formular hipótesis y validarlas. Y la automatización no es un lujo, es la base para la entrega continua de valor en el mundo ágil de hoy.

Como futuros profesionales que desarrollarán software para empresas en Colombia y Latinoamérica, la responsabilidad que adquieren es inmensa. El código que escriban gestionará transacciones financieras, datos de salud, logística de transporte y mucho más. La fiabilidad, seguridad y eficiencia de ese código tendrá un impacto directo en la vida de las personas y en el éxito de las organizaciones. Adoptar las prácticas vistas en este documento no solo los hará mejores programadores, sino profesionales íntegros y confiables, capaces de construir tecnología que genere valor y confianza.

Te invitamos a llevar estas lecciones más allá de la asignatura. Aplica TDD en tu próximo proyecto personal, por pequeño que sea. Utiliza el depurador como tu primera herramienta ante un *bug*, no como el último recurso. Cuestiona el diseño del código y pregúntate siempre: "¿Cómo puedo probar esto?". Al hacerlo, estarás construyendo no solo mejor software, sino una mejor versión de ti mismo como ingeniero.

Para finalizar, te dejamos dos preguntas para tu reflexión personal:

1. Más allá de la corrección funcional, ¿qué rol juega la ética profesional al momento de decidir si un software con "pequeños errores conocidos" está listo para ser entregado a un cliente o usuario final? ¿Dónde se traza la línea entre "suficientemente bueno" y la responsabilidad profesional?
2. Pensando en las empresas tecnológicas más exitosas de la actualidad (como Google, Netflix o Amazon), ¿cómo crees que su cultura interna de calidad, pruebas automatizadas e integración continua ha contribuido a su capacidad para innovar rápidamente y dominar sus respectivos mercados?



Referencias.

Deitel, P., & Deitel, H. (2020). [*Java How to Program, Late Objects \(11th ed.\)*](#). Pearson.

Joyanes Aguilar, L., & Zahonero Martínez, I. (2020). [*Programación en Java: de Cero a Avanzado*](#). McGraw-Hill Interamericana.

Pressman, R. S., & Maxim, B. R. (2021). [*Ingeniería de software: un enfoque práctico \(9.ª ed.\)*](#). McGraw-Hill Interamericana.

Sommerville, I. (2011). [*Ingeniería de software \(9.ª ed.\)*](#). Pearson Educación.